Data Modelling

From first principles to the eighteen core patterns you will actually meet in a modern data warehouse, analysed one by one with worked examples, SQL, diagrams and decision guides.

Dimensional | Normalized | Data Vault | Wide and Flat | Temporal | Modern

OPENING

About this masterclass

Who it is for

Analysts, data scientists, analytics engineers and machine learning engineers who query warehouses every day and now want to design them with confidence.

What you need

Comfort reading basic SQL (SELECT, JOIN, GROUP BY). No prior modelling theory is assumed; every term is defined before it is used.

How it is built

Thirty foundation slides, then six families. Each of the eighteen patterns gets exactly ten slides using the same analytical lens.

One running example

A fictional retailer, Aster Retail, sells products to customers in stores and online. Every pattern models the same business so you can compare like with like.

OPENING

How to read these slides: the visual vocabulary

Definition box

A tinted panel with a solid border states a formal definition you should be able to repeat from memory.

Annotation callout

Amber notes with a thick left edge point at part of a diagram or chart and explain why it matters.

SQL panel

Dark panels hold DDL and queries. Code is portable ANSI style SQL unless a dialect is named.

Checkpoint

Dashed green frames pose a question. Pause and answer before reading the next slide.

1 Numbered markers

Copper discs pin a step in a process or a hotspot on a diagram; the matching note carries the same number.

Family colour band

The strip on the left edge of every slide tells you which of the six families you are in.

OPENING

By the final slide you will be able to

01 Explain what a data model is and separate conceptual, logical and physical models.

02 Identify entities, attributes, keys and cardinality, and draw them in crow's foot notation.

03 Normalise a messy table to third normal form and justify each step.

04 Declare the grain of a fact table and design star, snowflake and galaxy schemas.

05 Build a Data Vault layer of hubs, links and satellites with hash keys and load dates.

06 Choose between One Big Table, activity schema and nested arrays for columnar engines.

07 Implement slowly changing dimensions of Types 1, 2 and 3 and answer "as of" questions.

08 Combine medallion layers, a semantic layer and anchor modeling into one architecture.

OPENING

Roadmap: ten parts, 260 slides

Part	Module	What you study	Slides
0	Opening	Purpose, outcomes, the map	1 to 6
1	Foundations	Entities, keys, grain, normal forms, storage	7 to 36
2	Dimensional	Star, Snowflake, Galaxy	37 to 69
3	Normalized	3NF, ER Model, Inmon CIF	70 to 102
4	Data Vault	Hubs, Satellites, Links	103 to 135
5	Wide and Flat	One Big Table, Activity Schema, Nested	136 to 168
6	Temporal	SCD Type 1, Type 2, Type 3	169 to 201
7	Modern	Medallion, Semantic Layer, Anchor	202 to 234
8	Comparison	Trade offs, decision tree, architecture	235 to 246
9	Capstone	Project, assessment, glossary, reading	247 to 260

OPENING

The map: six families, eighteen patterns

Dimensional	Star Schema	Snowflake Schema	Galaxy or Constellation
Normalized	Third Normal Form	ER Model	Inmon CIF
Data Vault	Hubs	Satellites	Links
Wide and Flat	One Big Table	Activity Schema	Nested or Struct Array
Temporal	SCD Type 1	SCD Type 2	SCD Type 3
Modern	Medallion Layers	Semantic or Metrics Layer	Anchor Modeling

> **Read the map by row and by column.** Rows group patterns that solve the same problem. Columns are not ranked; the first pattern in each row is simply the most common entry point.

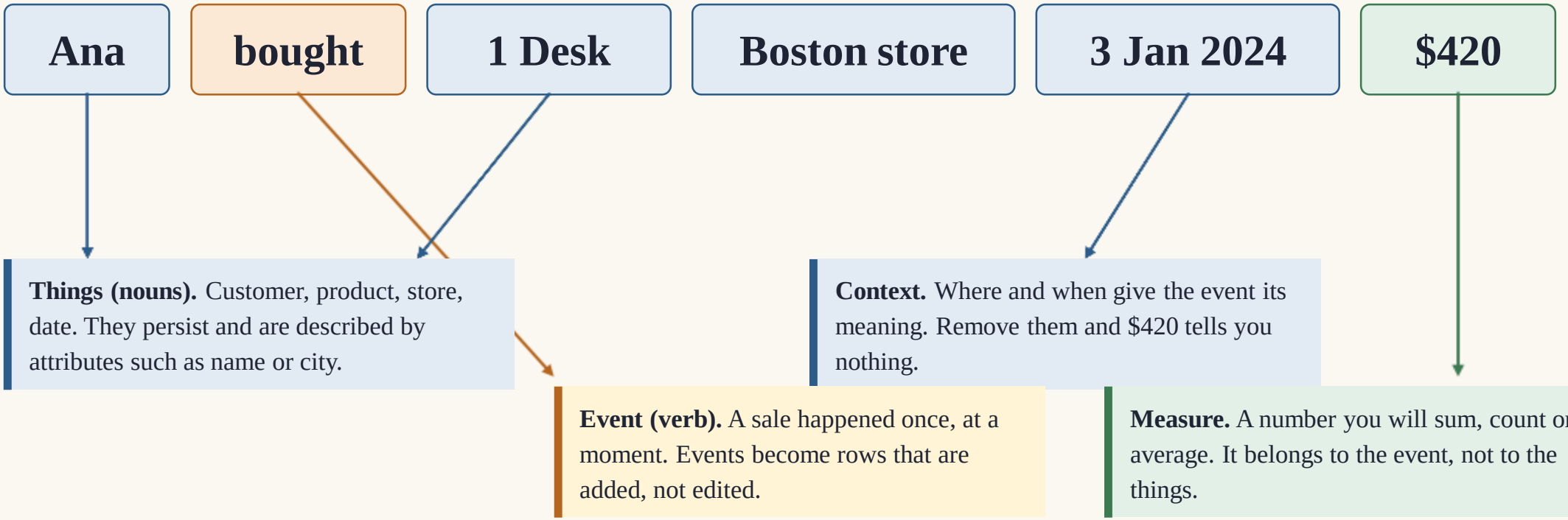
PART 1 OF 9

Foundations

Before any pattern makes sense, we need a shared vocabulary: entities, attributes, keys, relationships, grain, normal forms and the physical realities of storage.

FOUNDATIONS · DATA

Data is a record of things and the events between them



FOUNDATIONS · DEFINITION

What is a data model?

A **data model** is a deliberate, documented description of the data an organisation keeps: *what* the things and events are, *how* they relate, and *which rules* they must obey, expressed so that both people and databases can act on it.

Structure

Tables, columns, types and the keys that join them.

Meaning

Business definitions: what exactly counts as an "active customer".

Rules

Constraints: every order has exactly one customer; price is never negative.

Analogy. A model is to a database what an architect's drawing is to a building. You can build without one, but you cannot reason about load, change or safety without one.

FOUNDATIONS · MOTIVATION

Why model at all: four failures of unmodelled data

1 · Inconsistent numbers

Finance and marketing compute "revenue" differently and both are confident. Trust collapses.

2 · Slow, fragile queries

Every analyst rewrites the same twelve joins, each slightly differently, each scanning far too much data.



3 · Lost history

Sources overwrite values. When Ana moves city, last year's sales silently move with her.

4 · Integration pain

CRM calls her C1001, the web shop calls her ana@mail. Without a model nobody knows they are one person.

Three levels: conceptual, logical, physical

Conceptual · the business view

Entities and relationships in plain words. No columns, no types. "Customers place orders for products."

Audience: executives and domain experts. **Question it answers:** are we talking about the same things?

Logical · the design view

Attributes, keys, cardinality and normal forms. Independent of any database product.

Audience: data architects and analysts. **Question it answers:** is the design correct and complete?

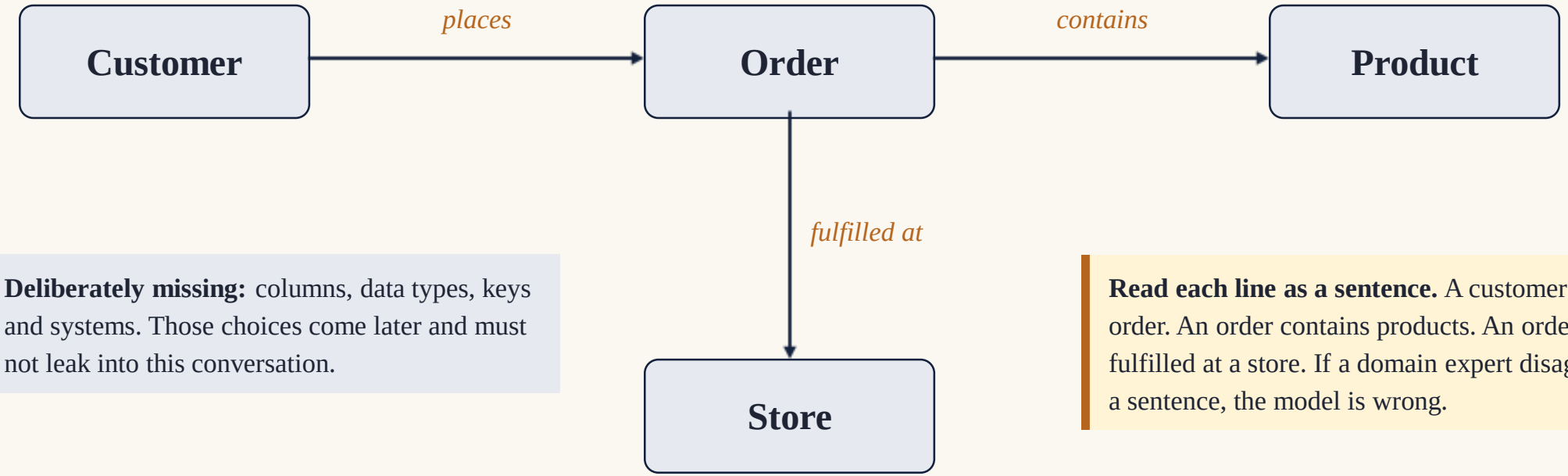
Physical · the engine view

Data types, partitions, clustering, indexes, file formats for one specific platform.

Audience: engineers and DBAs. **Question it answers:** will it be fast, cheap and reliable?

FOUNDATIONS · CONCEPTUAL LEVEL

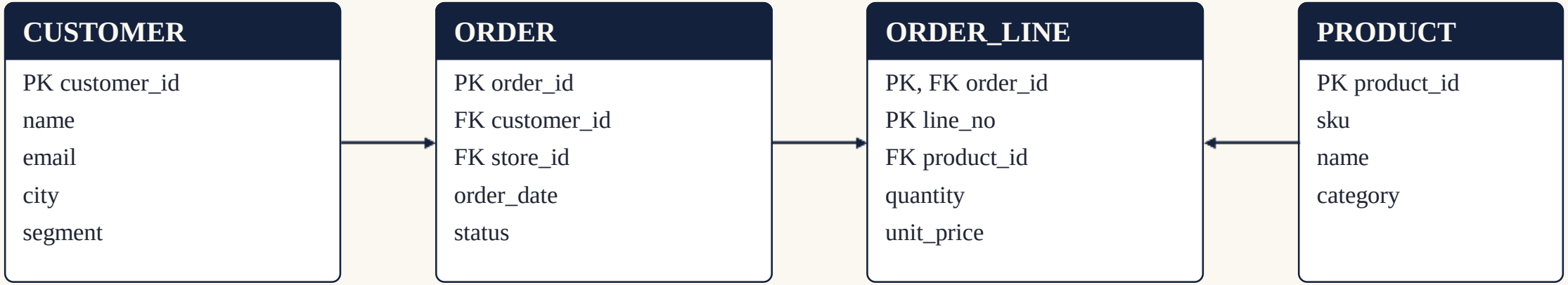
Aster Retail, conceptually: nouns joined by verbs



Deliberately missing: columns, data types, keys and systems. Those choices come later and must not leak into this conversation.

Read each line as a sentence. A customer places an order. An order contains products. An order is fulfilled at a store. If a domain expert disagrees with a sentence, the model is wrong.

The same business, logically: attributes and keys appear



PK, primary key: the attribute that uniquely identifies each row of an entity.

FK, foreign key: a copy of another entity's key that implements the relationship.

Order line appears because one order holds many products and one product sits in many orders.

Physically: one table, every engine decision made explicit

```
CREATE TABLE sales.order_header (
  order_id     BIGINT    NOT NULL,
  customer_id  BIGINT    NOT NULL,
  store_id     INTEGER   NOT NULL,
  order_date   DATE      NOT NULL,
  status       VARCHAR(12),
  CONSTRAINT pk_order PRIMARY KEY (order_id),
  CONSTRAINT fk_cust FOREIGN KEY (customer_id)
    REFERENCES sales.customer (customer_id)
)
PARTITION BY order_date
CLUSTER BY customer_id;
```

Data types are chosen for size and correctness. BIGINT leaves room for billions of orders; DATE avoids string parsing.

Constraints make logical rules enforceable. Many cloud warehouses accept but do not enforce them, so tests must.

Partitioning and clustering are purely physical. They change cost and speed, never meaning.

Entities are types; instances are the rows

Entity: a class of thing the business cares about and can tell apart, such as Customer. **Instance:** one member of that class, such as Ana.

Test for a real entity: can you count them, name them, and does the business keep facts about each one? "Weather" fails; "Store" passes.

Strong entities exist on their own (Customer). Weak entities only exist through another (Order Line needs Order).

CUSTOMER (the entity)

customer_id	name	city	segment
C1001	Ana	Boston	SMB
C1002	Raj	Seattle	ENT
C1003	Mila	Chicago	SMB

Each row is one **instance**. Each column is one **attribute**. The header row is the entity's shape.

Attributes, their kinds and their domains

Kind	Meaning	Aster example	Modelling consequence
Simple	One indivisible value	sku	One column
Composite	Made of meaningful parts	address	Split into street, city, postcode
Multi valued	Several values per instance	phone numbers	Child table or array column
Derived	Computed from others	age from birth date	Compute, do not store, unless for speed
Key	Identifies the instance	customer_id	Unique and not null
Optional	May be unknown	loyalty tier	Nullable, or an "Unknown" member

Domain: the set of legal values for an attribute. segment ∈ {SMB, ENT, CONSUMER}; quantity is a whole number of one or more. Domains become data types, check constraints and data tests.

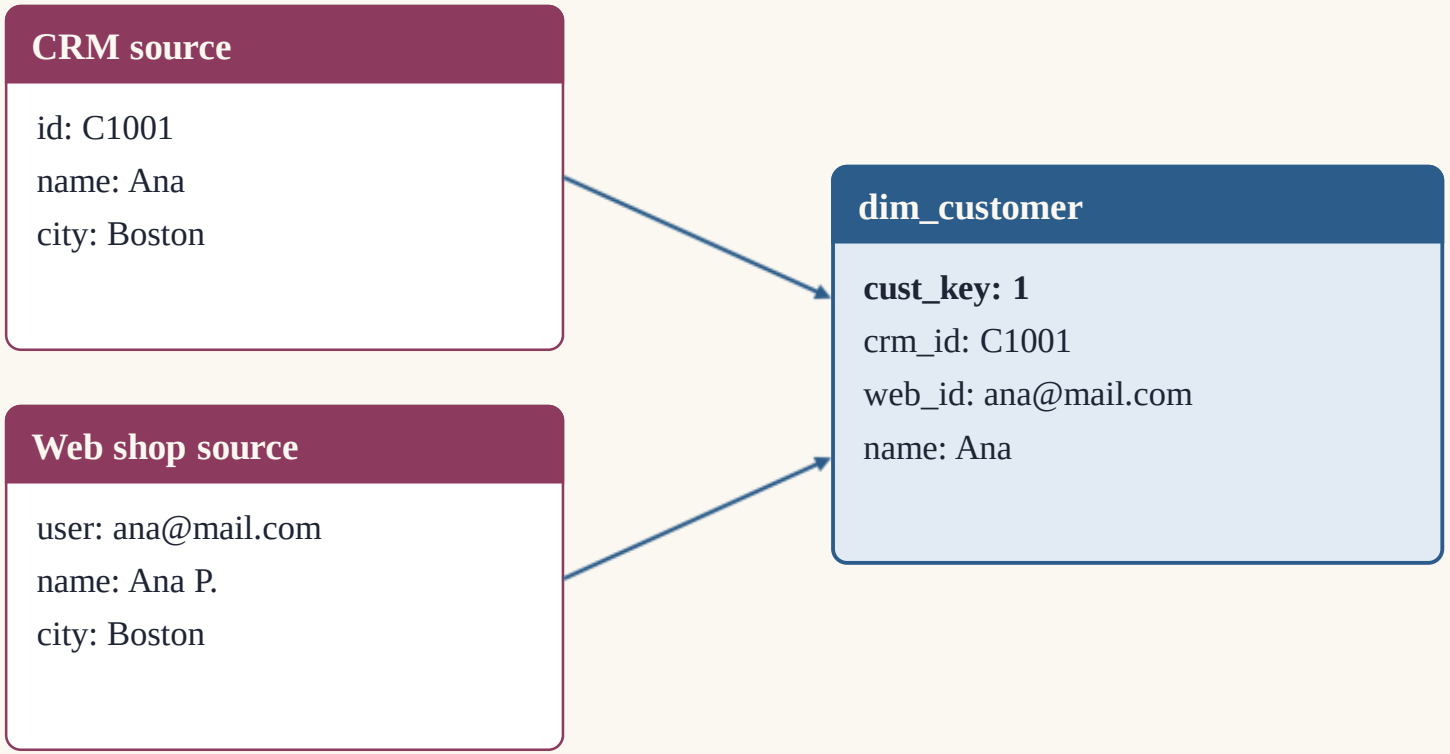
FOUNDATIONS · KEYS

A family of keys, each with a different job

Key	Definition	Aster example	Where it shines
Candidate	Any minimal set of attributes that is unique	customer_id; email	Design analysis
Primary	The candidate chosen to identify rows	customer_id	Every table
Natural / business	Meaningful in the real world, issued by a source	sku DESK1	Integration, vault hubs
Surrogate	Meaningless integer or hash created by the warehouse	cust_key 1	Dimensions, history
Composite	Two or more columns that are unique together	(order_id, line_no)	Line level tables
Foreign	A column that references another table's key	order.customer_id	Relationships
Hash	Deterministic digest of a business key	MD5('C1001')	Parallel loading

Minimal matters: (customer_id, name) is unique but not minimal, because customer_id alone already is. A key with spare columns is a superkey, not a candidate key.

Why warehouses add surrogate keys on top of natural keys



1 · Integration. Two systems, two natural keys, one person. The surrogate is the single identity.

2 · History. When Ana moves, Type 2 tracking needs two rows for one natural key; only a surrogate can tell them apart.

3 · Stability. Sources recycle or reformat ids. A warehouse key never changes meaning.

4 · Speed. Joining on a small integer is cheaper than on long strings.

Relationships and cardinality

1 : 1

One to one

Each store has exactly one manager contract. Often merged into one table, or split for security or optional data.

1 : N

One to many

One customer places many orders. The foreign key always lives on the many side: order.customer_id.

M : N

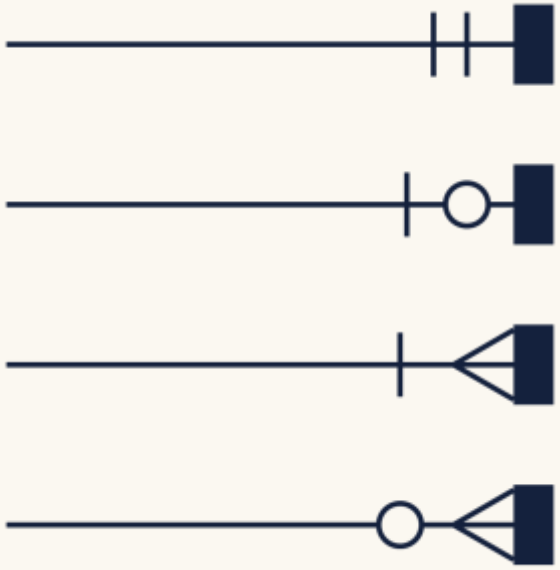
Many to many

Orders hold many products; products appear in many orders. Needs a third, associative table.

Cardinality is the maximum number of instances on each side.
Optionality (or modality) is the minimum: zero or one.

Ask both directions. "Can a customer exist with no orders?" Yes. "Can an order exist with no customer?" No. Those answers set the minimums.

Reading crow's foot notation



Exactly one · two bars

Zero or one · circle and bar

One or many · bar and crow's foot

Zero or many · circle and crow's foot

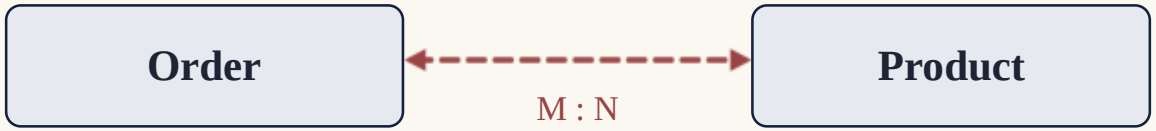
The symbol nearest the box is the maximum (cardinality). The symbol further away is the minimum (optionality).

The crow's foot looks like the toes of a bird: "many". A circle looks like a zero. A bar looks like a one.

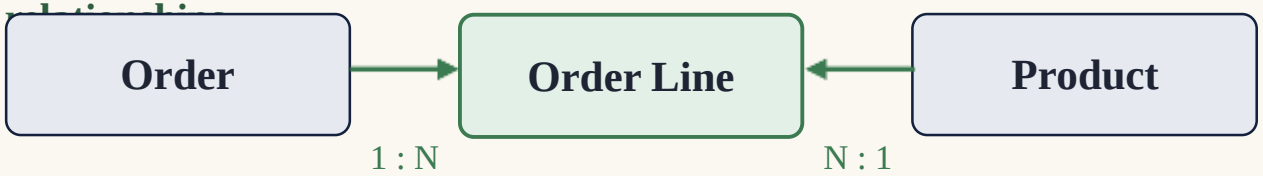
The dark block at each line end stands for the entity box the line attaches to.

Resolving many to many with an associative entity

Before: not storable



After: two 1 : N



Why "before" fails: a relational column holds one value. Neither table can hold a list of the other's keys without breaking first normal form.

The new entity carries its own facts: quantity and unit price belong to neither order nor product, but to the pairing.

Pattern preview: a Data Vault link and a dimensional bridge table are both descendants of this idea.

OLTP and OLAP: two workloads, two shapes of model

Dimension	OLTP · operational	OLAP · analytical
Purpose	Run the business	Understand the business
Typical query	Fetch or update one order	Sum revenue by month and region
Rows touched	A handful, by key	Millions, by scan
Writes	Constant small inserts and updates	Bulk loads, mostly appends
History	Current state only	Years of history, as of views
Favoured model	3NF, ER model	Dimensional, wide, vault plus marts
Storage	Row oriented	Column oriented

Core lesson: no single model is best. Normalisation protects writes; denormalisation speeds reads. Every family on the map is a position on that trade off.

FOUNDATIONS · DATA FLOW

Where models live along the data journey

Write path · normalised models

Read path · denormalised models



3NF and ER models sit here, protecting every insert and update the business makes.

Medallion layers organise this middle: bronze raw, silver cleaned, gold business ready.

Vaults, stars, OBTs and semantic layers live in the warehouse and serve the consumers.

FOUNDATIONS · GRAIN

Grain: the single most important modelling decision

Grain is a precise statement of what **one row** of a table represents. Declare it in words before choosing any column.

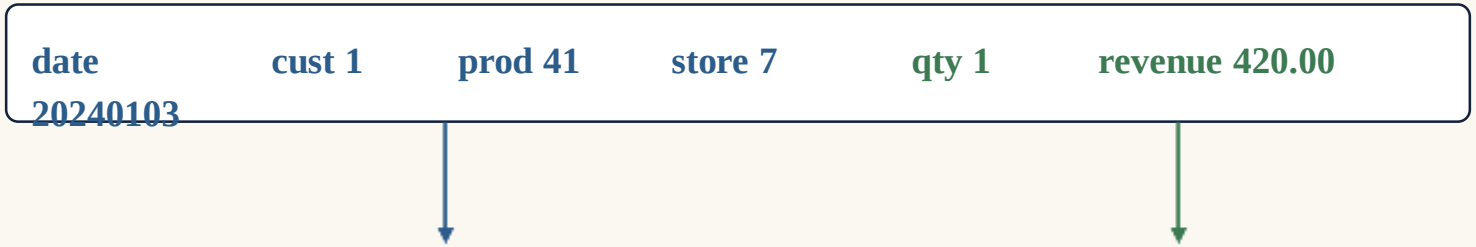
Fine grain	"One row per product per order line, as scanned at the till."
Medium grain	"One row per store per product per day."
Coarse grain	"One row per region per month."

Go atomic when you can. Fine grain can always be rolled up; coarse grain can never be drilled down.

Never mix grains in one table. Daily totals next to line items guarantee double counting.

Facts measure events; dimensions describe context

One fact row, dissected



Foreign keys to dimensions. They answer who, what, where and when. You filter and group by them.

Measures. Numbers produced by the event. You aggregate them.

Classic test: "Revenue by customer segment by month". Revenue is the fact; segment and month are dimension attributes.

Fact table

Tall and narrow. Grows with every event. Keys plus numbers.

Dimension table

Short and wide. Changes slowly. Rich descriptive text used for filters and labels.

Rule of thumb: if you would put it on a chart axis, it is a dimension attribute.

Additivity decides how a measure may be summed

Additive

Summable across every dimension.

Revenue, quantity sold, discount amount.

Semi additive

Summable across some dimensions, never across time.

Stock on hand, account balance, headcount.

Non additive

Never summable; recompute from parts.

Ratios, percentages, unit price, distinct counts.

Store · date	Desks in stock	Margin %
Boston · 1 Jan	10	40%
Boston · 2 Jan	8	20%
Naive SUM	18 (wrong)	60% (wrong)

Correct treatment: stock across days takes the last value (8) or the average (9). Margin is recomputed as total profit divided by total revenue, never by adding percentages.

FOUNDATIONS · NORMALISATION

The problem normalisation solves: update anomalies

order	cust	name	city	product	category	price
O1	C1001	Ana	Boston	Desk	Office	420
O2	C1001	Ana	Austin	Lamp	Home	60
O3	C1002	Raj	Seattle	Desk	Office	420

Update anomaly

Ana moved, but only row O2 was changed. The table now claims she lives in two cities.

Insert anomaly

A new product "Chair" cannot be recorded until someone orders it, because every row needs an order.

Delete anomaly

Cancel O3 and the warehouse forgets that Raj, a real customer, exists at all.

Normalisation is the disciplined removal of redundancy by splitting tables so that every fact is stored once, in the one place it depends on.

1NF: atomic values, no repeating groups

Violates 1NF

order	cust	products
O1	C1001	Desk, Lamp, Pad
O2	C1002	Desk

A list in one cell. You cannot count lamps, join to a product table, or index a single product.



In 1NF

order	line	cust	product
O1	1	C1001	Desk
O1	2	C1001	Lamp
O1	3	C1001	Pad
O2	1	C1002	Desk

One value per cell, a key of (order, line). Redundancy remains: cust repeats per line. 2NF fixes that next.

1NF rule: every column holds a single atomic value of one domain, every row is unique, and there are no repeating groups such as product1, product2, product3.

2NF and 3NF: depend on the key, the whole key, nothing but the key

2NF · the whole key

In order_line keyed by (order, line), **cust** depends on order alone. That is a *partial dependency*.

Fix: move cust to an ORDER table keyed by order. Lines keep only what depends on both parts.

ORDER(order, cust) · ORDER_LINE(order, line, product, qty)

3NF · nothing but the key

In ORDER(order, cust, cust_city), city depends on cust, which depends on order. That is a *transitive dependency*.

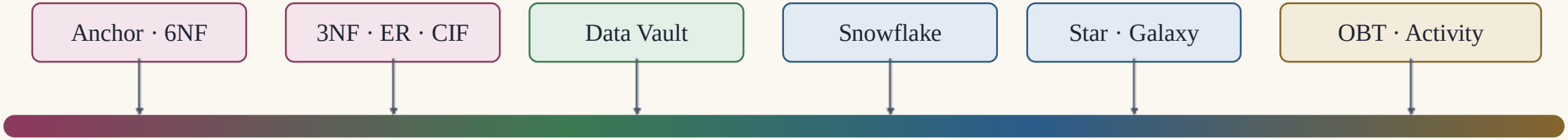
Fix: move city to CUSTOMER(cust, name, city). Now each fact lives once.

order → cust → city becomes two tables

Functional dependency A → B means: knowing A fixes exactly one value of B. Normal forms are rules about which dependencies a table may contain. Mnemonic attributed to William Kent: every non key attribute depends on *the key, the whole key, and nothing but the key*.

The normalisation spectrum: where each family sits

Nested arrays, SCD types, medallion and semantic layers are orthogonal: they can be applied at almost any point.



Fully normalised **Fully denormalised**

Left side wins on: storage efficiency, write integrity, flexibility to absorb new sources, auditability.

Right side wins on: query simplicity, read speed on columnar engines, ease for self service users.

Row stores and column stores read data differently

Row oriented · pages hold whole rows

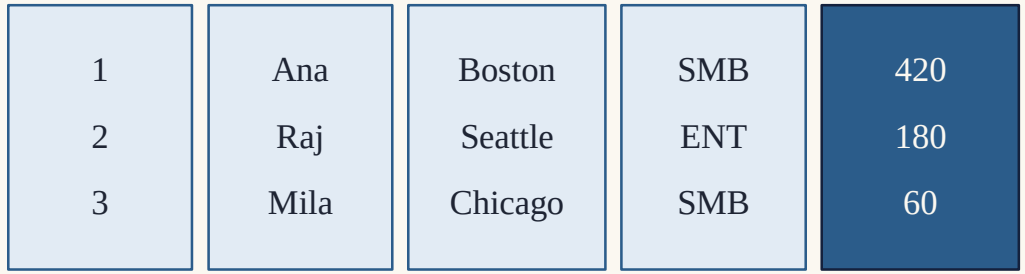
1 · Ana · Boston · SMB · 420

2 · Raj · Seattle · ENT · 180

3 · Mila · Chicago · SMB · 60

Fast for OLTP: "fetch Ana's full record" touches one page. **Slow for analytics:** SUM(revenue) must read every byte of every row.

Column oriented · files hold whole columns



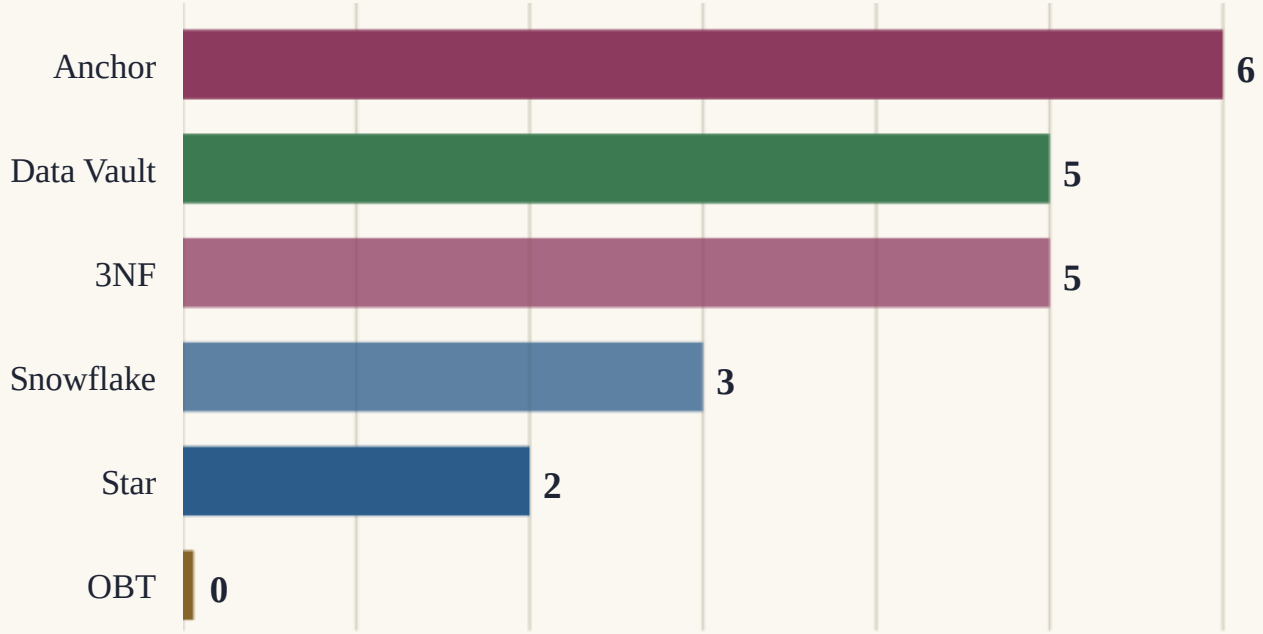
SUM(revenue) reads only the dark column. Repeated values like SMB compress superbly. Wide tables cost little if queries name few columns.

Modelling consequence: columnar engines such as BigQuery, Snowflake, Redshift and Databricks make wide, denormalised tables affordable. That is why OBT and nested patterns became popular.

FOUNDATIONS · COST OF JOINS

One question, six models: how many joins does it take?

Query: *revenue by product category by customer city* · joins required, counted by hand



Joins are not evil. Modern engines join well. But each join is a place where an analyst can choose the wrong key or fan out rows.

The pattern: integration models trade query joins for flexibility; presentation models trade storage for simplicity.

History: the dimension most models forget

Ana's home city over time



Question A: "What were Boston's sales in 2022?" Needs Ana counted as Boston, the value true *at the time*.

Question B: "Where do our current customers live?" Needs only today's value: Denver.

Question C: "Did customers who moved spend more?" Needs current and previous values side by side.

Preview: A is answered by SCD Type 2, B by SCD Type 1, C by SCD Type 3. Data Vault satellites and anchor modeling keep full history by construction.

FOUNDATIONS · WAREHOUSE LAYERS

A layered warehouse, and the patterns each layer uses

Consumption	Dashboards, notebooks, ML features, APIs · semantic layer
Presentation	Data marts for humans · star, snowflake, galaxy, OBT, SCD types
Integration	One version of the enterprise · 3NF EDW, Data Vault, anchor
Staging	Raw copies with load metadata · bronze, nested source shapes
Sources	CRM, ERP, web, events · ER models in OLTP systems

Read upward. Data gains structure and meaning as it rises. The medallion pattern renames these as bronze, silver and gold; the CIF names the integration layer the EDW.

FOUNDATIONS · RUNNING EXAMPLE

Aster Retail: the sources we will model eighteen ways

Source	Key tables	Business key	Change pattern
CRM	customer, address	customer_id C1001	Updates in place
ERP	product, category, brand, stock	sku DESK1	Occasional price changes
POS	order, order_line, store	order_id 1001	Append only events
Web	sessions, clicks, signups	email, anonymous id	High volume stream
HR	employee, store_manager	employee_no	Slow, audited

Customers: Ana (C1001), Raj (C1002), Mila (C1003).

Products: DESK1 $420, LAMP3 $30, PAD9 $15, CHAIR2 $180.

Stores: Boston, Austin, NYC, SFO and an online channel.

Checkpoint: test your foundations

Q1 · Grain

Write the grain statement for a table that stores daily stock levels per store per product.

Answer: one row per store per product per calendar day, measured at close of business.

Q2 · Additivity

Is "stock on hand" additive across stores? Across days?

Answer: yes across stores, no across days. It is semi additive.

Q3 · Normal forms

ORDER(order_id, cust_id, cust_email). Which normal form is broken?

Answer: 3NF, because email depends transitively on order through cust_id.

Q4 · Keys

Give two reasons a dimension uses a surrogate key instead of the CRM id.

Answer: it supports several rows per customer for history, and it unifies ids from several sources.

PART 2 OF 9 · FAMILY 1 OF 6

Dimensional Modelling

Facts surrounded by dimensions, designed for the way people actually ask questions: measure something, sliced by who, what, where and when.

Star Schema | Snowflake Schema | Galaxy or Constellation

The dimensional method: Kimball's four steps

1. **Select the business process.** A verb the business measures: selling, shipping, stocking.
2. **Declare the grain.** Exactly what one fact row means, in a single sentence.
3. **Identify dimensions.** Who, what, where, when, why and how describe each row.
4. **Identify facts.** The numeric measurements true at that grain.

The problem it solves: business users think in measures sliced by context. A dimensional model matches that mental model, so queries are short and predictable.

Shared building block: conformed dimensions, the same customer and date tables reused by every fact, tie the whole warehouse together.

Star, Snowflake and Galaxy side by side

Property	Star	Snowflake	Galaxy
Fact tables	One	One	Several
Dimensions	Denormalised, flat	Normalised into sub tables	Conformed, shared
Joins per query	Fewest	More	As star, plus drill across
Redundancy	Some, in dimensions	Lowest	Some, shared once
Ease of use	Highest	Moderate	High within each star
Typical scope	One business process	One process, large hierarchies	Enterprise, many processes
Infographic cue	Fact: Sales in the middle	Category, Brand, Month, Year	Fact: Sales and Fact: Inventory

Mental model: Snowflake is a star whose dimensions were normalised. Galaxy is several stars that share dimensions. Master the star and the other two follow.

STAR SCHEMA · 1 OF 10 · DEFINITION

Star schema: one central fact, flat dimensions around it

A **star schema** is a dimensional model in which a single fact table, holding measures at a declared grain, is joined directly to a set of **denormalised** dimension tables by surrogate keys. Drawn out, it looks like a star.

One hop

Every dimension is exactly one join from the fact. No chains.

Flat dimensions

Product holds category and brand as columns, repeated per product.

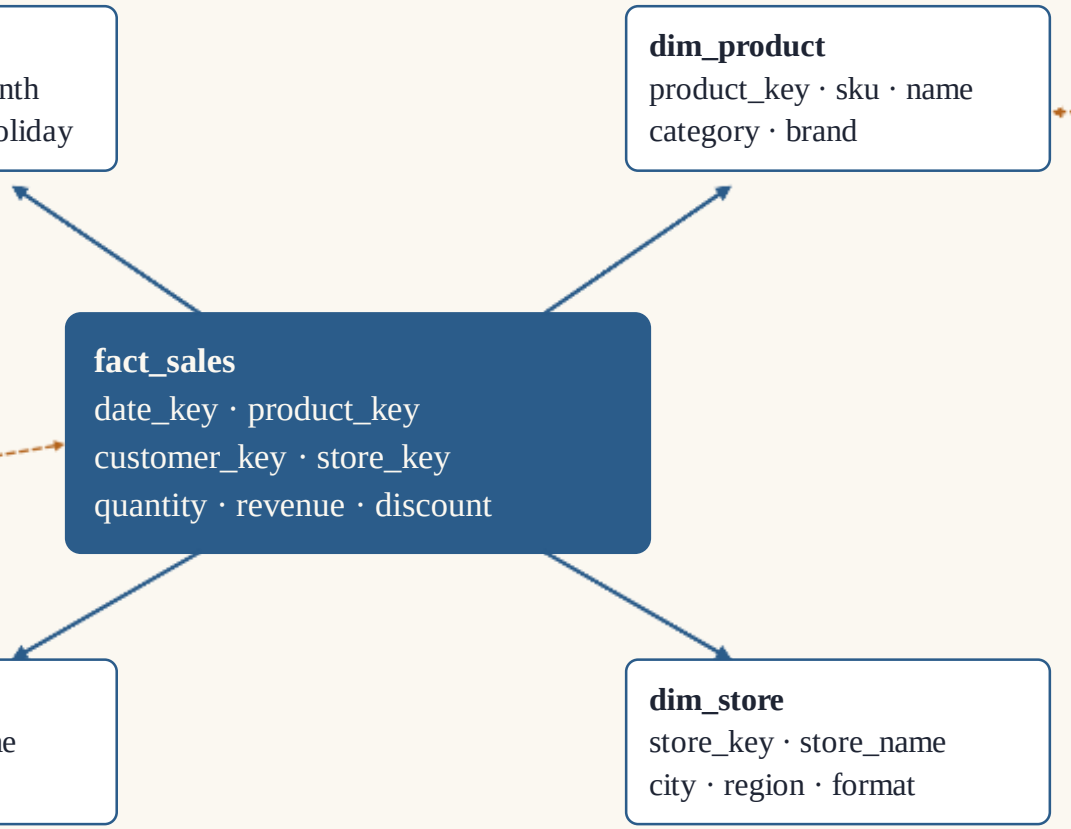
Faster analytics

As the infographic says: denormalised dimensions trade storage for speed.

Intuition: the fact table is a ledger of events; each dimension is a richly labelled lookup card. Answering a question means picking measures from the ledger and labels from the cards.

Anatomy of the Aster sales star

Date is a real table, not a raw timestamp: it carries holidays, fiscal periods and week numbers.



Category and brand live inside product. That is the denormalisation.

Fact columns are of two kinds only: foreign keys and numeric measures.

Every arrow is many facts to one dimension row: N to 1.

The star filled with Aster data

fact_sales

date_key	cust	prod	store	qty	revenue
20240103	1	41	7	1	420.00
20240103	1	43	7	2	60.00
20240103	2	41	9	1	420.00
20240104	3	44	8	4	60.00

Narrow rows of integers and numbers. Millions of these are cheap to store and fast to scan.

dim_product

key	sku	category	brand
41	DESK1	Office	Oakline
43	LAMP3	Home	Lumo
44	PAD9	Office	Oakline

dim_customer

key	name	city	segment
1	Ana	Boston	SMB
2	Raj	Seattle	ENT
3	Mila	Chicago	SMB

Spot the redundancy: "Office" and "Oakline" repeat on every office product. In a star that is intentional. Snowflake will remove it.

DDL for the star, with design choices annotated

```
CREATE TABLE dim_customer (
  customer_key  INTEGER   PRIMARY KEY,
  customer_id   VARCHAR(10),    natural key kept
  name VARCHAR(80), city VARCHAR(40), segment VARCHAR(10)
);

CREATE TABLE fact_sales (
  date_key      INTEGER NOT NULL REFERENCES dim_date,
  customer_key INTEGER NOT NULL REFERENCES dim_customer,
  product_key   INTEGER NOT NULL REFERENCES dim_product,
  store_key     INTEGER NOT NULL REFERENCES dim_store,
  order_id      VARCHAR(12),    degenerate dimension
  quantity      INTEGER,
  revenue       DECIMAL(12,2),
  discount      DECIMAL(12,2)
);
```

Surrogate keys are compact integers, assigned by the warehouse, never by a source system.

NOT NULL on every key. Missing context points to a special "Unknown" row (key 0 or a negative key), never to NULL.

Degenerate dimension: order_id has no attributes worth a table, so it lives in the fact for grouping and tracing.

DECIMAL for money, never floating point, so sums reconcile to the cent.

"Revenue by category and customer segment, 2024"

```
SELECT p.category,
       c.segment,
       SUM(f.revenue) AS revenue
FROM  fact_sales f
JOIN  dim_product  p ON p.product_key  = f.product_key
JOIN  dim_customer c ON c.customer_key = f.customer_key
JOIN  dim_date     d ON d.date_key     = f.date_key
WHERE d.year = 2024
GROUP BY p.category, c.segment;
```

1. **Labels come from dimensions**, the measure from the fact. The SELECT list reads like the business question.

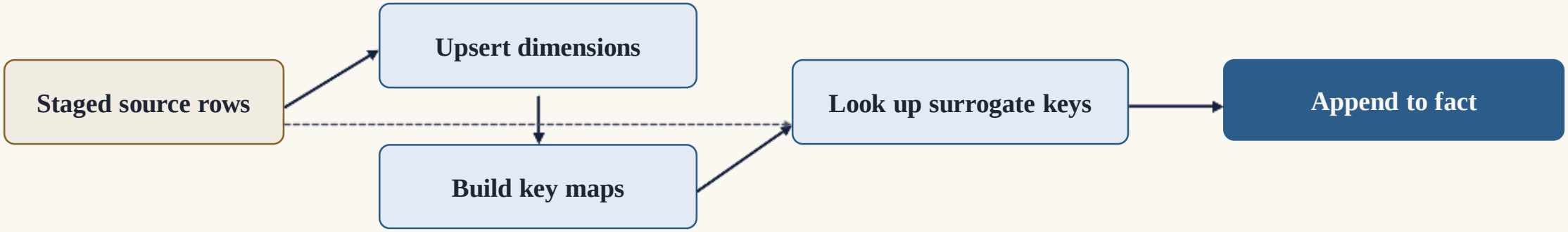
2. **Every join has the same shape:** fact key equals dimension key. Three joins for three contexts, all one hop.

3. **Filters hit small dimension tables first.** Engines push the year filter down and prune fact partitions: the "star join" optimisation.

Result for the four sample rows: Office / SMB 480.00 · Office / ENT 420.00 · Home / SMB 60.00

STAR SCHEMA · 6 OF 10 · LOADING

Load order: dimensions first, then facts



Why dimensions first? A fact row cannot point to a customer key that does not yet exist. Load order enforces referential integrity.

The lookup swaps natural keys (C1001, DESK1) for surrogates (1, 41) by joining staging to the dimensions.

Late arriving dimension? Insert an "inferred member" placeholder, load the fact, and fill in attributes when the data arrives.

Fact loads are append only and idempotent: delete and reload the affected date partition, or MERGE on the grain key, so a rerun never double counts.

Why the star has dominated BI for three decades

Understandable

Business users can read the diagram and name every table.

Fast

Few joins, small dimensions, star join optimisers, easy partition pruning.

Predictable SQL

Every query has the same skeleton, so BI tools generate it reliably.

Extensible

New dimensions or attributes add columns without breaking queries.

History ready

Surrogate keys make SCD Type 2 tracking natural.

Tool support

Power BI, Tableau, Looker and OLAP cubes all expect it.

What the star costs you

Weakness	What happens	Usual mitigation
Dimension redundancy	Category text repeats on every product; a rename touches many rows	Accept it; dimensions are small
Rigid grain	Questions finer than the grain cannot be answered	Model at atomic grain
Process specific	One star per process; cross process questions need care	Conformed dimensions, galaxy
Many to many pain	Multi valued dimensions (several sales reps per order) do not fit	Bridge tables with weights
Source changes ripple	A new source system can force dimension redesign	Integrate upstream in a vault or 3NF layer
Load complexity	Key lookups and SCD logic make pipelines heavier than plain copies	Standard macros and frameworks

> **Balanced verdict:** the star is optimised for reading, not for integrating. Its weaknesses mostly appear when it is asked to be the only layer in a complex enterprise.

Choose a star when, and avoid it when

Use it when

- The audience is analysts and BI tools
- The business process and grain are clear
- Queries aggregate many rows by a few labels
- You need a stable, documented presentation layer
- Point in time history of attributes matters

Avoid it when

- The system is operational, with heavy updates
- Sources are many, volatile and unintegrated
- Raw audit of every source value is required
- The data is a sparse event log with variable attributes
- Requirements are unknown and still exploratory

Aster decision: yes for the sales mart used by store managers and finance. Not for the raw web clickstream, which we model later with an activity schema.

Common star mistakes, and a question to test you

- **Mixed grain:** monthly targets stored in the daily sales fact. Use a separate fact.
- **Snowflaking by habit:** splitting category out of product with no benefit.
- **Text in facts:** customer names in the fact bloat it and break history.
- **Null foreign keys:** rows vanish from inner joins. Use an Unknown member.
- **Smart keys:** encoding meaning such as 2024BOS into the key.

Checkpoint

Aster wants "units returned by reason and store". Returns happen days after the sale and have their own reason code. Do you add columns to fact_sales or build a new fact?

Answer: a new fact_returns. It is a different business process with a different grain (one row per returned line) and a new dimension, dim_return_reason. It shares conformed date, product and store dimensions with sales.

SNOWFLAKE SCHEMA · 1 OF 10 · DEFINITION

Snowflake schema: a star whose dimensions are normalised

A **snowflake schema** keeps the central fact table of a star but splits each dimension into a chain of normalised tables, one per hierarchy level, so that every descriptive value is stored once. Drawn out, the branches resemble a snowflake.

Hierarchies as tables

Product → Category; Product → Brand; Date → Month → Year.

Less redundancy

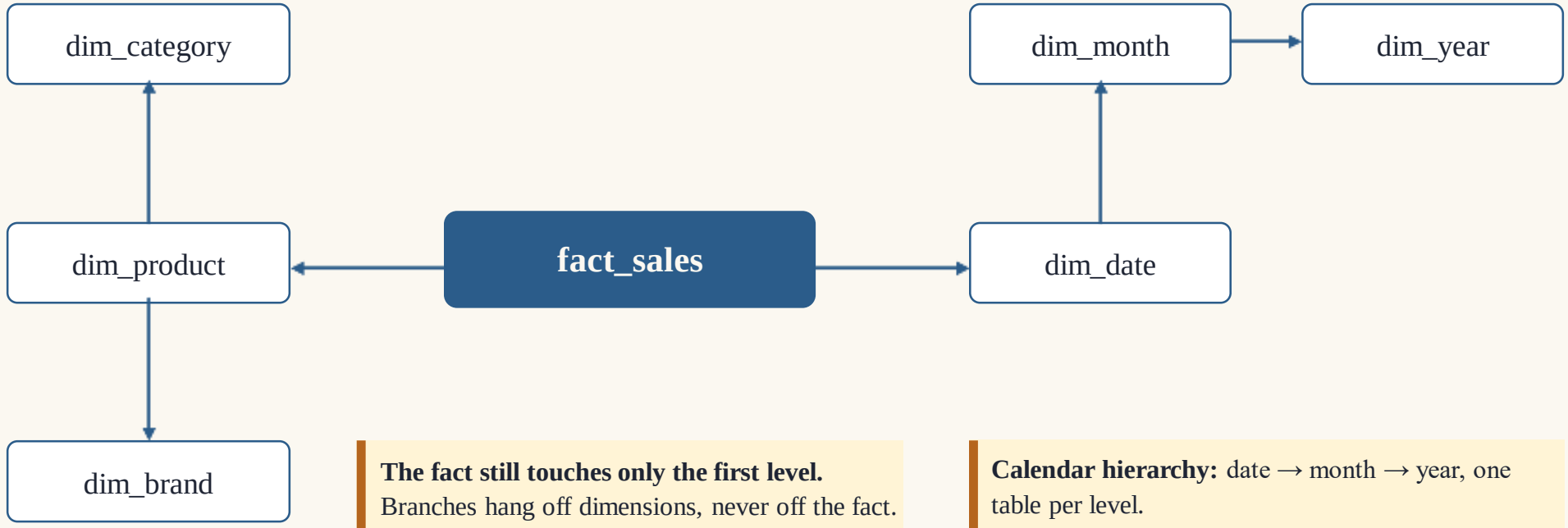
"Office" is stored once, in dim_category, not on every product.

More joins

Reaching category now costs two hops instead of one.

Intuition: the snowflake applies third normal form to the dimensions while leaving the fact untouched. It is a compromise between the star's simplicity and the integrity of normalised design.

Anatomy: branches grow out of each dimension



Outrigger level: one row per category.

The fact still touches only the first level. Branches hang off dimensions, never off the fact.

Calendar hierarchy: date → month → year, one table per level.

Matches the infographic: Category, Brand and Product on the left; Date, Month and Year on the right; Fact: Sales in the middle.

SNOWFLAKE SCHEMA · 3 OF 10 · WORKED EXAMPLE

The product dimension, snowflaked

dim_product

key	sku	cat_key	brand_key
41	DESK1	1	10
43	LAMP3	2	11
44	PAD9	1	10
45	CHAIR2	1	10

dim_category

cat_key	category
1	Office
2	Home

dim_brand

brand_key	brand
10	Oakline
11	Lumo

Rename "Office" to "Workspace": one row changes in dim_category, instead of three rows in a star.

But listing products with their category now needs a join that the star did not.

DDL: hierarchy levels become referenced tables

```
CREATE TABLE dim_category (
  category_key INTEGER PRIMARY KEY,
  category     VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE dim_brand (
  brand_key INTEGER PRIMARY KEY,
  brand       VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE dim_product (
  product_key  INTEGER PRIMARY KEY,
  sku          VARCHAR(12) NOT NULL,
  category_key INTEGER REFERENCES dim_category,
  brand_key    INTEGER REFERENCES dim_brand
);
```

UNIQUE on the label guarantees "Office" exists exactly once. This is the integrity gain the snowflake buys.

Foreign keys inside dimensions: dim_product now depends on two other dimension tables.

fact_sales is unchanged from the star. It still holds product_key only, never category_key.

Tip: publish a view, v_product, that joins the branches back into a flat star dimension for BI tools.

Same question, one extra hop per hierarchy level

```
SELECT cat.category, c.segment,
       SUM(f.revenue) AS revenue
FROM  fact_sales f
JOIN  dim_product  p   ON p.product_key = f.product_key
JOIN  dim_category cat ON cat.category_key = p.category_key
JOIN  dim_customer c   ON c.customer_key = f.customer_key
JOIN  dim_date     d   ON d.date_key = f.date_key
JOIN  dim_month    m   ON m.month_key = d.month_key
JOIN  dim_year     y   ON y.year_key = m.year_key
WHERE y.year = 2024
GROUP BY cat.category, c.segment;
```

Gold lines are new. The star needed three joins; the fully snowflaked model needs six for the same answer.

The year filter is now three hops away from the fact, which can weaken partition pruning on some engines.

Result is identical: Office / SMB 480 · Office / ENT 420 · Home / SMB 60. Shape changes; meaning does not.

In practice most teams keep the date dimension flat even in a snowflake, because calendar attributes never change.

SNOWFLAKE SCHEMA · 6 OF 10 · LOADING

Load from the outermost branch inwards



Rule: a table may load only after every table it references. In a snowflake this is a topological order of the hierarchy graph.

Key lookups multiply: product loading must look up both category_key and brand_key before inserting.

History question: if a product moves from Office to Home, a Type 2 change on dim_product simply points the new row to category_key 2. If the category itself is renamed, a Type 1 change in dim_category updates every product's label at once. Normalisation decides where history lives.

Orchestrators such as Airflow, Dagster or dbt infer this order automatically from declared dependencies.

SNOWFLAKE SCHEMA · 7 OF 10 · STRENGTHS

What the snowflake does well

Single point of change

Hierarchy labels live once. Renames and corrections are one row, so dimensions cannot contradict themselves.

Big hierarchies shrink

With millions of products and long text attributes per level, normalising saves real storage and load time.

Reusable levels

dim_category can serve a budget fact at category grain without inventing a fake product.

Closer to sources

ERP systems already store hierarchies this way, so mapping and reconciliation are simpler.

The strongest argument is the third card: shrunken, reusable dimensions for facts that arrive at a coarser grain. That is a modelling need, not a storage optimisation.

What the snowflake costs you

Weakness	Effect	Severity today
More joins	Longer SQL, more chances to join wrongly	Medium
Harder for users	Diagrams with twenty tables intimidate analysts	High
BI tool friction	Some tools expect one table per dimension	Medium
Storage saving is small	Dimensions are tiny next to facts; columnar compression already removes repetition	Makes the main benefit weak
Pruning weaker	Filters far from the fact are harder for optimisers	Engine dependent
Load ordering	More dependencies, longer pipelines	Low with orchestrators

Why the balance shifted: on columnar engines, a repeated value such as "Office" is dictionary encoded and costs almost nothing, which removes the snowflake's original justification.

Choose a snowflake when, and avoid it when

Use it when

- A hierarchy is very large or deep
- Facts arrive at several levels of one hierarchy
- Hierarchy labels are maintained by a separate team
- Integrity of reference data outweighs query ease
- A view layer will flatten it for BI users

Avoid it when

- Self service analysts write their own SQL
- The only motive is saving dimension storage
- Your BI tool works best on single table dimensions
- Hierarchies are short and stable
- Query latency is the top requirement

Aster decision: snowflake only the product category level, because the merchandising team plans budgets per category. Keep date, customer and store flat.

Snowflake traps, and a question to test you

Fact keyed to a branch: adding category_key to the fact as a shortcut. It goes stale when products move.

Nullable branch keys: products with no brand drop out of inner joins. Use an Unknown brand row.

Ragged hierarchies forced into fixed levels: some products have subcategories, some do not.

No flattening view: every analyst rebuilds the same six joins differently.

Checkpoint

Finance sets revenue targets per category per month. Products do not appear in the targets. Star or snowflake for the category level, and why?

Answer: a snowflaked (or separately conformed) dim_category, so that fact_target can reference category_key directly at its own grain, while fact_sales reaches the same category through dim_product. Actual versus target then compares on one shared category table.

Galaxy schema: several facts sharing conformed dimensions

A **galaxy** or **fact constellation** is a collection of star schemas, one per business process, whose fact tables share the same **conformed dimensions**. Because the dimensions are identical, measures from different processes can be compared on the same labels.

Conformed dimension

Same keys, same attribute names, same meaning, wherever it is used.

Drill across

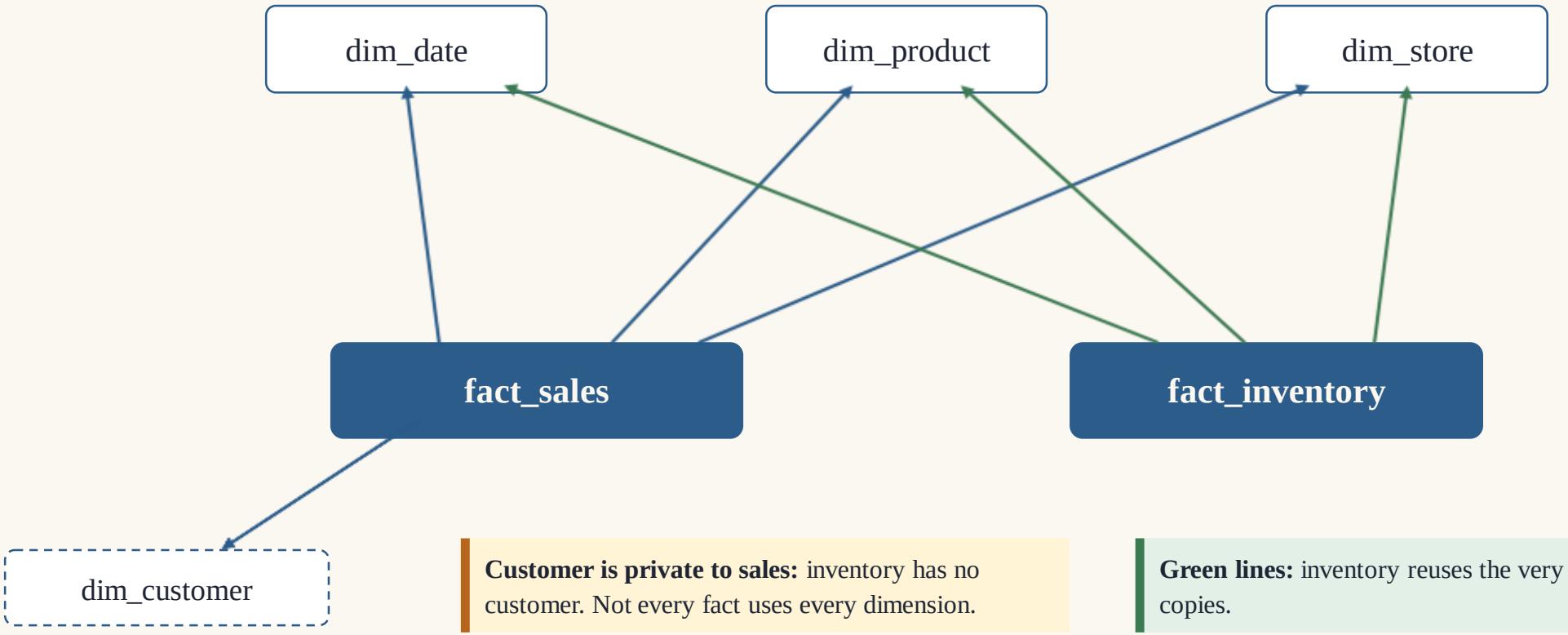
Aggregate each fact separately, then join the results on shared labels.

Bus architecture

The enterprise warehouse built incrementally, one process at a time.

Intuition: conformed dimensions are a shared vocabulary. Sales and Inventory can disagree about numbers, but never about what "Store 7" or "March 2024" means.

Anatomy: two facts, three shared dimensions, one private



Shared row of conformed dimensions across the top.

Customer is private to sales: inventory has no customer. Not every fact uses every dimension.

Green lines: inventory reuses the very same tables, not copies.

The Aster bus matrix: the galaxy's blueprint

Business process	Date	Product	Store	Customer	Category	Reason
Sales	Yes	Yes	Yes	Yes	via product	
Returns	Yes	Yes	Yes	Yes	via product	Yes
Inventory	Yes	Yes	Yes		via product	
Targets	Month		Yes		Yes	

Rows are facts (business processes). **Columns are conformed dimensions.** A column with many "Yes" cells is a dimension worth conforming first.

"Month" for Targets means a shrunken, conformed month dimension: same labels as dim_date, coarser grain.

fact_inventory · one row per product per store per day

date_key	product_key	store_key	on_hand
20240103	41	7	8
20240103	41	9	5

Three fact table types live comfortably in one galaxy

Transaction

One row per event. fact_sales. Sparse, additive, grows forever.

Periodic snapshot

One row per period per entity. fact_inventory. Dense, semi additive.

Accumulating snapshot

One row per process instance, updated at milestones. fact_order_fulfilment.

```
CREATE TABLE fact_inventory (
  date_key     INTEGER NOT NULL REFERENCES dim_date,
  product_key INTEGER NOT NULL REFERENCES dim_product,
  store_key    INTEGER NOT NULL REFERENCES dim_store,
  on_hand_qty INTEGER,
  on_order_qty INTEGER,
  PRIMARY KEY (date_key, product_key, store_key)
);
```

Same REFERENCES targets as fact_sales: that is conformance made physical.

Composite primary key equals the grain statement, word for word.

on_hand_qty is semi additive: never SUM it across dates.

Drill across: units sold versus closing stock, by product

```
WITH sold AS (
  SELECT product_key, SUM(quantity) AS units
  FROM fact_sales WHERE date_key BETWEEN 20240101 AND 20240131
  GROUP BY product_key ),
stock AS (
  SELECT product_key, SUM(on_hand_qty) AS closing
  FROM fact_inventory WHERE date_key = 20240131
  GROUP BY product_key )
SELECT p.sku, s.units, k.closing
FROM dim_product p
LEFT JOIN sold  s ON s.product_key = p.product_key
LEFT JOIN stock k ON k.product_key = p.product_key;
```

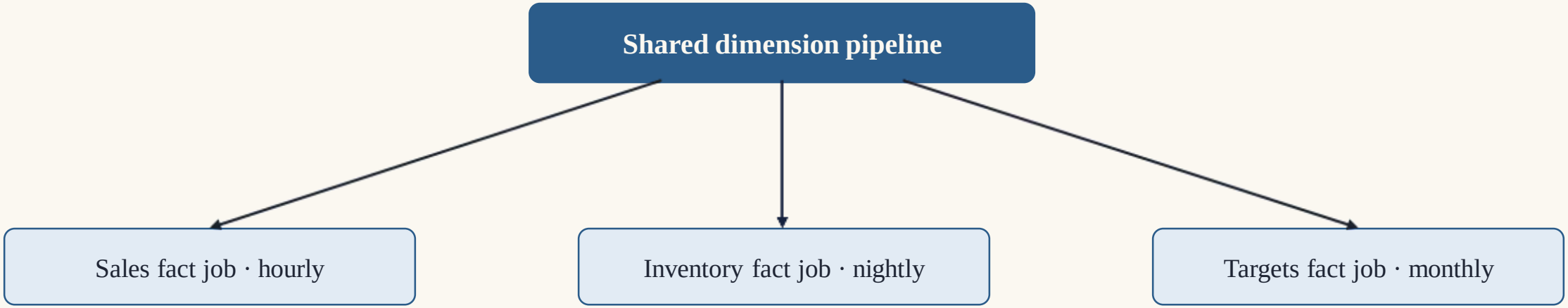
Step 1: aggregate each fact to the common grain (product) on its own. Never join two facts row to row.

Step 2: semi additive stock is filtered to one date (the 31st) before summing across stores.

Step 3: join the two results through the conformed dimension. LEFT JOIN keeps products that sold nothing.

Fan trap avoided: joining fact_sales directly to fact_inventory would multiply rows and inflate both measures.

Load conformed dimensions once; let facts load independently



One owner per dimension. A data steward team owns dim_product; fact teams consume it and never fork it.

Different cadences are fine. Each fact loads on its own schedule, after the dimension refresh it depends on.

Conformance tests: every fact key must exist in the shared dimension; row counts by label must reconcile.

Incremental delivery: the bus matrix lets Aster ship Sales first, then add Inventory months later without touching the Sales star.

What the galaxy gives the enterprise

Cross process insight

Sell through rate, stock cover, return rate: ratios that need two processes become routine.

One vocabulary

Every report filters by the same "Region" and "Fiscal Quarter". Numbers line up.

Incremental delivery

Add one star at a time. Value arrives early instead of after a big bang project.

Less duplication

Dimensions are built, tested and governed once, then reused by every fact.

Key insight: the galaxy's value grows with each fact added, because each new star can be compared with every existing one at no extra cost.

What the galaxy demands in return

Weakness	Effect	Mitigation
Governance load	Agreeing one definition of customer across departments is political work	Data stewards, a glossary
Change coupling	Altering a shared dimension can break many facts at once	Versioned contracts, additive changes only
Fan and chasm traps	Naive joins across facts double count	Drill across, semantic layer
Complex diagrams	Dozens of facts and dimensions overwhelm users	Present one star per subject area
Grain mismatches	Targets by month, sales by day need shrunken dimensions	Conformed rollups

Chasm trap explained: two facts each linked to customer. Join both to customer in one query and every sale pairs with every return of that customer, inflating totals.

Choose a galaxy when, and avoid it when

Use it when

- Two or more business processes are analysed together
- Leadership wants one set of KPIs across departments
- You plan to grow the warehouse in increments
- Shared dimensions have a clear owner

Avoid it when

- There is only one process; a single star is enough
- Departments cannot agree definitions yet
- Consumers cannot be trusted to drill across correctly and no semantic layer exists

Aster decision: yes. Sales, Returns, Inventory and Targets share Date, Product and Store. The bus matrix on slide 62 is the roadmap.

Galaxy traps, and the family summary

"Almost conformed": two customer tables that share 90% of rows. Worse than none, because they look identical.

Fact to fact joins in dashboards. Always aggregate first.

Summing snapshots across time, such as stock over a month.

Checkpoint

Why must fact_returns use the same dim_product rather than its own copy?

Answer: so return rate per category divides like by like; a copy could classify a product differently.

Dimensional family in one breath

Star: one fact, flat dimensions, fastest to read.

Snowflake: normalised dimensions, fewer duplicates, more joins.

Galaxy: many facts, conformed dimensions, enterprise scale.

All three share grain discipline, surrogate keys and the fact versus dimension split.

PART 3 OF 9 · FAMILY 2 OF 6

Normalized Modelling

Every fact stored once, in the place it depends on. The family that protects integrity, absorbs change and feeds the rest of the warehouse.

Third Normal Form | ER Model | Inmon CIF

The problem this family solves: trustworthy writes and integration

1970

Codd publishes the relational model: data as sets of tuples, manipulated by algebra, free of physical pointers.

1976

Chen proposes the entity relationship model, a diagram language for designing relational schemas.

1990s

Inmon defines the enterprise data warehouse as subject oriented, integrated, time variant and non volatile.

Shared principle: design from the meaning of the data, not from the reports. Reports change monthly; the business's entities change slowly.

Where it lives: operational databases, and the integration layer of a warehouse that then feeds dimensional marts.

3NF, ER Model and Inmon CIF side by side

Property	3NF	ER Model	Inmon CIF
What it is	A rule set for tables	A design method and notation	A warehouse architecture
Main output	Normalised tables	Entity diagram, then schema	EDW plus departmental marts
Scope	Any relational schema	One system or domain	The whole enterprise
Key question	Is each fact stored once?	What exists and how does it relate?	How do many sources become one truth?
Infographic cue	Customer, Order, Order Line, Product	Customer Places Order, attributes as ovals	CRM, ERP, Web → EDW → marts
Relationship	The target state	The way to design it	The place to apply it at scale

Mental model: you draw an ER model, you verify it reaches 3NF, and in a CIF you build the enterprise warehouse from that 3NF schema.

Third normal form: every non key attribute depends on the key

A table is in **third normal form** when it is in 2NF and no non key attribute depends on another non key attribute. Formally: for every functional dependency X → A, either X is a superkey or A is part of a candidate key.

One fact, one place

A customer's city is stored in exactly one row of one table.

No anomalies

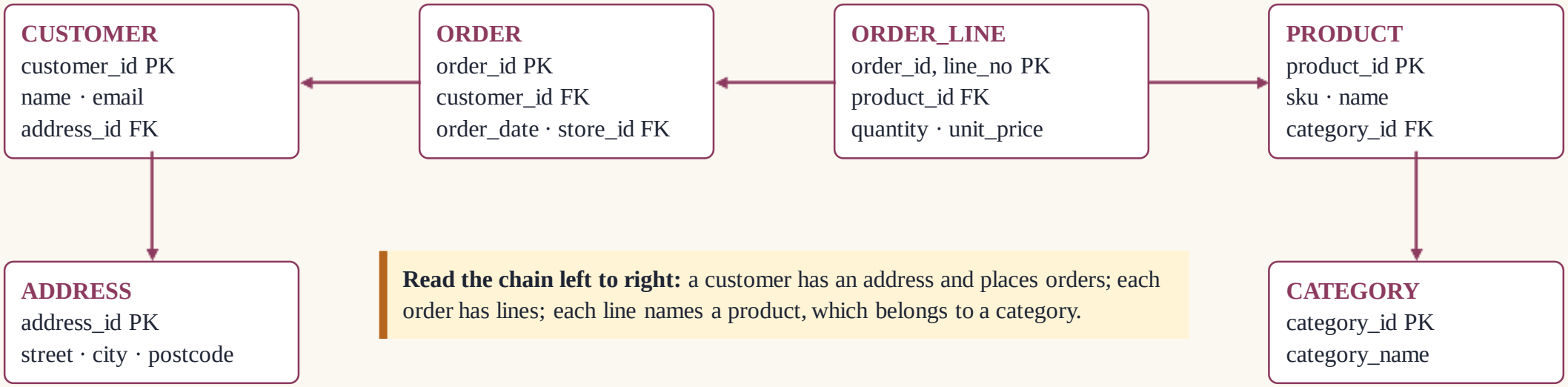
Insert, update and delete anomalies from slide 27 disappear.

Many narrow tables

Meaning is recovered at query time by joining them back.

Intuition: each table describes exactly one kind of thing, and nothing about any other thing. If a column describes something else, it belongs in that something else's table.

Anatomy of Aster's order system in 3NF



Read the chain left to right: a customer has an address and places orders; each order has lines; each line names a product, which belongs to a category.

unit_price lives on the line, not only on the product: it records the price actually charged, which can differ from today's list price.

The anomaly table from slide 27, now in 3NF

CUSTOMER

id	name	addr
C1001	Ana	A1
C1002	Raj	A2

ADDRESS

addr	city
A1	Austin
A2	Seattle

PRODUCT

id	name	cat
P1	Desk	K1
P2	Lamp	K2

ORDER

order	cust
O1	C1001
O2	C1001
O3	C1002

ORDER_LINE

order	prod	price
O1	P1	420
O2	P2	60
O3	P1	420

CATEGORY

cat	name
K1	Office
K2	Home

Update fixed: Ana's move is one cell, A1 to Austin.

Insert fixed: a Chair can exist with no orders.

Delete fixed: cancelling O3 keeps Raj.

DDL: constraints carry the business rules

```
CREATE TABLE order_line (
  order_id    BIGINT      NOT NULL,
  line_no     SMALLINT NOT NULL,
  product_id BIGINT      NOT NULL,
  quantity    INTEGER    NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
  PRIMARY KEY (order_id, line_no),
  FOREIGN KEY (order_id)
    REFERENCES orders (order_id) ON DELETE CASCADE,
  FOREIGN KEY (product_id)
    REFERENCES product (product_id) ON DELETE RESTRICT
);
CREATE INDEX ix_line_product ON order_line (product_id);
```

CHECK constraints encode domains: quantity is at least one, price is never negative.

Composite primary key mirrors the weak entity: a line exists only within its order.

Referential actions state intent: delete an order and its lines go too; a product with sales history may not be deleted.

Index on foreign keys keeps joins and constraint checks fast in a row store.

Two very different queries against the same 3NF schema

Operational: show order O1

```
SELECT l.line_no, p.name, l.quantity, l.unit_price
FROM order_line l
JOIN product p ON p.product_id = l.product_id
WHERE l.order_id = 'O1';
```

Ideal for 3NF: a primary key lookup touching a few rows, served by indexes in milliseconds.

Analytical: revenue by category and city

```
SELECT k.name, a.city,
  SUM(l.quantity * l.unit_price)
FROM order_line l
JOIN orders o     ON o.order_id = l.order_id
JOIN customer c ON c.customer_id = o.customer_id
JOIN address a  ON a.address_id = c.address_id
JOIN product p  ON p.product_id = l.product_id
JOIN category k ON k.category_id = p.category_id
GROUP BY k.name, a.city;
```

Five joins, a computed measure, and the city is today's city, not the city at the time of sale.

How data enters a 3NF model: one transaction, many tables

```
BEGIN;
INSERT INTO orders (order_id, customer_id, store_id, order_date)
  VALUES (1004, 'C1003', 8, CURRENT_DATE);
INSERT INTO order_line VALUES (1004, 1, 'P9', 4, 15.00);
INSERT INTO order_line VALUES (1004, 2, 'P2', 1, 30.00);
UPDATE stock SET on_hand = on_hand   minus 4
  WHERE product_id = 'P9' AND store_id = 8;
COMMIT;
```

Atomicity: all four statements succeed or none do. Half an order can never exist.

Each fact touched once: stock decreases in exactly one row because it lives in exactly one row.

Constraints check every row: an unknown product_id is rejected at write time.

In a 3NF warehouse layer (the CIF's EDW), loading is batch instead: stage each source, match records to existing entities by business key, then MERGE inserts and updates table by table in dependency order, parents before children.

3NF · 7 OF 10 · STRENGTHS

Why 3NF is the default for operational systems

Integrity

Anomalies are impossible by design; constraints catch bad data at the door.

Fast writes

Small rows, single updates, high concurrency without lock contention.

Compact

No repeated values; storage grows only with genuinely new facts.

Query neutral

Not shaped for any one report, so it can answer questions nobody planned.

Change friendly

A new attribute goes in one table; nothing else moves.

Rigorous theory

Decades of mathematics make correctness provable, not a matter of taste.

What 3NF costs analysts

Weakness	Effect	Mitigation
Join heavy reads	Analytical queries chain five to fifteen joins	Build marts or views on top
Hard to navigate	Hundreds of tables in a real ERP; analysts get lost	Documentation, curated layers
No history by default	Updates overwrite; past states vanish	Audit tables, temporal tables, vault
Computed measures	Revenue must be derived from quantity times price each time	Derived columns in marts
Columnar mismatch	Cloud warehouses favour wide scans over many joins	Denormalise for presentation

Balanced verdict: 3NF is optimised for the moment of writing. Every weakness above appears at the moment of reading, which is why warehouses add a presentation layer.

Choose 3NF when, and avoid it when

Use it when

- Building an application database (OLTP)
- Many concurrent writers update shared data
- Building an integration layer that must stay neutral
- Master data management of customers or products
- Correctness matters more than query convenience

Avoid it when

- Business users query the tables directly
- The workload is large scans and aggregations
- You need full history and the engine lacks temporal tables
- Sources change shape often and you need agility

Aster decision: the POS and CRM applications are 3NF. The warehouse does not expose 3NF to analysts; it uses it only as the input to integration.

Beyond 3NF, common traps, and a test

Form	Rule	Typical violation
BCNF	Every determinant is a candidate key	Overlapping composite keys
4NF	No independent multi valued facts in one table	Customer, phone and hobby in one table
5NF	No join dependencies not implied by keys	Rare three way relationships
6NF	One non key attribute per table	Basis of anchor modeling

Over normalising lookup values such as yes and no into their own tables.

Dropping historical prices: unit_price must stay on the line.

Constraints declared but not enforced in a warehouse, with no tests.

Checkpoint

STORE(store_id, city, region, region_manager). Region decides the manager. Is it 3NF?

Answer: no. store_id → region → region_manager is transitive. Split into STORE(store_id, city, region) and REGION(region, region_manager).

The entity relationship model: designing with pictures first

The **entity relationship (ER) model** describes a domain as **entities** (things), **attributes** (their properties) and **relationships** (associations between them), drawn as a diagram that is then translated mechanically into relational tables.

Entity · rectangle

Customer, Order, Line.

Relationship · diamond

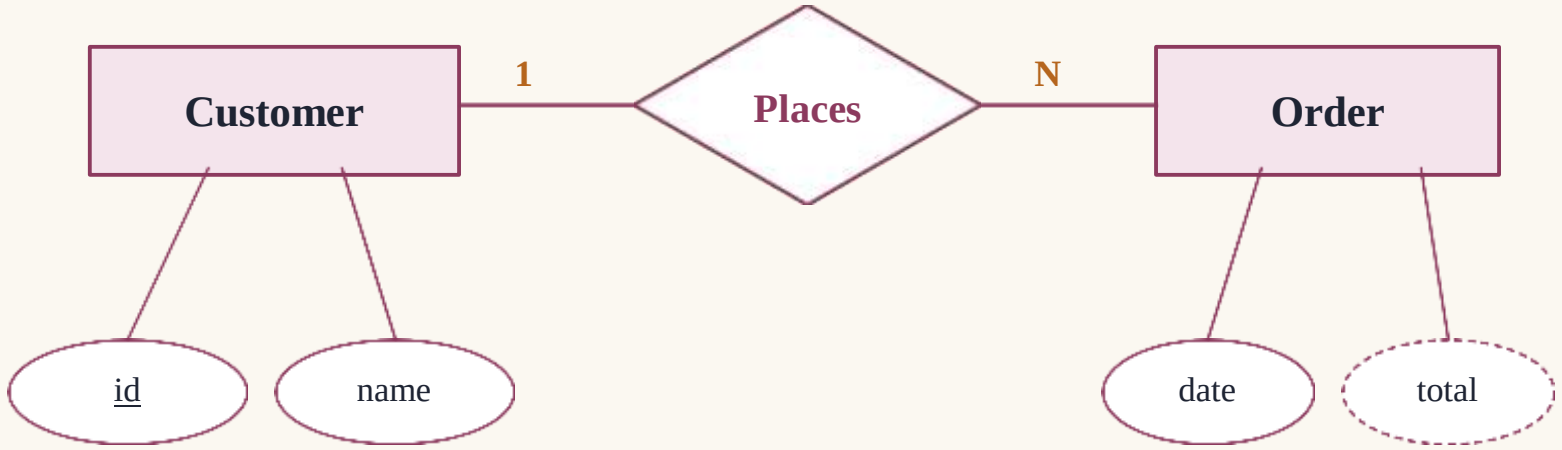
Places, Contains. A verb.

Attribute · oval

name, email, date, total.

Intuition: the ER model is the bridge between a conversation with the business and a database schema. It is the method; 3NF is the quality test the result must pass.

Anatomy of a Chen diagram: Customer places Order



Diamond = relationship, named with a verb so the diagram reads as a sentence.

1 and N on the lines give cardinality: one customer, many orders.

Underlined id marks the key. **Dashed oval** marks a derived attribute: total is computed from lines.

The infographic's ER panel shows Customer, Places, Order and Line along the top with name, email, date and total hanging below: this exact structure.

From an interview to entities: the noun and verb method

*"A **customer** **places** **orders** at a **store** or online. Each order **contains** one or more **products**. A **store** is **managed by** exactly one **employee**, who may manage none."*

Relationship	Entities	Cardinality	Participation
Customer places Order	Customer, Order	1 : N	Order mandatory, customer optional
Order contains Product	Order, Product	M : N	Order needs one or more
Order fulfilled at Store	Order, Store	N : 1	Online orders: store optional
Store managed by Employee	Store, Employee	1 : 1	Store mandatory, employee optional

Method: nouns become candidate entities (plum), verbs become relationships (copper), quantifiers such as "one or more", "exactly one", "may" set cardinality and participation.

Seven rules that turn any ER diagram into tables

#	ER construct	Relational result
1	Strong entity	A table; simple attributes become columns; the key becomes the primary key
2	Weak entity	A table whose key combines the owner's key with a partial key (order_id, line_no)
3	1 : 1 relationship	Foreign key on the side with total participation, marked UNIQUE
4	1 : N relationship	Foreign key on the N side
5	M : N relationship	A new associative table holding both keys plus relationship attributes
6	Multi valued attribute	A child table (owner key, value)
7	Composite or derived attribute	Split into parts; derived values usually computed, not stored

Payoff: a well drawn ER diagram mapped with these rules lands in 3NF almost automatically, because each table describes one entity or one relationship.

Applying the rules to the interview: the generated schema

```
CREATE TABLE employee (employee_no INT PRIMARY KEY, name VARCHAR(80));

CREATE TABLE store (
  store_id INT PRIMARY KEY, city VARCHAR(40),
  manager_no INT NOT NULL UNIQUE REFERENCES employee );

CREATE TABLE orders (
  order_id BIGINT PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customer,
  store_id INT NULL REFERENCES store,
  channel VARCHAR(10) CHECK (channel IN ('STORE','ONLINE')) );

CREATE TABLE order_line (
  order_id BIGINT REFERENCES orders, line_no SMALLINT,
  product_id BIGINT NOT NULL REFERENCES product,
  quantity INT, PRIMARY KEY (order_id, line_no) );
```

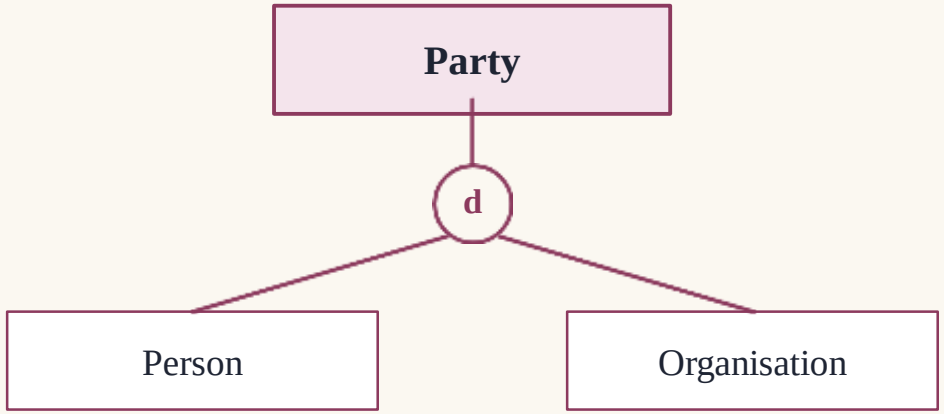
Rule 3, 1 : 1: manager_no sits on store (total participation) with UNIQUE so no employee manages two stores.

Rule 4, N : 1: store_id on orders is NULL allowed because online orders have no store.

"Online" became a CHECK constrained attribute, not an entity.

Rules 2 and 5: order_line is both the weak entity and the M : N resolver.

Extended ER: subtypes, recursion and how they map





Recursive relationship: an employee reports to another employee. Maps to a self referencing foreign key, manager_no on employee.

Generalisation: a customer can be a person or an organisation. "d" means disjoint: never both. Map as one table with a type column, one table per subtype, or both.

Weak entity reminder: Order Line is drawn with a double rectangle and an identifying double diamond to Order.

Recursive hierarchies (org charts, product taxonomies) later become parent child dimensions or bridge tables in the dimensional world.

What the ER model gives a team

A shared language

Business experts can review a diagram long before they could review SQL.

Errors found early

A wrong cardinality costs a pencil stroke on paper and a migration in production.

Mechanical translation

The mapping rules make schema generation repeatable; many tools forward engineer DDL.

Platform independent

The same diagram can target PostgreSQL, Oracle or a document store.

Underrated strength: the ER diagram is the best documentation a system will ever have, because it explains intent, not just structure.

Where ER modelling falls short

Weakness	Effect	Mitigation
Static snapshot	Describes structure, not behaviour or history	Add temporal design explicitly
Diagram sprawl	Enterprise models reach hundreds of entities	Subject area views
Notation wars	Chen, crow's foot, IDEF1X, UML confuse readers	Pick one house notation
Drift from reality	Diagrams go stale as schemas evolve	Generate diagrams from code
Not analytics shaped	Says nothing about grain, measures or aggregation	Pair with dimensional design

Balanced verdict: the ER model is a thinking tool. Its weaknesses appear when teams treat the diagram as the deliverable instead of the understanding it captures.

Reach for an ER model when, and skip it when

Use it when

- Starting any new domain or application
- Onboarding to an unfamiliar source system
- Building the conceptual model for a vault or EDW
- Resolving disputes about what an entity means

Skip it when

- Modelling a single event stream with no entities to speak of
- A dimensional bus matrix already captures the scope
- Nobody will maintain the diagram after launch

Aster decision: draw an ER model of each source (CRM, ERP, POS) before designing hubs and links. Vault design starts from exactly this picture.

ER traps, and a question to test you

- **Attributes as entities:** "Colour" with one column does not need a box.
- **Relationship attributes on an entity:** quantity belongs on Contains, not on Product.
- **Only asking one direction** of each relationship, so minimums are wrong.
- **Modelling the screens** of an application instead of the business.

Checkpoint

"A product can be supplied by several suppliers, each at its own cost price." How many tables, and where does cost price live?

Answer: three: PRODUCT, SUPPLIER and an associative PRODUCT_SUPPLIER(product_id, supplier_id, cost_price). Cost depends on the pair, so it lives on the relationship table.

The Corporate Information Factory: integrate first, then serve

The **Corporate Information Factory (CIF)** is Bill Inmon's architecture in which all sources feed one normalised **enterprise data warehouse (EDW)**, and departmental **data marts** are derived from that EDW, never directly from sources.

Inmon's definition of a data warehouse has four properties

Subject oriented

Organised by customer, product, sale; not by application.

Integrated

One key, one code set, one unit of measure across sources.

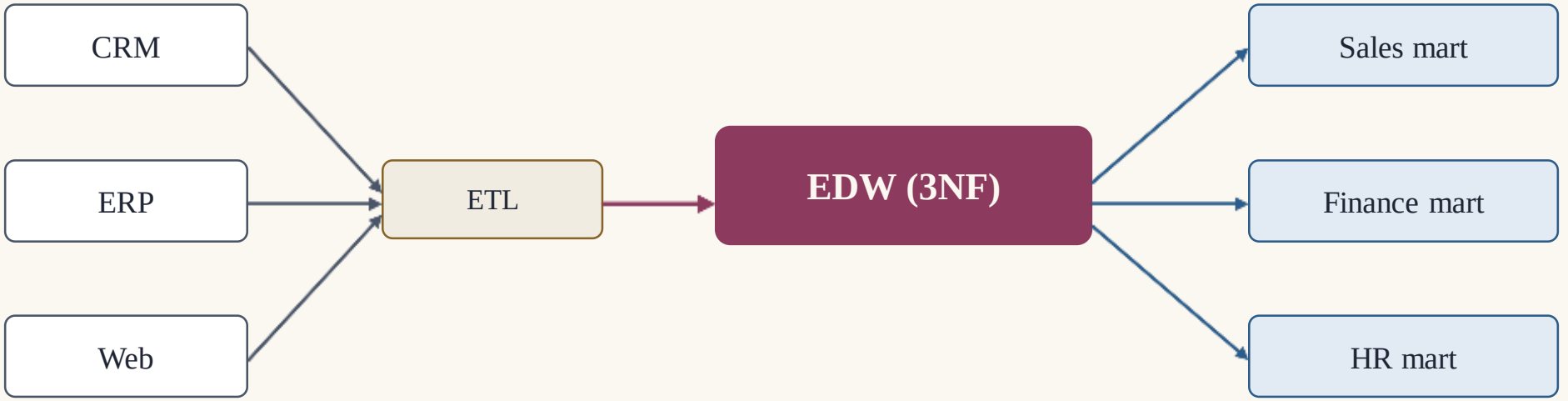
Time variant

Every record is tied to a moment; history is kept.

Non volatile

Loaded and read, not updated in place by users.

Anatomy: a hub of integration with spokes of delivery



Marts are dimensional, often stars, built only from the EDW.

Single entry point: every source passes through one integration step. No mart reads a source directly, so marts cannot disagree on raw values.

The EDW is normalised and atomic: the system of record for analytics, neutral to any department's questions.

Integration in action: two sources, one customer

CRM.customer

id	name	seg
C1001	Ana	S
C1002	Raj	E

WEB.user

email	tier
ana@mail.com	small
mila@mail.com	small



EDW.party · integrated

party_id	name	segment	crm_id	web_id
1	Ana	SMB	C1001	ana@...
2	Raj	ENT	C1002	
3	Mila	SMB		mila@...

Code harmonisation: S, small → SMB; E → ENT.

Identity resolution: Ana matched across systems by email held in CRM.

EDW tables: 3NF plus time, to make it time variant

```
CREATE TABLE edw.party_address (
  party_id       BIGINT  NOT NULL REFERENCES edw.party,
  address_id     BIGINT  NOT NULL REFERENCES edw.address,
  effective_from DATE NOT NULL,
  effective_to   DATE,
  source_system  VARCHAR(10) NOT NULL,
  load_ts        TIMESTAMP NOT NULL,
  PRIMARY KEY (party_id, effective_from)
);
```

Effective dating makes the relationship time variant: Ana's Boston and Austin addresses both survive, each with its own period.

Lineage columns (source_system, load_ts) record where and when every row came from.

Still 3NF: no category, city or segment text is copied here; only keys and time.

Compare with an OLTP 3NF schema: same entities, but the warehouse version adds time, source and load metadata to every changing relationship.

Deriving a mart from the EDW, with history preserved

```
CREATE TABLE sales_mart.fact_sales AS
SELECT o.order_date,
       p.party_id,
       a.city  AS city_at_sale,
       l.product_id,
       l.quantity * l.unit_price AS revenue
FROM edw.order_line l
JOIN edw.orders o ON o.order_id = l.order_id
JOIN edw.party p  ON p.party_id = o.party_id
JOIN edw.party_address pa
  ON pa.party_id = p.party_id
 AND o.order_date >= pa.effective_from
 AND (pa.effective_to IS NULL OR o.order_date < pa.effective_to)
JOIN edw.address a ON a.address_id = pa.address_id;
```

The mart is a projection of the EDW: denormalised, measure precomputed, ready for BI.

The range join picks the address valid on the order date. 2022 orders get Boston; 2024 orders get Austin.

Rebuildable: drop the mart and recreate it at any time. The EDW is the source of truth; marts are disposable views of it.

Answers Question A from slide 33.

The CIF load cycle in five stages

1

Extract

Pull changed rows from CRM, ERP, web into staging.

2

Cleanse

Validate types, trim, standardise codes and units.

3

Integrate

Match identities, assign enterprise keys, resolve conflicts.

4

Load EDW

Close old periods, insert new ones, parents first.

5

Refresh marts

Rebuild or incrementally update each mart.

Stage 3 is the bottleneck. Survivorship rules decide which source wins when CRM and web disagree on a name. These rules are business decisions, not code.

Coupling risk: a new source changes integration rules and may force EDW redesign. This is the pain Data Vault was invented to remove.

What a CIF gives a large organisation

Single source of truth

Every mart is cut from the same integrated cloth.

Full history

Time variance is built into the EDW, not bolted onto marts.

Disposable marts

New departmental views can be built without touching sources.

Auditability

Lineage columns trace each number to a source row.

Enterprise scope

Cross functional analysis without reconciling silos.

Governance

One controlled place to apply security and quality rules.

What a CIF costs

Weakness	Effect	Mitigation
Slow time to value	The EDW must be designed before the first mart ships	Phase by subject area
Big upfront design	Enterprise wide modelling needs rare skills and long workshops	Industry reference models
Brittle to new sources	Integration logic is hard coded into EDW tables	Use a vault as the EDW
Two layers to maintain	Every change ripples from EDW to each mart	Automated mart generation
Business rules early	Cleansing and conforming before storing can lose raw values	Keep a raw staging archive

Balanced verdict: the CIF optimises for long term consistency at the expense of short term speed. Modern teams keep its hub and spoke shape but implement the hub as a Data Vault.

Inmon or Kimball? Choosing the architecture

Criterion	Inmon CIF · top down	Kimball bus · bottom up
Core layer	Normalised EDW	Conformed dimensional marts
First delivery	Months, after EDW design	Weeks, one star at a time
Best when	Many sources, strict regulation, stable scope	Clear processes, fast feedback, BI first
Risk	Never finishing the EDW	Silos if dimensions do not conform
Team	Central architecture group	Smaller, iterative teams

Use a CIF when integration and audit across dozens of sources is the primary requirement.

Avoid it when a small team needs answers this quarter from a handful of sources.

Aster decision: a hybrid. An integration hub (built as a vault, next family) feeds Kimball style marts: Inmon's shape, Kimball's presentation.

CIF traps, and the family summary

Marts reading sources directly "just this once". The EDW loses its authority.

Boiling the ocean: modelling every subject before shipping anything.

Analysts on the EDW: it is for integration, not self service.

Checkpoint

Which of Inmon's four properties is violated if users can UPDATE EDW rows?

Answer: non volatile. History becomes untrustworthy.

Normalized family in one breath

3NF: each fact once, depending on the key.

ER Model: the method that discovers entities and relationships.

Inmon CIF: a 3NF EDW that integrates all sources and feeds marts.

Strong at integrity and integration; weak at direct analytical use.

PART 4 OF 9 · FAMILY 3 OF 6

Data Vault

Split every source into three kinds of table: what the business identifies, how those things relate, and what was said about them over time. Add sources without redesign; keep every version forever.

Hubs Satellites Links

DATA VAULT · FAMILY OVERVIEW

The problem Data Vault solves: change without redesign

In a 3NF EDW, a new source means...

altering existing tables · rewriting integration rules · reloading history · retesting every mart that reads those tables

In a Data Vault, a new source means...

adding new satellites (and maybe links) beside existing hubs · nothing already loaded is altered

Hub: the list of business keys. The "nouns". Never updated.

Link: associations between hubs. The "verbs".

Satellite: descriptive attributes with full history. The "adjectives".

> **The core insight:** keys are stable, relationships change occasionally, descriptions change constantly. Storing each in its own table type isolates change exactly where it happens.

Hubs, Satellites and Links side by side

Property	Hub	Satellite	Link
Stores	Unique business keys	Descriptive attributes	Relationships between keys
Grain	One row per key	One row per key per change	One row per unique key combination
Changes	Never updated	New row on every change	Never updated
Key columns	hash key, business key	parent hash key, load date	link hash key, hub hash keys
Metadata	load_dt, rec_src	load_dt, rec_src, hash_diff	load_dt, rec_src
Dimensional analogue	Natural key of a dimension	Type 2 dimension attributes	Fact keys or a bridge

Rule of thumb: a hub has no descriptive columns, a link has no descriptive columns, and a satellite has no foreign keys except to its one parent.

Hub: the unique list of business keys, never updated

A **hub** is a table holding one row for every distinct **business key** of a core business concept, such as customer or product, plus a hash of that key and the metadata of when and from where it was first seen.

Business key

The identifier people actually use: customer number, SKU, IBAN, VIN.

Insert only

Once a key arrives it stays forever, even if the source deletes it.

The integration point

Every source that knows customer C1001 attaches to the same hub row.

Intuition: a hub is the warehouse's address book. It only records that someone exists and when we first heard of them. Everything we know about them lives elsewhere.

Anatomy of hub_customer: four columns, four jobs

Hash key: MD5 or SHA of the business key. Primary key; computed, not looked up.

Business key: as the business uses it, unique across the hub.



hub_customer_hk	customer_id	load_dt	rec_src
a1f3...	C1001	2024/01/03	crm

Load date: when the warehouse first saw this key. Not a business date.

Record source: which system first delivered the key; vital for audit.

HUBS · 3 OF 10 · WORKED EXAMPLE

Aster's hubs after the first three loads

hub_customer

huv_key	customer_id	load_dt	rec_src
a1f3...	C1001	2024/01/03	crm
9c02...	C1002	2024/01/03	crm
e77b...	C1003	2024/02/11	web

hub_product

hk	sku	load_dt	rec_src
p410...	DESK1	2024/01/02	erp
p522...	LAMP3	2024/01/02	erp

Mila arrived first via the web shop, so rec_src is web. When CRM later sends C1003, no new row is added: the key already exists.

Three rows, forever. Ana moving city, changing name or closing her account never touches this table.

Rows match the infographic: a1f3, 9c02, e77b with C1001 to C1003.

DDL and hashing rules for a hub

```
CREATE TABLE raw_vault.hub_customer (
  hub_customer_hk CHAR(32)    NOT NULL PRIMARY KEY,
  customer_id      VARCHAR(20) NOT NULL UNIQUE,
  load_dt          TIMESTAMP   NOT NULL,
  rec_src          VARCHAR(20) NOT NULL
);
```

Why hash? Any loader can compute the key from the business key alone, in parallel, with no lookup against the hub. Hubs, links and satellites load at the same time.

```
hub_customer_hk = MD5( UPPER( TRIM( customer_id ) ) )
composite keys: MD5( UPPER(TRIM(a)) || '||' || UPPER(TRIM(b)) )
```

Standardise before hashing: trim and upper case, so " c1001" and "C1001" produce the same key.

Delimiters between parts stop collisions such as "AB"+"C" versus "A"+"BC".

Cross platform tip: the same hash on Snowflake, Databricks and Python means keys can be computed anywhere in the pipeline and still match.

What a hub alone can answer, and its role as the spine

Existence questions

```
SELECT DATE_TRUNC('month', load_dt) AS month,
    rec_src,
    COUNT(*) AS new_customers
FROM raw_vault.hub_customer
GROUP BY 1, 2
ORDER BY 1;
```

New customers per month by acquisition channel: answered from four columns, no joins.

The spine pattern

```
SELECT h.customer_id, crm.name, web.email
FROM raw_vault.hub_customer h
LEFT JOIN sat_customer_crm crm
  ON crm.hub_customer_hk = h.hub_customer_hk
LEFT JOIN sat_customer_web web
  ON web.hub_customer_hk = h.hub_customer_hk;
```

Start from the hub, LEFT JOIN everything else. Every known customer appears, even if a source has nothing about them.

The spine query omits the "latest row only" filter for clarity; slide 121 adds it.

Hub load: insert keys never seen before, nothing else

```
INSERT INTO raw_vault.hub_customer
SELECT DISTINCT
      MD5(UPPER(TRIM(s.customer_id))),
      UPPER(TRIM(s.customer_id)),
      s.load_dt,
      'crm'
FROM staging.crm_customer s
WHERE NOT EXISTS (
  SELECT 1 FROM raw_vault.hub_customer h
  WHERE h.customer_id = UPPER(TRIM(s.customer_id)) );
```

DISTINCT: a batch may mention the same customer many times; the hub wants one row.

NOT EXISTS: idempotent. Run it twice and the second run inserts nothing.

No UPDATE, no DELETE statement exists anywhere in vault loading.

One template, every hub: because every hub has the same four columns, the same load pattern is generated for all of them. This is why vaults are built with automation tools such as dbt packages (AutomateDV, datavault4dbt), WhereScape or VaultSpeed.

HUBS · 7 OF 10 · STRENGTHS

Why hubs are the vault's foundation

Stable for decades

Business keys outlive applications. A hub designed today rarely changes.

Natural integration

Sources sharing a key meet automatically, without a matching project.

Parallel loading

Hash keys remove load order dependencies between tables.

Complete inventory

Every key ever seen, with first sighting and origin: a ready made audit list.

Key insight: by holding nothing but identity, hubs are immune to the most common kind of change, attribute change, which is pushed entirely into satellites.

Where hubs struggle

Weakness	Effect	Mitigation
Conflicting keys	CRM says C1001, web says an email; they do not meet in one hub	Same as links, or a business vault match
Key collisions	Two source systems both use "1001" for different customers	Add a business key collision code
No meaning alone	A hub shows ids, never names; useless to analysts	Always query via marts
Hash cost	32 character keys are wider than integers	Binary storage
Key discovery effort	Finding true business keys needs business interviews	Start from the ER model

Balanced verdict: a hub is only as good as its business key. Choose a technical surrogate from one application by mistake and the whole integration promise breaks.

Does this deserve a hub? A four question test

1 · Is it a core business concept?

Customer, product, store: yes. Colour, status code: no, those are satellite attributes.

2 · Does it have its own key?

Something people quote on the phone or on paper.

3 · Is the key shared or stable?

Used across systems, or at least never reused for something else.

4 · Does it exist independently?

Order lines do not; they become link or dependent child keys.

Aster hubs: customer, product, store, order, employee.

Not hubs: category (a product attribute here), line number, web session id.

Hub traps, and a question to test you

- **Source surrogate as business key:** an auto increment id from one app.
- **Descriptive columns in the hub:** name "just for convenience".
- **One hub per source system:** hub_crm_customer and hub_web_customer defeats integration.
- **Inconsistent hashing** across pipelines, so keys never match.

Checkpoint

The ERP is replaced next year and the new system renumbers products. What happens to hub_product?

Answer: new SKUs arrive as new hub rows; old ones stay. A same as link (or a mapping satellite) records old SKU to new SKU, so history joins across the migration without rewriting anything.

SATELLITES · 1 OF 10 · DEFINITION

Satellite: descriptive attributes with complete history

A **satellite** stores the descriptive attributes of exactly one hub or link, keyed by the parent's hash key and a **load date**. Each time any attribute changes, a new row is inserted, so every past state remains queryable.

One parent

A satellite hangs off a single hub or link, never two.

Insert on change

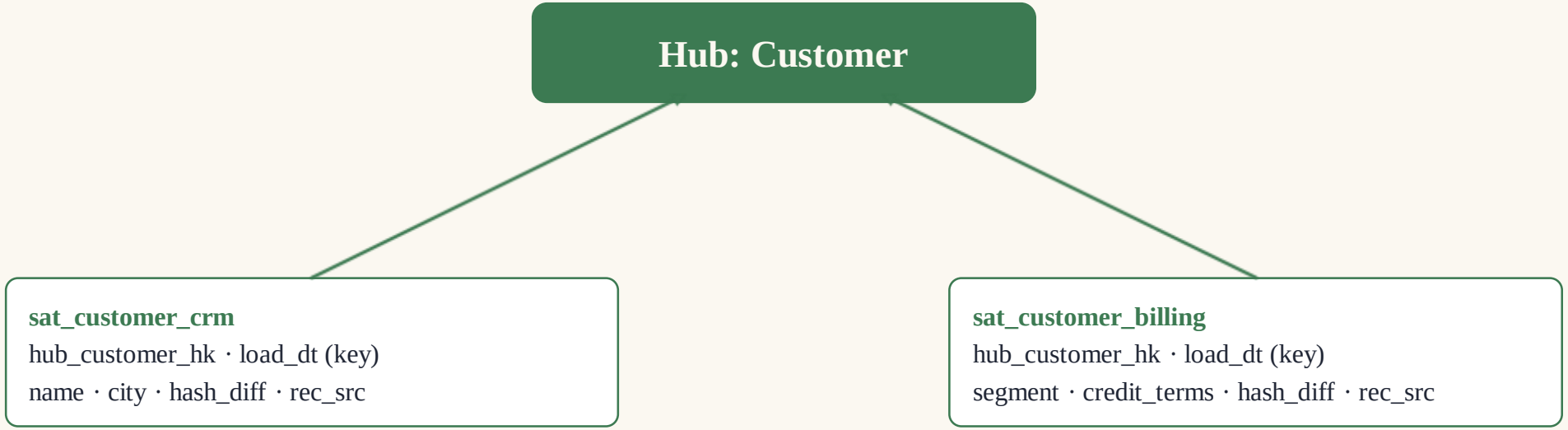
Unchanged data is not reloaded; a changed attribute adds a row.

Split by source and rate

Typically one satellite per source system, sometimes split further by how often attributes change.

Intuition: a satellite is a diary kept about one thing by one witness. Each entry records what that witness said, and when the warehouse heard it.

Anatomy: one hub, two satellites, split by source



Composite key: parent hash key plus load date. Ana appears once per change.

hash_diff: a hash of all descriptive columns, used to detect change in one comparison.

Why two satellites? Different sources and different change rates. A billing change never rewrites CRM rows.

Ana's life story, as two satellites recorded it

sat_customer_crm

hub_hk	name	city	load_dt	hash_diff	rec_src
a1f3...	Ana	Boston	2021/01/01	7d1...	crm
a1f3...	Ana	Austin	2023/06/01	b20...	crm
a1f3...	Ana	Denver	2025/02/15	c94...	crm

sat_customer_billing

hub_hk	segment	load_dt	rec_src
a1f3...	SMB	2021/01/01	erp
a1f3...	ENT	2024/05/01	erp

Five rows, two clocks. City and segment changed at different times; each satellite only grew when its own data changed.

Question: what was Ana's segment and city on 1 Jan 2024? Austin, SMB.

DDL and the hash_diff technique

```
CREATE TABLE raw_vault.sat_customer_crm (
  hub_customer_hk CHAR(32)    NOT NULL,
  load_dt         TIMESTAMP   NOT NULL,
  hash_diff       CHAR(32)    NOT NULL,
  rec_src         VARCHAR(20) NOT NULL,
  name VARCHAR(80), city VARCHAR(40),
  PRIMARY KEY (hub_customer_hk, load_dt)
);
```

```
hash_diff = MD5( COALESCE(name,'') || '|' || COALESCE(city,'') )
```

No end date column. Insert only means rows are never updated to close them; end dates are derived at query time or in a PIT table.

hash_diff: compare one 32 character value instead of every column, which matters when a satellite has 80 attributes.

COALESCE stops a NULL from making the whole hash NULL and hiding a change.

Optional satellite flavours: multi active (several rows valid at once, such as phone numbers), effectivity (start and end of a link), record tracking (key seen in a load).

Satellite load: insert only when the hash_diff changed

```
INSERT INTO raw_vault.sat_customer_crm
SELECT s.hub_customer_hk, s.load_dt, s.hash_diff,
       'crm', s.name, s.city
FROM staging.crm_customer_hashed s
LEFT JOIN (
  SELECT hub_customer_hk, hash_diff,
    ROW_NUMBER() OVER (PARTITION BY hub_customer_hk
      ORDER BY load_dt DESC) AS rn
  FROM raw_vault.sat_customer_crm ) cur
  ON cur.hub_customer_hk = s.hub_customer_hk AND cur.rn = 1
WHERE cur.hub_customer_hk IS NULL
   OR cur.hash_diff <> s.hash_diff;
```

Step 1: find the current row of each customer (rn = 1, latest load date).

Step 2: brand new customer (no current row)? Insert.

Step 3: hash_diff differs from the current row? Something changed. Insert.

Otherwise: identical data, skip. Daily full extracts add zero rows on quiet days.

The hash key and hash_diff are computed once in staging so every vault table can load from the same prepared rows, in parallel.

Querying history: current state and point in time

Current state

```
SELECT h.customer_id, s.city
FROM hub_customer h
JOIN sat_customer_crm s
  ON s.hub_customer_hk = h.hub_customer_hk
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY s.hub_customer_hk
  ORDER BY s.load_dt DESC) = 1;
```

Result: Ana · Denver. The latest row per key.

As of 1 January 2024

```
SELECT h.customer_id, s.city
FROM hub_customer h
JOIN sat_customer_crm s
  ON s.hub_customer_hk = h.hub_customer_hk
WHERE s.load_dt <= TIMESTAMP '2024/01/01'
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY s.hub_customer_hk
  ORDER BY s.load_dt DESC) = 1;
```

Result: Ana · Austin. Latest row on or before the date.

QUALIFY filters on a window function (Snowflake, BigQuery, Databricks). Elsewhere, wrap the query and filter rn = 1 outside. Date literal format follows your engine.

SATELLITES · 7 OF 10 · STRENGTHS

What satellites make possible

Total recall

Every state of every attribute, from every source, forever.

Audit ready

"What did CRM say about Ana on 3 March?" has an exact answer.

Add sources freely

A new system is a new satellite. Nothing existing is altered.

Efficient deltas

Only changes are stored, even from full extracts.

Privacy isolation

Put personal data in its own satellite for separate security and erasure.

Raw truth kept

Business rules apply later, so they can be changed and rerun.

What satellites cost at query time

Weakness	Effect	Mitigation
Window functions everywhere	Every "current" or "as of" query must rank rows	Current views, PIT tables
Satellite sprawl	Ten sources times twenty hubs can mean two hundred satellites	Naming standards, automation
Timeline alignment	Two satellites change at different moments; combining them is a temporal join	PIT tables
Raw values	Codes like S and E are not yet harmonised	Business vault satellites
Unbounded growth	Volatile attributes in a big satellite bloat it	Split fast changing columns out

Balanced verdict: satellites push complexity from load time to query time. That is the right trade for an integration layer, provided nobody points a dashboard straight at it.

Point in time tables: aligning many satellite timelines

pit_customer · one row per customer per snapshot date

hub_hk	snapshot	crm_load_dt	billing_load_dt
a1f3...	2023/12/31	2023/06/01	2021/01/01
a1f3...	2024/06/30	2023/06/01	2024/05/01
a1f3...	2025/06/30	2025/02/15	2024/05/01

Each cell is a pointer to the satellite row valid at that snapshot. Queries become plain equality joins, no window functions.

Use satellites when

History, audit and multi source integration matter.

Avoid querying them directly when

Users need speed and simplicity; build PITs and marts.

Bridge tables do the same for chains of links: precomputed paths across hubs.

Satellite traps, and a question to test you

Mixing sources in one satellite: CRM and web overwrite each other's rows.

Load timestamps from the source: load_dt must be the warehouse's own arrival time.

Including volatile columns such as last_login in the hash_diff: a new row every day.

Applying business rules in the raw satellite, so the raw truth is lost.

Checkpoint

The product table has a price that changes weekly and a description that changes once a year. One satellite or two?

Answer: two: sat_product_price and sat_product_desc. Splitting by rate of change stops the long description being copied 52 times a year just because the price moved.

Link: relationships between business keys, captured flexibly

A **link** records a unique association between two or more hubs: one row per distinct combination of their hash keys, with its own hash key, load date and record source. Links are always **many to many**, whatever the source says.

Always M : N

If the business changes a 1 : N rule to M : N, nothing in the model changes.

Keys only

No descriptive columns; those go in a link satellite.

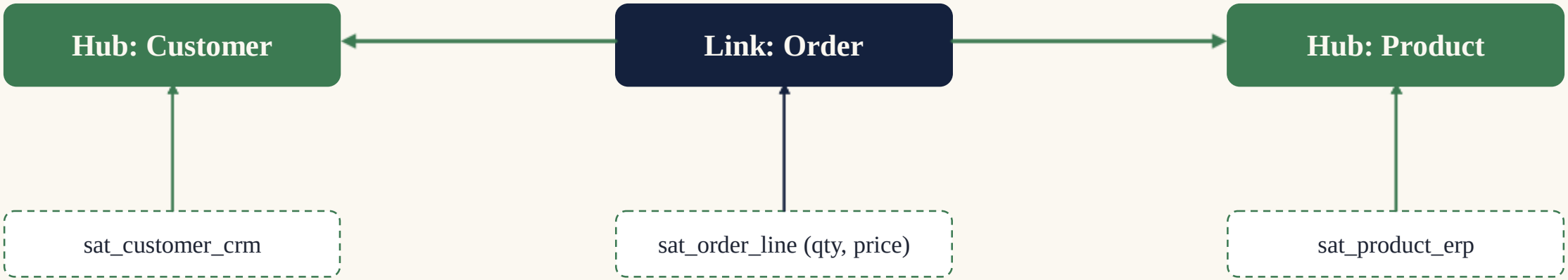
Insert only

An association once seen is never deleted; effectivity satellites say when it ended.

Intuition: if hubs are the nodes of a graph, links are its edges. The vault is a graph of business keys whose shape can grow without rewiring existing edges.

Anatomy: the full vault pattern in one picture

The link holds only hash keys of the hubs it connects, plus its own hash key and metadata.



Link satellite: quantity and price describe the association, not either hub, so they hang off the link. Exactly the logic of the associative entity on slide 21.

link_order_product with its satellite

link_order_product

link_key	hub_cust_key	hub_prod_key	load_dt	rec_src
77ab...	a1f3...	p410...	2024/01/03	pos
81c0...	a1f3...	p522...	2024/01/05	pos

sat_link_order_product

link_key	load_dt	order_id	qty	revenue
77ab...	2024/01/03	1001	1	420.00
81c0...	2024/01/05	1002	2	60.00

Same rows as the infographic: 77ab and 81c0 link Ana (a1f3) to p410 and p522.

Simplification: a fuller design would include hub_order, making this a three hub link at line grain.

DDL for a three hub transactional link

```
CREATE TABLE raw_vault.link_order_line (
  link_order_line_hk CHAR(32) NOT NULL PRIMARY KEY,
  hub_order_hk       CHAR(32) NOT NULL,
  hub_customer_hk    CHAR(32) NOT NULL,
  hub_product_hk     CHAR(32) NOT NULL,
  line_no            SMALLINT NOT NULL,
  load_dt            TIMESTAMP NOT NULL,
  rec_src            VARCHAR(20) NOT NULL
);
```

Link hash key is the hash of all participating business keys, in a fixed order.

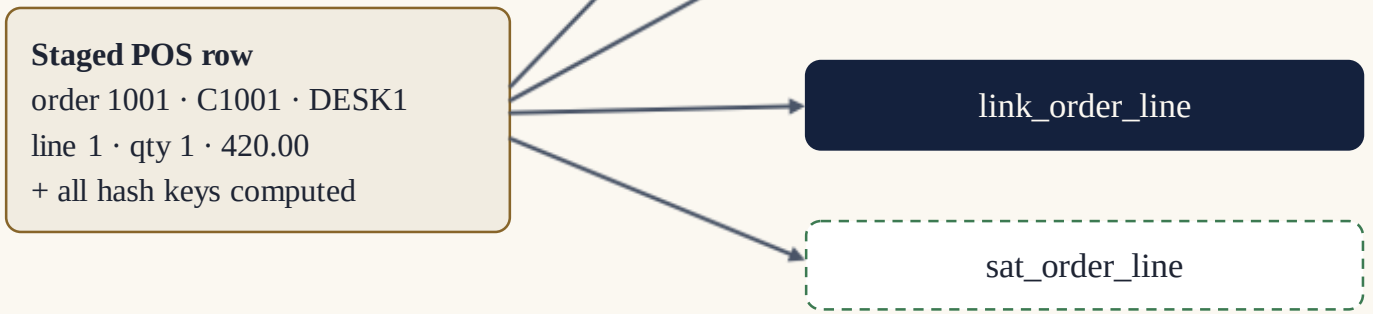
line_no is a dependent child key: it is part of the link's grain but has no hub of its own.

No REFERENCES clauses: many vault platforms skip enforced foreign keys to allow parallel loads; tests verify integrity instead.

```
link_order_line_hk = MD5(order_id || '||' || customer_id
  || '||' || sku || '||' || line_no)
```

Hubs, links and satellites can all load at once because every key is computed, not looked up.

One staged row fans out to hubs, links and satellites in parallel



Hub rule: insert keys not yet present.

Link rule: insert key combinations not yet present (same NOT EXISTS pattern on the link hash key).

Satellite rule: insert when hash_diff changed.

Three templates load the entire vault. Every table is one of three shapes, so every load is one of three generated statements. A new source adds rows to a configuration file, not new hand written logic.

Building fact_sales from the vault

```
CREATE VIEW mart.fact_sales AS
SELECT hc.customer_id, hp.sku,
       s.order_date, s.quantity, s.revenue
FROM raw_vault.link_order_line l
JOIN raw_vault.hub_customer hc
  ON hc.hub_customer_hk = l.hub_customer_hk
JOIN raw_vault.hub_product hp
  ON hp.hub_product_hk = l.hub_product_hk
JOIN raw_vault.sat_order_line s
  ON s.link_order_line_hk = l.link_order_line_hk
QUALIFY ROW_NUMBER() OVER (PARTITION BY s.link_order_line_hk
  ORDER BY s.load_dt DESC) = 1;
```

The link is the fact's spine: one row per order line, exactly the star's grain.

Hubs supply business keys, which the mart swaps for dimension surrogates.

The link satellite supplies measures, latest version only, so corrections in the source flow through.

Pattern: link → fact; hub plus satellites → dimension.

LINKS · 7 OF 10 · STRENGTHS

Why links make the vault resilient

Cardinality proof

"Each order has one customer" can become "orders can be shared" with zero schema change.

New relationships, new tables

Adding "customer referred customer" is one new link, nothing else touched.

Cross source joins

A link loaded from POS can join hubs first populated by CRM and ERP.

Relationship history

With effectivity satellites, "who managed store 7 in 2022" is answerable.

Key insight: in 3NF a foreign key column freezes a cardinality decision into a table. In a vault the decision lives in data, so it can change.

What links, and vaults overall, cost you

Weakness	Effect	Mitigation
Table explosion	A 3NF model of 50 tables can become 150 vault tables	Automation, naming standards
Many joins	Every hop goes hub, link, hub, plus satellites	PIT and bridge tables, marts
Cardinality not enforced	Bad data with two customers per order is accepted	Data quality tests
Grain mistakes	Wrong link grain silently loses repeat transactions	Include transaction keys
Skills and tooling	Needs trained modellers and generators	Certified training, packages

Balanced verdict: Data Vault is an investment that pays back in large, changing, regulated estates. For a startup with three sources it is usually overkill; a star built straight from staging serves better.

Link variants, and when a Data Vault is the right choice

Variant	Purpose
Standard link	Association between two or more hubs
Same as link	Two keys in one hub denote the same thing (C1001 and ana@mail.com)
Hierarchical link	Parent and child keys in one hub (employee reports to employee)
Non historised link	Immutable events such as payments, with measures stored on the link row

Use a vault when

Many sources, frequent change, audit or regulation, long lived platform, automation available.

Avoid it when

Few stable sources, a small team, no history requirement, or users who would query it directly.

Link traps, and the family summary

Link to link references: links connect hubs only.

Descriptive columns on links instead of in a link satellite.

Too coarse a grain, dropping repeat purchases.

Checkpoint

Where does "discount applied to this line" live?

Answer: in a satellite on link_order_line: it describes the association, not a hub.

Data Vault in one breath

Hubs: business keys, once, forever.

Satellites: descriptions, every version, per source.

Links: associations, always many to many.

Insert only, hash keyed, pattern loaded, built to absorb change; served to users through marts.

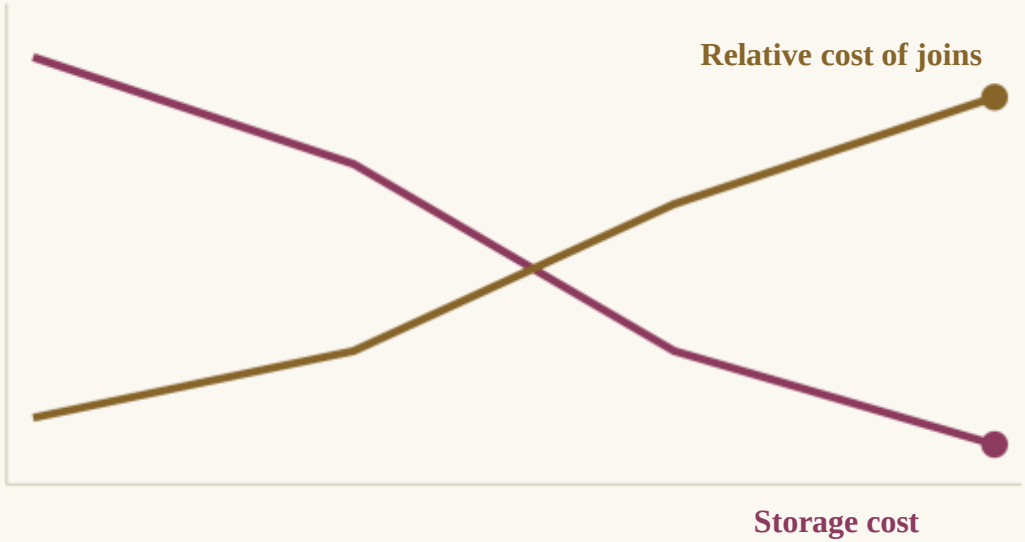
PART 5 OF 9 · FAMILY 4 OF 6

Wide and Flat Models

When storage is cheap and columns are read independently, joins become the expensive part. This family pre joins, flattens and nests data so that queries scan instead of join.

One Big Table | Activity Schema | Nested or Struct Array

The problem this family solves: joins are the bottleneck



1990s row stores → 2010s cloud columnar → today

Then: disk was expensive and rows were read whole, so normalising to save bytes made sense.

Now: storage is cheap, columns compress, and distributed joins shuffle data across machines. Avoiding joins is the new optimisation.

Three answers: pre join everything (OBT), standardise on one event stream (activity), or store children inside parents (nested).

OBT, Activity Schema and Nested side by side

Property	One Big Table	Activity Schema	Nested Struct Array
Shape	Every attribute prejoined into one wide row	One narrow time series of activities	Child rows inside parent rows
Grain	Usually one fact row	One activity by one entity	One parent; children nested
Joins at query	Zero	Self joins in time	UNNEST instead of join
Schema change	Add columns, rebuild	New activity names, no DDL	Add struct fields
Best question	Any slice of a known process	Funnels, retention, attribution	Orders with their items
Infographic cue	order_id, date, customer, segment, product...	visited, signed_up, upgraded	items ARRAY of STRUCT

Common thread: each is a presentation shape built from upstream models. None replaces an integration layer; each replaces joins at read time.

One Big Table: everything prejoined, zero joins at query time

A **One Big Table (OBT)** is a single, wide, fully denormalised table in which each row is a fact at a declared grain, carrying every descriptive attribute of every related dimension as ordinary columns, so that any question about that process can be answered without a join.

Star, collapsed

Take a star and join every dimension into the fact, once, at build time.

Wide by design

Fifty to several hundred columns is normal.

Columnar friendly

Queries read only the columns they name, so width costs little.

Intuition: an OBT is the spreadsheet every business user secretly wants: one sheet, every column they could need, filter and pivot away. The warehouse does the joining once so nobody else has to.

Anatomy: where each OBT column came from

Colour bands show lineage: each run of columns is a former dimension, now embedded. order_id is a degenerate key kept for drill through.



No keys to other tables: surrogate keys disappear. Labels replace them, because nobody will join this table to anything.

obt_sales: three rows that answer a thousand questions

order_id	date	customer	segment	product	category	store	region	revenue
1001	Jan 3	Ana	SMB	Desk	Office	NYC	East	420
1002	Jan 3	Raj	ENT	Chair	Office	SFO	West	180
1003	Jan 4	Mila	SMB	Lamp	Home	NYC	East	60

Revenue by region?
GROUP BY region. East 480, West 180.

SMB share of Office?
Filter category, split segment. 420 of 600.

Best day?
GROUP BY date. Jan 3 with 600.

Every answer is a filter and a GROUP BY on one table. This is why OBTs dominate self service tools and drag and drop dashboards: the user never meets the word JOIN.

Building the OBT: the joins happen once, here

```
CREATE OR REPLACE TABLE mart.obt_sales
PARTITION BY order_date
CLUSTER BY region, category AS
SELECT f.order_id, d.order_date, d.fiscal_qtr,
       c.name AS customer, c.segment, c.city,
       p.name AS product, p.category, p.brand,
       s.store_name AS store, s.region,
       f.quantity, f.revenue
FROM fact_sales f
JOIN dim_date     d USING (date_key)
JOIN dim_customer c USING (customer_key)
JOIN dim_product  p USING (product_key)
JOIN dim_store    s USING (store_key);
```

Built from a star, not from raw sources. The OBT is a presentation artefact on top of a proper model.

Partition and cluster on the most common filters. This replaces the role indexes played in row stores.

Rename for humans: c.name becomes customer, so self service users see business language.

Syntax shown is BigQuery style; Snowflake and Databricks differ only in the clustering clause.

The same question in a star and in an OBT

Star · three joins

```
SELECT p.category, c.segment,
  SUM(f.revenue)
FROM fact_sales f
JOIN dim_product p USING (product_key)
JOIN dim_customer c USING (customer_key)
JOIN dim_date d USING (date_key)
WHERE d.year = 2024
GROUP BY 1, 2;
```

OBT · zero joins

```
SELECT category, segment,
  SUM(revenue)
FROM mart.obt_sales
WHERE order_date
  BETWEEN DATE(2024,1,1)
  AND DATE(2024,12,31)
GROUP BY 1, 2;
```

Same answer. Shorter SQL, no join mistakes possible.

Partition pruning on order_date skips every other year's files.

Only three columns read out of the table's many: columnar storage at work.

Keeping an OBT fresh: three refresh strategies

Full rebuild

Recreate the whole table nightly. Simplest; always consistent with dimensions. Cost grows with history.

Incremental append

Insert only new fact rows. Cheap, but old rows keep old dimension values.

Partition overwrite

Rebuild only recent partitions, such as the last seven days, to catch late data.

The hidden modelling decision: "as was" or "as is"?

Ana moved to Austin. Incremental loading leaves her 2022 rows saying **Boston** (as was, like SCD Type 2). A full rebuild from a Type 1 dimension rewrites them all to **Austin** (as is). Choose deliberately and document it in the table description.

Many teams publish both: customer_city_at_sale and customer_city_current as two columns, so users pick the meaning they need.

Why OBTs became popular on modern stacks

Self service

One table to understand; drag and drop tools work instantly.

Fast scans

No shuffle joins across nodes; pruning and compression do the work.

No join errors

Fan traps and wrong keys are impossible when there is nothing to join.

ML ready

One row per example with every feature as a column: a training set.

Export friendly

One file for reverse ETL, spreadsheets or partners.

Cache friendly

BI extracts and result caches thrive on a single source table.

What an OBT costs you

Weakness	Effect	Mitigation
One process only	Combining sales and inventory still needs two OBTs and care	One OBT per process
Duplicated logic	Customer attributes copied into every OBT may drift	Build all OBTs from shared dimensions
Costly updates	A renamed category rewrites millions of rows	Partition overwrite, rebuild
Grain lock	Customer level questions on a line level OBT double count	COUNT DISTINCT, separate OBTs
Column sprawl	Hundreds of columns become hard to document and govern	Naming prefixes, catalog

Balanced verdict: an OBT built on top of a dimensional model is excellent. An OBT built directly from raw sources, with no model beneath it, becomes an unmaintainable copy of business logic.

Choose an OBT when, and avoid it when

Use it when

- The engine is columnar and joins are the bottleneck
- Users are non technical or use drag and drop tools
- A dashboard or model needs one specific shape
- You need a flat extract or ML training table

Avoid it when

- It would be the only model you have
- Dimensions change often and history matters
- Many processes must be compared together
- The engine is a row store (OLTP database)

Aster decision: obt_sales for store managers' self service dashboards, generated nightly from the sales star. Analysts with SQL skills keep using the star.

OBT traps, and a question to test you

- **Joining a one to many child in:** adding order lines to an order level OBT multiplies order revenue.
- **Precomputed ratios** such as margin %, which users then SUM.
- **Undocumented as is / as was** semantics on embedded attributes.
- **OBT per dashboard** until fifty near duplicates exist.

Checkpoint

An OBT at order line grain has order_shipping_fee repeated on every line. A user sums it by month. What goes wrong and how do you fix the design?

Answer: shipping is counted once per line, overstating it. Allocate it across lines (shipping_fee_allocated) or keep it in a separate order grain OBT.

Activity schema: one time series table for every customer action

An **activity schema** models a business as a single append only table of **activities**, each row recording that one **entity** (usually a customer) did one named thing at one timestamp, with a small fixed set of feature columns. Analyses are built by relating activities to each other in time.

Fixed columns

The same columns for every activity type, forever.

Verbs as data

visited, signed_up, ordered, upgraded are values, not tables.

Temporal joins

"First visit before signup" replaces key joins.

Intuition: instead of a table per noun, keep one diary per customer. Every question about journeys becomes "find activity A, then the first or last activity B after or before it".

Anatomy: Ana's journey as a stream of activities



feature_1: page /pricing
feature_2: source ads

feature_1: plan free

revenue_impact: 420
link: order 1001

feature_1: pro · **feature_2:** annual
revenue_impact: 480

Standard columns (per the open spec): activity_id · ts · customer · activity · feature_1 · feature_2 · feature_3 · revenue_impact · link · activity_occurrence · activity_repeated_at

activity_occurrence numbers each customer's 1st, 2nd, 3rd time doing an activity; activity_repeated_at points to the next occurrence. Both make "first" and "next" queries trivial.

customer_stream: the whole business in one table

id	customer	ts	activity	feature_1	feature_2	revenue	occurrence
a1	Ana	Jan 2 09:02	visited	/pricing	ads		1
a2	Ana	Jan 2 09:10	signed_up	free			1
a3	Ana	Jan 3 14:20	ordered	DESK1	web	420	1
a4	Ana	Jan 9 11:45	upgraded	pro	annual	480	1
a5	Raj	Jan 2 16:30	visited	/home	search		1
a6	Raj	Jan 5 10:00	visited	/pricing	email		2

Web, CRM, POS and billing all land in the same columns. No new table for any of them.

Raj's occurrence 2 shows a repeat visit; he has not signed up yet. A funnel will count him at the first step only.

DDL: one table, designed never to change shape

```
CREATE TABLE analytics.customer_stream (
  activity_id          VARCHAR(64) NOT NULL,
  ts                   TIMESTAMP   NOT NULL,
  customer             VARCHAR(64),
  anonymous_customer_id VARCHAR(64),
  activity             VARCHAR(64) NOT NULL,
  feature_1, feature_2, feature_3  VARCHAR,
  revenue_impact       DECIMAL(12,2),
  link                 VARCHAR,
  activity_occurrence  INTEGER,
  activity_repeated_at TIMESTAMP
)
PARTITION BY DATE(ts) CLUSTER BY activity, customer;
```

anonymous_customer_id holds cookie or device ids before signup, so pre identity activity can be stitched to the customer later.

Generic VARCHAR features keep the shape fixed. Numbers needing arithmetic go in revenue_impact or are cast.

Cluster by activity because nearly every query begins "WHERE activity = ...".

The column list is shortened to one line for features for display.

Funnel: first visit to signup within seven days

```
WITH first_visit AS (
  SELECT customer, ts FROM customer_stream
  WHERE activity = 'visited'
    AND activity_occurrence = 1 ),
signup AS (
  SELECT customer, ts FROM customer_stream
  WHERE activity = 'signed_up'
    AND activity_occurrence = 1 )
SELECT COUNT(*) AS visitors,
       COUNT(s.customer) AS converted
FROM first_visit v
LEFT JOIN signup s
  ON s.customer = v.customer
 AND s.ts BETWEEN v.ts AND v.ts + INTERVAL '7' DAY;
```

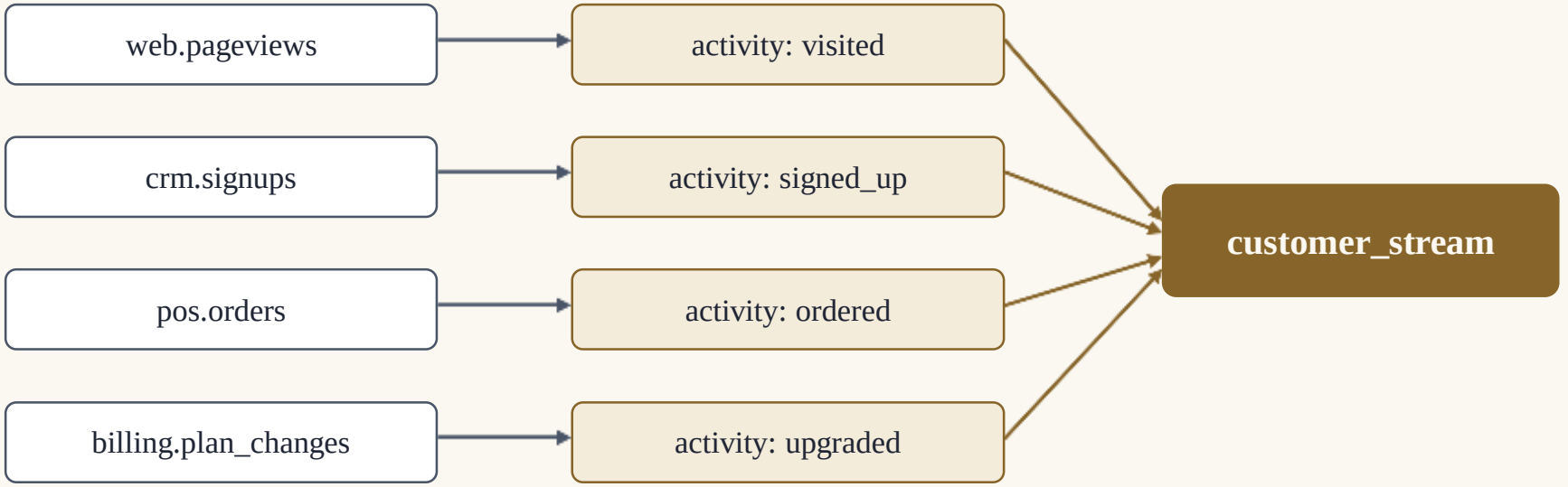
Both CTEs read the same table, filtered by activity name. That is the whole data model.

The join is temporal: same customer, and signup within the window after the visit.

Result on our sample: 2 visitors, 1 converted (Ana). 50%.

Other relationships: first ever, last before, first in between, aggregate in between. Every journey question is one of these.

One small transformation per activity, unioned into the stream



Each activity is its own small model mapping source columns to the fixed shape.

Then UNION ALL, compute occurrence and repeated_at with window functions.

Adding a new source means writing one more small activity transform. No schema change, no rework of existing analyses, which automatically gain the new activity as an option.

What the activity schema does brilliantly

Journey questions, natively

Funnels, retention cohorts, time between steps and multi touch attribution are all native.

Schema never changes

New behaviour is new data, so pipelines and dashboards keep working.

Cross source by default

Web clicks, support tickets and payments sit side by side on one timeline.

Reusable query patterns

A handful of temporal templates answer most product analytics questions.

Key insight: star schemas model state at a grain; activity schemas model sequence. When the question is about order of events, sequence wins.

What the activity schema costs you

Weakness	Effect	Mitigation
Opaque features	feature_1 means a page here, a plan there	An activity dictionary
Weak for state	"Stock on hand today" is not an activity	Keep snapshot facts elsewhere
Heavy self joins	Temporal joins on billions of rows can be expensive	Clustering, occurrence columns
Loose typing	Numbers stored as text need casting	Typed revenue_impact, conventions
Entity centric only	Questions about products or stores need another stream	Multiple streams
Less familiar	BI tools expect stars, not streams	Materialise datasets for BI

Balanced verdict: superb for behavioural analytics on one entity; a poor fit for financial reporting, inventory or anything measured as state rather than event.

Choose an activity schema when, and avoid it when

Use it when

- Product, growth or marketing analytics
- Questions about order and timing of behaviour
- Many event sources about one kind of entity
- New behaviours appear every sprint

Avoid it when

- Reporting balances, inventory or positions
- Rich descriptive attributes drive analysis
- Regulators need fully typed, audited tables
- Users only know drag and drop BI

Aster decision: customer_stream for the online team's acquisition funnel and retention cohorts; the sales star remains the finance source.

Activity traps, and a question to test you

Too many activity names: "clicked_blue_button_v2". Model business actions, not UI events.

Changing a feature's meaning for an existing activity without a new name.

Time zone chaos: mixing local and UTC timestamps breaks ordering.

Ignoring identity stitching: funnels undercount pre signup steps.

Checkpoint

Marketing asks: "Which acquisition source gets the credit for each upgrade?" Which temporal relationship answers it?

Answer: "last before": for each upgraded activity, find the last visited activity before it and read its feature_2 (source). "First ever" visit gives first touch attribution instead.

Nested model: one to many data stored inside the parent row

A **nested** or **struct array** model stores child records as a typed **ARRAY of STRUCT** column inside their parent row, so an order carries its lines with it. Children are expanded with **UNNEST** only when a query needs them.

STRUCT

A named group of typed fields, like a mini row: sku, qty, price.

ARRAY

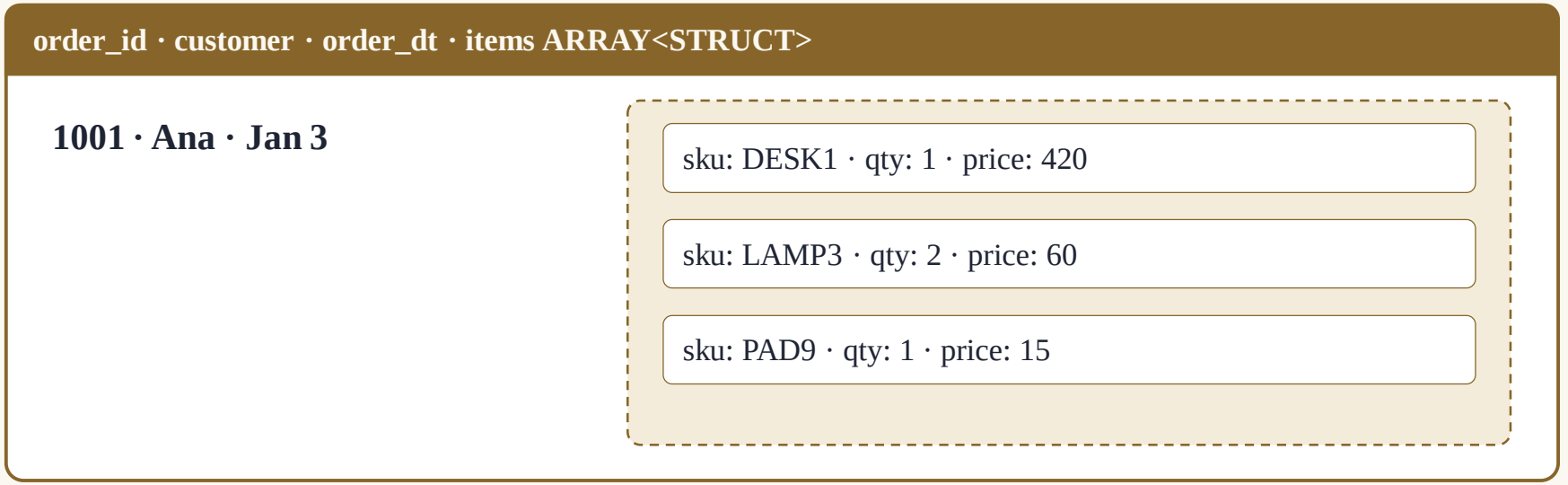
An ordered list of values of one type, here a list of structs.

UNNEST

Turns an array back into rows at query time.

Intuition: a paper receipt. The order header and its line items are printed on one slip, not filed in two drawers that must be matched again later.

Anatomy: child rows live inside the parent row



One row in the table, three lines inside it. Order level counts need no DISTINCT.

Each struct is typed: qty is an integer, price a decimal, enforced by the engine.

UNNEST(items) at query time produces three rows when line detail is needed.

Two tables versus one nested table, row for row

Relational: orders plus order_lines (4 rows total)

order_id	customer	order_dt
1001	Ana	Jan 3

order_id	sku	qty	price
1001	DESK1	1	420
1001	LAMP3	2	60
1001	PAD9	1	15

Nested: orders_nested (1 row)

```
{ order_id: 1001,
  customer: "Ana",
  order_dt: "Jan 3",
  items: [
    {sku: "DESK1", qty: 1, price: 420},
    {sku: "LAMP3", qty: 2, price: 60},
    {sku: "PAD9",  qty: 1, price: 15} ] }
```

Same information, different physical locality: the nested row keeps parent and children together on disk, so reading an order with its lines needs no join and no shuffle.

DDL: declaring a typed array of structs

```
CREATE TABLE mart.orders_nested (
  order_id   INT64 NOT NULL,
  customer   STRING,
  order_dt   DATE,
  items      ARRAY<STRUCT<
               sku   STRING,
               qty   INT64,
               price NUMERIC>>
)
PARTITION BY order_dt;
```

The schema is still strict: every struct field has a type. This is not a free text blob.

Columnar inside the array: engines store items.sku, items.qty and items.price as separate column chunks, so reading prices skips SKUs.

Nesting can go deeper: items could hold an ARRAY of discounts. Two levels is usually the sensible maximum.

Dialects: BigQuery ARRAY<STRUCT> as shown · Databricks ARRAY<STRUCT<...>> · Snowflake ARRAY or VARIANT with semi structured paths · Redshift SUPER.

Two query styles: stay nested, or UNNEST

Order level, no unnesting

```
SELECT order_id,
  ARRAY_LENGTH(items) AS n_lines,
  (SELECT SUM(i.price)
   FROM UNNEST(items) i) AS total
FROM mart.orders_nested;
```

Result: 1001 · 3 lines · 495. The subquery aggregates inside each row; the outer row count never changes.

Line level, flattened

```
SELECT i.sku,
  SUM(i.qty)   AS units,
  SUM(i.price) AS revenue
FROM mart.orders_nested o,
    UNNEST(o.items) AS i
GROUP BY i.sku;
```

Result: DESK1 1 · 420; LAMP3 2 · 60; PAD9 1 · 15. The comma join with UNNEST is a correlated join to the row's own array.

Fan out warning: after UNNEST, parent columns repeat per child. SUM(order_shipping_fee) would now count shipping three times.

Two loading routes: ingest nested, or nest relational data

Route A · sources are already nested

JSON events, API payloads, MongoDB documents and Avro or Parquet files arrive with arrays inside. Load them directly into STRUCT and ARRAY columns, preserving shape and typing them on the way in.

Benefit: no shredding step; the bronze layer mirrors the source exactly.

Route B · nest from tables

```
SELECT o.order_id, o.customer, o.order_dt,
  ARRAY_AGG(STRUCT(l.sku, l.qty, l.price)
    ORDER BY l.line_no) AS items
FROM orders o
JOIN order_lines l USING (order_id)
GROUP BY 1, 2, 3;
```

ARRAY_AGG is the inverse of UNNEST: it folds child rows back into their parent.

Updates are whole row rewrites: changing one line means rewriting the order's row. Nested tables suit immutable or rarely updated parents.

What nesting buys you

No join, no shuffle

Parent and children are co located, so distributed engines never move data to pair them.

Grain stays honest

One row per order, yet line detail is available. Order counts cannot be inflated.

Mirrors modern sources

APIs and event payloads are nested already; no lossy flattening on ingest.

Fewer tables

Header plus lines plus discounts becomes one object to govern and document.

Key insight: nesting solves the fan out problem of OBTs. An OBT must choose order grain or line grain; a nested table offers both without duplicating parent values.

What nesting costs you

Weakness	Effect	Mitigation
Unfamiliar SQL	UNNEST, ARRAY_AGG and correlated syntax confuse many analysts	Flattened views
BI tool support	Many tools cannot display arrays	Expose UNNESTed views
Portability	Syntax differs across BigQuery, Snowflake, Spark	Abstraction in dbt macros
Child updates	Updating one item rewrites the whole parent row	Use for immutable data
Child to child joins	Relating items of different orders is awkward	Keep a flat line table too
Unbounded arrays	A customer with 100,000 events in one row is impractical	Nest bounded children only

Balanced verdict: nest children that are small, bounded, owned by one parent and read with it. Keep unbounded or shared children in their own table.

Nest when, and stay flat when

Nest when

- The child is owned by exactly one parent
- Children are few and bounded
- They are usually read with the parent
- The source is JSON or document shaped
- The engine supports nested types natively

Stay flat when

- Children are shared (products across orders)
- Children change independently and often
- Users rely on BI tools without array support
- You need to port across many engines

Aster decision: orders_nested for the online order API and data science features; the star's line level fact remains for finance.

Nesting traps, and the family summary

Summing parent columns after UNNEST.

Inner UNNEST dropping parents with empty arrays; use LEFT JOIN UNNEST.

Storing JSON strings instead of typed structs.

Checkpoint

Should product details (name, category) be nested inside each item struct?

Answer: only a snapshot at sale time, if needed. Products are shared, so the master copy stays in its own dimension.

Wide and Flat in one breath

OBT: every attribute prejoined; zero joins for one process.

Activity: one stream of verbs; temporal joins for journeys.

Nested: children inside parents; UNNEST when needed.

All three trade storage and update cost for read simplicity on columnar engines.

PART 6 OF 9 · FAMILY 5 OF 6

Temporal Modelling

Slowly changing dimensions: three standard answers to one question. When Ana moves from Boston to Austin, what should last year's report say?

SCD Type 1 · overwrite

SCD Type 2 · new row

SCD Type 3 · previous column

TEMPORAL · FAMILY OVERVIEW

The problem: facts are fixed in time, dimensions are not

fact_sales row, 2022
customer_key → Ana
revenue 420

which version of Ana?

dim_customer, today
Ana · city ?

Type 1 says

"Austin. History is rewritten to today's truth."

Type 2 says

"Boston. Each period keeps its own row."

Type 3 says

"Austin now, Boston before: both, side by side."

None is wrong. Each answers a different business question. The modelling skill is choosing per attribute, not per table.

SCD Types 1, 2 and 3 side by side

Property	Type 1	Type 2	Type 3
Mechanism	UPDATE in place	Close old row, INSERT new	Shift value to a previous column
Rows per key	Always one	One per version	Always one
History kept	None	Complete	One step back
Facts link to	The only row	The version valid at event time	The only row
Answers	What is true now?	What was true at the time?	What changed from what?
Complexity	Lowest	Highest	Low
Typical use	Corrections, typos	Territory, segment, address	Reorganisations

Remember with the infographic: Type 1 overwrites; Type 2 adds a row with valid_from, valid_to, is_current; Type 3 keeps the previous value in a dedicated column next to the current one.

SCD Type 1: overwrite, and keep no history

In an **SCD Type 1** attribute, a change in the source simply **overwrites** the value in the existing dimension row. The dimension always shows the current value, and every fact, past or present, is reported against it.

One row per key

The surrogate and natural key stay one to one forever.

"As is" reporting

All history is restated in today's terms.

Lowest complexity

One MERGE statement; no date logic.

Intuition: correcting a typo in an address book with an eraser. Once rubbed out, the old spelling is gone, and nobody needs it back.

Anatomy: before and after one overwrite

Before

cust_key	name	city
1	Ana	Boston



UPDATE

After

cust_key	name	city
1	Ana	Austin

Same key, same row. cust_key 1 never changes, so no fact needs to be touched.

Boston is gone. Nothing in the warehouse remembers it ever existed.

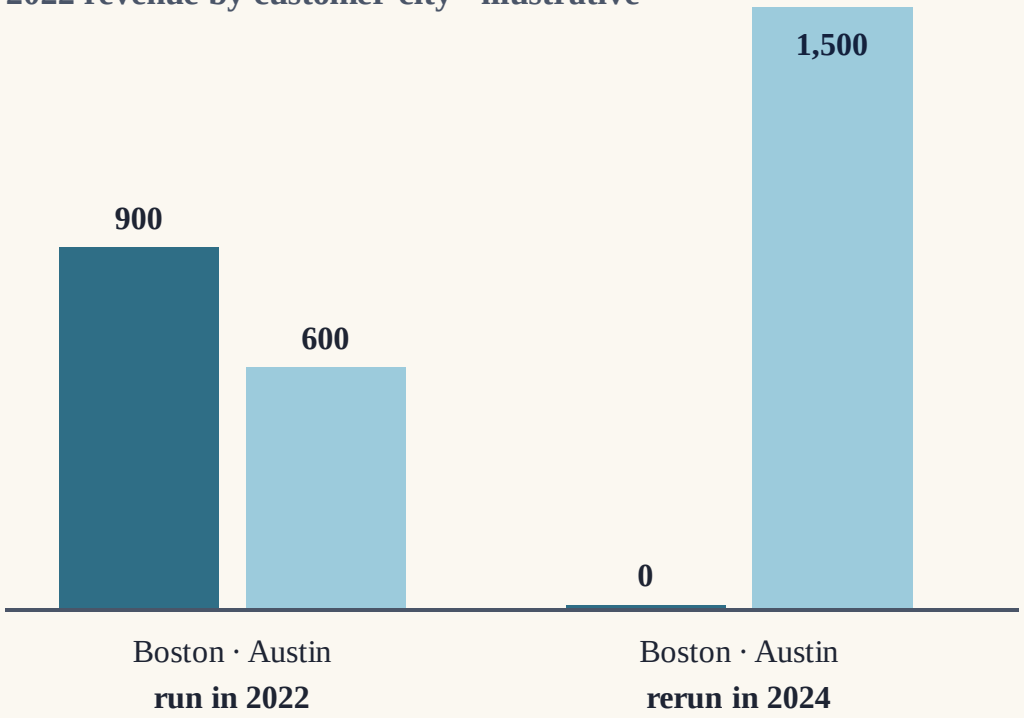
All past facts move: 2022 Boston sales now report as Austin sales.

This is the infographic's Type 1 panel: cust_key 1, Ana, Boston, then the update, then Ana, Austin, with the note "Boston is gone".

SCD TYPE 1 · 3 OF 10 · WORKED EXAMPLE

The 2022 report, run before and after Ana's move

2022 revenue by customer city · illustrative



Same query, same year, different answer. Ana's 900 of 2022 spend moved from Boston to Austin when her row was overwritten.

Historical reports are not reproducible under Type 1. A board pack from 2022 no longer reconciles with the warehouse.

When this is right: if the 2022 city was a typo ("Bostn"), restating history is exactly what you want.

Dark bars: Boston. Light bars: Austin. Values are invented for illustration.

DDL: a Type 1 dimension needs nothing special

```
CREATE TABLE dim_customer_t1 (
  cust_key      INTEGER PRIMARY KEY,
  customer_id VARCHAR(10) NOT NULL UNIQUE,
  name          VARCHAR(80),
  city          VARCHAR(40),
  updated_at    TIMESTAMP
);
```

UNIQUE on the natural key: guarantees one row per customer, the defining property of Type 1.

updated_at is housekeeping: it says when the row last changed, not what it used to say.

Type 0 for contrast: some attributes must never change once set, such as original signup date or date of birth. Type 0 ignores source updates entirely. Type 1 accepts them and forgets the old value.

One MERGE handles inserts and overwrites

```
MERGE INTO dim_customer_t1 d
USING staging.crm_customer s
  ON d.customer_id = s.customer_id
WHEN MATCHED AND (d.name <> s.name
                  OR d.city <> s.city) THEN
  UPDATE SET name = s.name, city = s.city,
             updated_at = CURRENT_TIMESTAMP
WHEN NOT MATCHED THEN
  INSERT (cust_key, customer_id, name, city, updated_at)
  VALUES (NEXT VALUE FOR seq_cust, s.customer_id,
          s.name, s.city, CURRENT_TIMESTAMP);
```

MATCHED and different: overwrite. Ana's city becomes Austin.

The change test avoids touching unchanged rows, which keeps updated_at meaningful and cuts write cost.

NOT MATCHED: a brand new customer gets a fresh surrogate key.

NULL safety: d.city <> s.city is unknown when either side is NULL. Use IS DISTINCT FROM where supported, or compare hashes.

Querying Type 1: the simplest join there is

```
SELECT c.city,
       SUM(f.revenue) AS revenue
FROM fact_sales f
JOIN dim_customer_t1 c
  ON c.cust_key = f.customer_key
WHERE f.date_key BETWEEN 20220101
                     AND 20221231
GROUP BY c.city;
```

No date logic in the join. Each fact meets exactly one dimension row, guaranteed.

The answer is "2022 sales by where customers live today". Label reports accordingly.

Valid questions: lifetime value of today's Austin customers; current segment mix of all historic revenue.

Invalid question: "How did the Boston store region perform in 2022?" when customers have since moved.

SCD TYPE 1 · 7 OF 10 · STRENGTHS

Why Type 1 is the right default for many attributes

Simplest to build

One MERGE, no versioning logic, no effective dates.

Smallest table

Row count equals entity count; it never grows from changes.

Simplest to query

Plain equality joins; no risk of picking the wrong version.

Fixes propagate

A corrected spelling instantly fixes every historical report.

Key insight: for attributes whose history has no business meaning (typos, formatting, phone extensions), keeping history adds cost and confusion with zero value.

What Type 1 destroys

Weakness	Effect	Mitigation
History lost	Past attribute values are unrecoverable from the dimension	Keep raw history in a vault or bronze layer
Reports drift	Last year's numbers change each time attributes change	Snapshot published reports
Aggregates break	Precomputed summaries by city no longer match details	Rebuild aggregates on change
Audit failure	Cannot show what the warehouse said on a past date	Use Type 2 for audited attributes

Balanced verdict: Type 1 is safe only when the history of the attribute genuinely does not matter, or when full history is preserved somewhere upstream and can be replayed.

SCD TYPE 1 · 9 OF 10 · WHEN TO USE

Choose Type 1 when, and avoid it when

Use it when

- The change is a correction of an error
- Users always want the current label
- History of the attribute has no analytical value
- Personal data must not linger

Avoid it when

- Attributes drive territory, commission or regulation
- Published reports must reproduce exactly
- Analysts ask "at the time of the sale" questions
- No raw history is kept upstream

Aster decision: Type 1 for customer name, email and phone; not for city or segment, which drive regional and commercial reporting.

Type 1 traps, and a question to test you

- **Type 1 by default** because it is easy, discovering a year later that history was needed.
- **Stale aggregates** that were built before the overwrite.
- **NULL comparisons** that silently skip real changes.
- **Unlabelled semantics:** "city" that users assume is historical.

Checkpoint

A product is recategorised from Office to Home. Finance wants last year's category totals unchanged; merchandising wants them restated. Can one Type 1 column satisfy both?

Answer: no. Finance needs Type 2 (or a historical category on the fact); merchandising needs current. Provide both: a Type 2 version for finance and a current_category column (Type 1) for merchandising.

SCD TYPE 2 · 1 OF 10 · DEFINITION

SCD Type 2: a new row for every change

In **SCD Type 2**, a change creates a **new dimension row** with a new surrogate key, while the previous row is closed by setting its **valid_to** date and clearing its **is_current** flag. Facts point to the version valid when they happened.

Many rows per key

One natural key, several surrogate keys: one per version.

Validity window

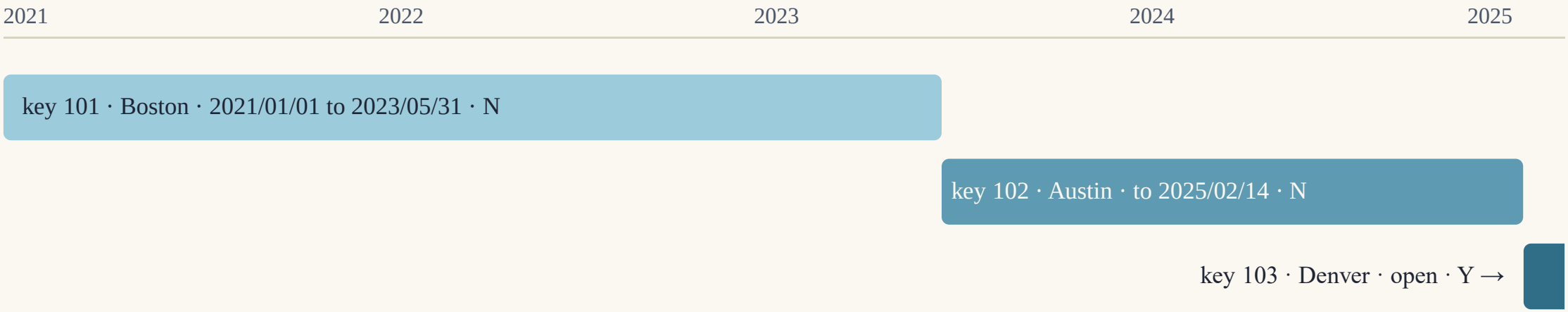
valid_from and valid_to bracket when each version was true.

"As was" reporting

Each fact keeps the context it really had.

The standard answer, in the infographic's words, to "what was true at the time?". Type 2 is the most widely used SCD type in analytical warehouses.

Anatomy: three versions of Ana on one timeline



No gaps, no overlaps. Each version starts where the previous one ended, so any date falls in exactly one row.

The open row has valid_to 9999/12/31 and is_current = Y. There is exactly one per customer.

Three surrogate keys, one natural key C1001 tying them together.

The dimension and the facts that point into it

dim_customer_t2

cust_key	customer_id	name	city	valid_from	valid_to	current
101	C1001	Ana	Boston	2021/01/01	2023/05/31	N
102	C1001	Ana	Austin	2023/06/01	2025/02/14	N
103	C1001	Ana	Denver	2025/02/15	9999/12/31	Y

fact_sales

date_key	cust_key	revenue
20220415	101	900
20240103	102	420
20250610	103	180

Each fact was stamped at load with the version current that day. 2022 revenue stays in Boston forever.

Ana's lifetime total still works: group by customer_id, not cust_key, to get 1,500.

DDL: the four columns that make history work

```
CREATE TABLE dim_customer_t2 (
  cust_key      INTEGER PRIMARY KEY,
  customer_id VARCHAR(10) NOT NULL,
  name VARCHAR(80), city VARCHAR(40),
  valid_from    DATE NOT NULL,
  valid_to      DATE NOT NULL,
  is_current    BOOLEAN NOT NULL,
  row_hash      CHAR(32) NOT NULL,
  UNIQUE (customer_id, valid_from)
);
```

valid_to NOT NULL with a far future date (9999/12/31) keeps BETWEEN and range joins simple.

is_current is redundant with valid_to, but makes "current version" filters cheap and obvious.

row_hash of the tracked columns detects change in one comparison, like a vault hash_diff.

Integrity rules to test: exactly one is_current = true per customer_id; no overlapping windows; each valid_from equals the previous valid_to plus one day (or plus one tick for timestamps).

The Type 2 load: expire the old version, insert the new

1 · Expire changed current rows

```
UPDATE dim_customer_t2 d
SET valid_to = s.change_date_minus_1,
    is_current = FALSE
FROM staging.customer_changes s
WHERE d.customer_id = s.customer_id
  AND d.is_current
  AND d.row_hash <> s.row_hash;
```

2 · Insert new current versions

```
INSERT INTO dim_customer_t2
SELECT NEXT VALUE FOR seq_cust,
  s.customer_id, s.name, s.city,
  s.change_date, DATE(9999,12,31),
  TRUE, s.row_hash
FROM staging.customer_changes s
LEFT JOIN dim_customer_t2 d
  ON d.customer_id = s.customer_id
 AND d.is_current
WHERE d.customer_id IS NULL;
```

Order matters: after step 1, changed customers have no current row, so step 2's anti join picks up both changed and brand new customers.

Run both in one transaction, or a failure between them leaves customers with no current version.

Three questions, three query shapes

As was: sales by city at the time

```
SELECT d.city,
 SUM(f.revenue)
FROM fact_sales f
JOIN dim_customer_t2 d
 ON d.cust_key
  = f.cust_key
GROUP BY d.city;
```

Boston 900 · Austin 420 · Denver 180.

As is: sales by city today

```
SELECT cur.city,
 SUM(f.revenue)
FROM fact_sales f
JOIN dim_customer_t2 d
 ON d.cust_key = f.cust_key
JOIN dim_customer_t2 cur
 ON cur.customer_id
  = d.customer_id
 AND cur.is_current
GROUP BY cur.city;
```

Denver 1,500.

As of: where was Ana on a date?

```
SELECT city
FROM dim_customer_t2
WHERE customer_id
  = 'C1001'
AND DATE(2024,1,1)
 BETWEEN valid_from
 AND valid_to;
```

Austin.

SCD TYPE 2 · 7 OF 10 · STRENGTHS

Why Type 2 is the analyst's favourite

True history

Facts keep the context they really had.

Reproducible

Last year's report gives the same answer next year.

Three views in one

As was, as is and as of from one table.

Change analysis

How many customers moved segment last quarter?

Fast fact joins

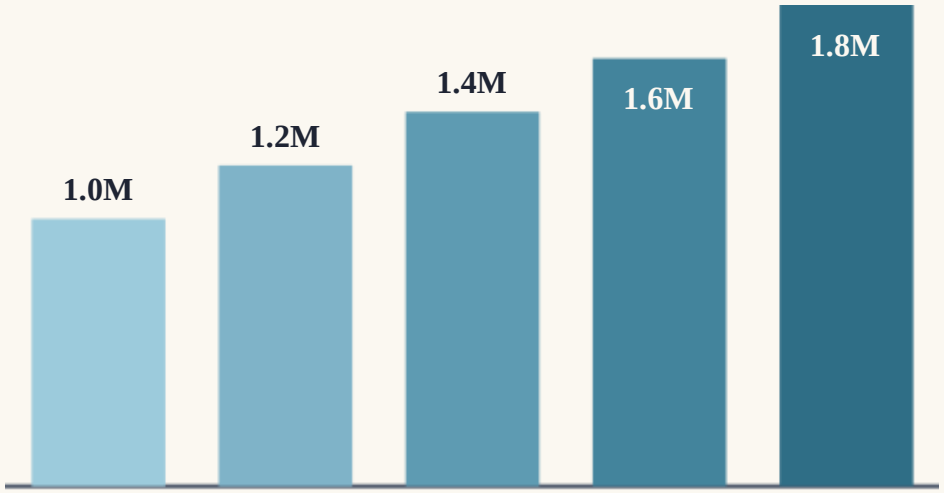
As was queries use plain key equality.

ML correctness

Point in time features avoid leaking future values into training.

What Type 2 costs: growth, complexity and the wrong join

Rows in dim_customer for 1 million customers · illustrative



Year 0 · 1 · 2 · 3 · 4 (20% of customers change one tracked attribute each year)

Growth: every tracked change adds a row. Track a volatile column (last_login) and the table explodes.

Load complexity: two step logic, transactions, late arriving changes and backdated corrections.

Counting traps: COUNT(*) on the dimension counts versions, not customers. Use COUNT(DISTINCT customer_id) or filter is_current.

Mitigation: track only attributes with analytical meaning; split volatile attributes into a mini dimension (Type 4).

Choose Type 2 when, and avoid it when

Use it when

- Reports must reproduce over time
- Attributes drive territory, pricing, commission
- Regulators or auditors ask "as of" questions
- Point in time ML features are needed

Avoid it when

- The change is a correction (use Type 1)
- The attribute changes daily (use a fact or snapshot)
- Only the previous value matters (use Type 3)
- Full history already lives in a vault and marts are rebuilt

Aster decision: Type 2 on customer city and segment, and on product category and price band; Type 1 on names and contact details.

Type 2 traps, and a question to test you

- **Joining facts on the natural key** instead of the version key: every fact multiplies by the number of versions.
- **Overlapping windows** from reruns or backdated changes.
- **Two current rows** after a failed load.
- **Late facts** stamped with today's version instead of the version valid on the event date.

Checkpoint

A sale from 10 May 2023 arrives late, on 1 March 2025. Which cust_key should it get?

Answer: 101 (Boston), the version whose window contains 10 May 2023, found with a range lookup on the event date, not the current row 103. Loading facts by event date is what keeps Type 2 honest.

SCD TYPE 3 · 1 OF 10 · DEFINITION

SCD Type 3: keep the previous value in its own column

In **SCD Type 3**, the dimension keeps one row per key but adds a **previous** column beside the **current** one. On change, the current value shifts into previous, the new value becomes current, and a **changed_on** date records when.

Limited history

Exactly one step back (or a fixed few).

Side by side

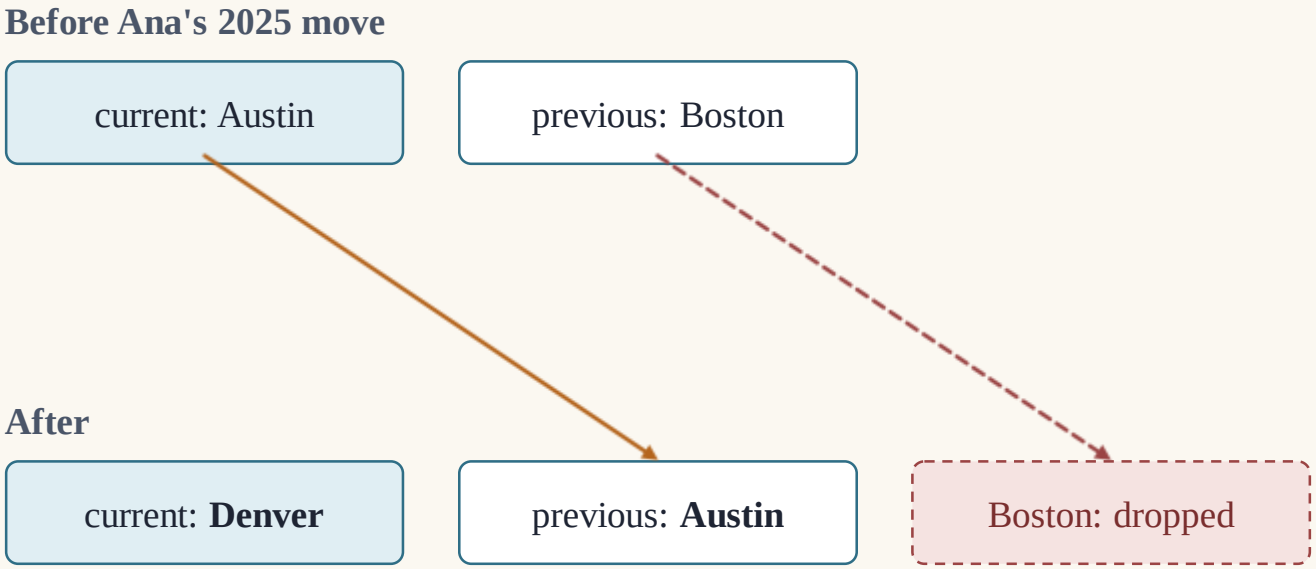
Old and new on the same row, for direct comparison.

Alternate reality

Report all history by old or new structure at will.

Intuition: a sticky note on a file saying "was in Boston until Feb 2025". You lose anything older, but the most recent change is right there, on the same page.

Anatomy: the shift that happens on each change



Copper arrow: the old current value slides into the previous column.

Dashed arrow: the old previous value falls off the edge. Type 3 has a memory of exactly one.

Same row, same key. Facts are never repointed; only columns change.

changed_on moves to 2025/02/15 at the same time, recording when the current value took effect.

SCD TYPE 3 · 3 OF 10 · WORKED EXAMPLE

dim_customer_t3, as it stood in mid 2024

cust_key	name	current_city	previous_city	changed_on
1	Ana	Austin	Boston	2023/06/01
2	Raj	Seattle	(none)	(none)
3	Mila	Denver	Chicago	2024/02/15

"Who moved?"
WHERE previous_city IS NOT NULL. Ana and Mila.

"From where to where?"
Read two columns of one row: Boston to Austin, Chicago to Denver.

"Did movers spend more?"
Group facts by mover flag. Question C from slide 33.

Raj never moved, so his previous column is empty. Type 3 answers change questions without any self joins or window functions.

DDL: one pair of columns per tracked attribute

```
CREATE TABLE dim_customer_t3 (
  cust_key         INTEGER PRIMARY KEY,
  customer_id      VARCHAR(10) NOT NULL UNIQUE,
  name             VARCHAR(80),
  current_city     VARCHAR(40),
  previous_city    VARCHAR(40),
  city_changed_on DATE
);
```

Still one row per customer: the UNIQUE natural key and single surrogate are the same as Type 1.

Columns scale with attributes: tracking segment too means current_segment, previous_segment, segment_changed_on.

Name the date per attribute because city and segment change at different times.

Design consequence: Type 3 is a schema change for every newly tracked attribute, whereas Type 2 tracks any number of attributes with the same four control columns.

The Type 3 load: a single UPDATE with a shift

```
MERGE INTO dim_customer_t3 d
USING staging.crm_customer s
  ON d.customer_id = s.customer_id
WHEN MATCHED AND d.current_city <> s.city THEN
  UPDATE SET
    previous_city    = d.current_city,
    current_city     = s.city,
    city_changed_on = CURRENT_DATE
WHEN NOT MATCHED THEN INSERT (...)
  VALUES (..., s.city, NULL, NULL);
```

The gold lines are the whole pattern. SET uses the values from before the update, so previous receives the old current.

One statement, no versions, no transactions across two steps. Nearly as simple as Type 1.

Two changes between loads? Only the latest pair survives. Intermediate values are invisible.

In standard SQL, all expressions on the right of SET see the row as it was before the statement, which is what makes the swap safe in a single UPDATE.

SCD TYPE 3 · 6 OF 10 · QUERY WALKTHROUGH

"Alternate reality": a reorganisation reported both ways

In 2025 Aster moved the Austin store from region South to region West. Leadership wants all of 2024 shown under both structures.

```
SELECT s.previous_region AS old_region,
       s.current_region  AS new_region,
       SUM(f.revenue)    AS revenue_2024
FROM fact_sales f
JOIN dim_store_t3 s USING (store_key)
WHERE f.date_key BETWEEN 20240101
                     AND 20241231
GROUP BY 1, 2;
```

Group by old_region to compare with last year's published plan.

Group by new_region to restate history in the new structure for this year's targets.

Both from one table, no versions and no date windows: the classic Type 3 use case.

What it cannot do: report 2023 under the region structure of 2023 if the store had moved before.

SCD TYPE 3 · 7 OF 10 · STRENGTHS

What Type 3 does well

Before and after, side by side

Transition analysis needs no self join.

Restate either way

All facts can roll up under the old or the new structure.

Stable row count

No version explosion; still one row per key.

Simple loads

One UPDATE; trivially understood by business users.

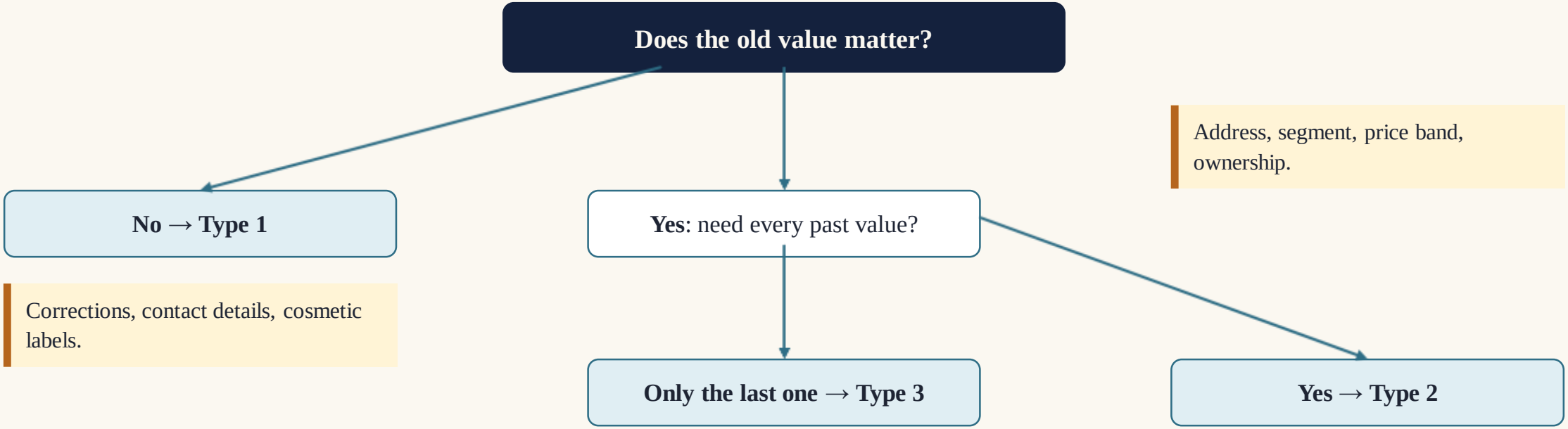
Key insight: Type 3 is not a cheap Type 2. It answers a different question: comparison across a single, deliberate change, often one the business itself made.

Where Type 3 falls short

Weakness	Effect	Mitigation
Memory of one	The second change erases the first value forever	Type 2 for frequent change
No true "as was"	Facts are not tied to the version valid at event time	Combine with Type 2
Schema per attribute	Each tracked attribute adds two or three columns	Track few attributes
Ambiguous timing	Changes between loads are invisible	Load frequently, keep raw history
Sparse columns	Most rows have NULL previous values	Document NULL meaning

Balanced verdict: excellent for one known, planned change; inadequate as a general history mechanism.

A decision guide for all three SCD types



Avoid Type 3 when changes are frequent, unplanned, or when facts must link to the exact version valid at event time. **Aster decision:** Type 3 on store region during the 2025 reorganisation only.

The rest of the SCD family, and a final test

Type	Mechanism	Use
0	Never change the original value	Signup date, original credit score
4	Volatile attributes in a mini dimension	Age band, income band, scores
5	Mini dimension plus a Type 1 outrigger to its current row	Current profile without a join chain
6	Type 2 rows carrying Type 1 current columns (1 + 2 + 3)	As was and as is in one row
7	Fact carries both version key and durable natural key	Choose as was or as is at query time

Checkpoint

A customer changed segment three times this year. Under Type 3, what can you still report?

Answer: only the last two values and the latest change date.

Temporal family in one breath

Type 1 overwrites. Type 2 versions. Type 3 remembers one step. Choose per attribute.

PART 7 OF 9 · FAMILY 6 OF 6

Modern Patterns

Patterns from the lakehouse and metrics era: organise data by quality layer, define metrics once for every tool, and model for extreme change with attributes stored separately.

Medallion Layers | Semantic or Metrics Layer | Anchor Modeling

MODERN · FAMILY OVERVIEW

Three modern problems, three modern patterns

Problem: lakes became swamps; nobody knew which files were raw and which were trusted.	→	**Medallion:** label every table by quality tier: bronze, silver, gold.
Problem: every BI tool, notebook and app recomputed "revenue" its own way.	→	**Semantic layer:** define each metric once, in code, and serve it to all.
Problem: schemas that change weekly break wide tables and pipelines.	→	**Anchor modeling:** store each attribute separately so change is always additive.

Different scopes: medallion organises the platform, the semantic layer governs consumption, anchor modeling shapes storage. They combine freely with every earlier family.

Medallion, Semantic Layer and Anchor side by side

Property	Medallion	Semantic layer	Anchor modeling
Organises	Pipeline stages	Business metrics	Physical tables
Unit	Layer of tables	Metric and dimension definitions	Anchor, attribute, tie, knot
Main benefit	Clear data quality contract	One number, every tool	Non destructive evolution
Pairs with	Any model in silver and gold	Star or OBT underneath	Views for consumption
Typical tools	Delta Lake, Iceberg, dbt	dbt Semantic Layer, LookML, Cube	Anchor Modeler, custom DDL
Infographic cue	Bronze, Silver, Gold	revenue = sum(amount)	Anchor Customer with name, email, city

Adoption today: medallion and semantic layers are mainstream in modern stacks; anchor modeling is a specialist technique, rare but intellectually important.

Medallion architecture: raw, cleaned, business ready

The **medallion architecture** organises a lakehouse into three quality tiers: **bronze** holds data as ingested, **silver** holds cleaned and conformed data, and **gold** holds aggregated, business ready data products. Each tier is built only from the one below.

Bronze

Raw, as ingested. Append only. Full fidelity.

Silver

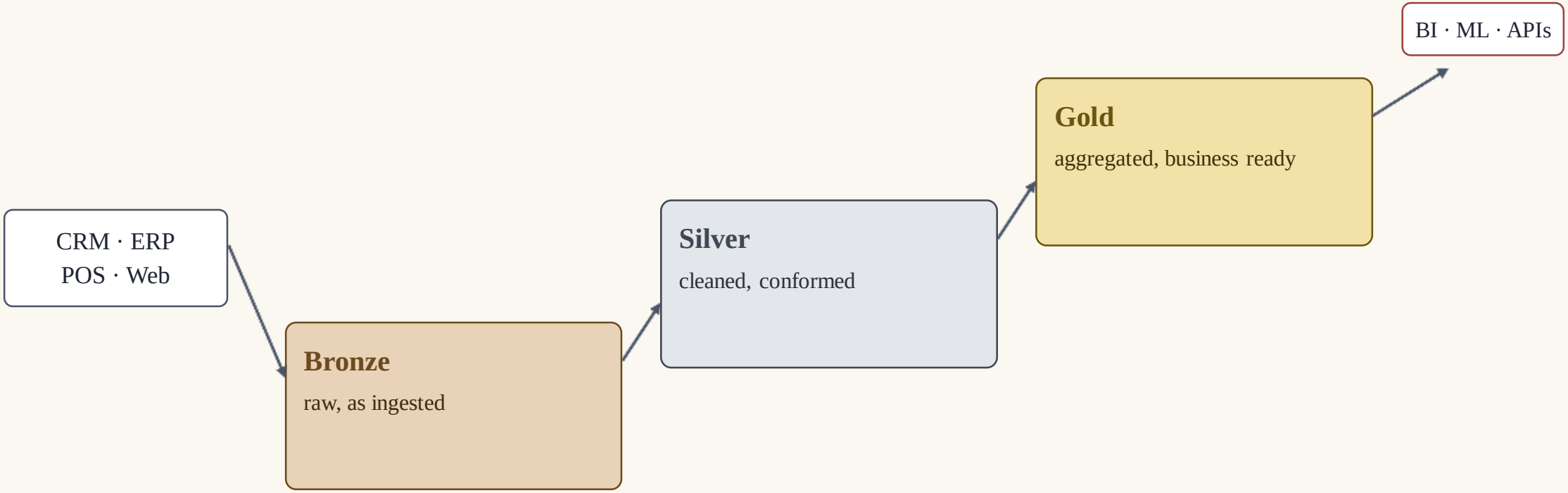
Cleaned, deduplicated, typed, conformed.

Gold

Aggregated, modelled, ready for business.

Intuition: refining ore. Bronze is what came out of the mine, silver has been smelted and purified, gold is the finished jewellery. You can always go back to the ore if a refining step was wrong.

Anatomy: a staircase of increasing trust



Bronze audience: engineers only. Never a dashboard.

Silver audience: data scientists, analytics engineers.

Gold audience: everyone; governed and documented.

One order's journey through the three layers

Bronze	{"ord":"1001","cust":" c1001 ","ts":"03/01/2024 14:20","items":[{"sku":"desk1","q":"1","p":"420.00"}],"_file":"pos_0103.json","_ingested":"2024/01/03 14:25"}
Silver	order_id 1001 · customer_id C1001 · order_ts 2024/01/03 14:20 UTC · sku DESK1 · qty 1 · amount 420.00 (typed, trimmed, deduplicated, keys standardised)
Gold	fact_sales row (keys 20240103, 1, 41, 7) and daily_revenue_by_region: East · 2024/01/03 · 420.00

Bronze keeps the mess: spaces, lower case, strings for numbers, plus file and ingest metadata.

Silver fixes form, not meaning: no business rules yet.

Gold applies meaning: dimensional keys and aggregates.

Physical conventions: schemas, names and table formats

Layer	Schema and naming	Typical model inside	Write mode
Bronze	bronze.pos_orders_raw	Source shaped, nested allowed	Append only
Silver	silver.orders, silver.customers	3NF like entities or a Data Vault	MERGE, deduplicate
Gold	gold.fact_sales, gold.obt_sales	Stars, OBTs, aggregates	Rebuild or MERGE

```
CREATE TABLE bronze.pos_orders_raw (
  payload       STRING,
  _source_file STRING,
  _ingested_at TIMESTAMP
) USING DELTA
PARTITIONED BY (DATE(_ingested_at));
```

Open table formats (Delta, Iceberg, Hudi) add ACID, time travel and schema evolution to lake files.

Underscore metadata columns mark lineage fields, not business data.

Bronze to silver: the cleaning query, annotated

```
CREATE OR REPLACE TABLE silver.order_lines AS
SELECT
  CAST(j.ord AS BIGINT)           AS order_id,
  UPPER(TRIM(j.cust))         AS customer_id,
  TO_TIMESTAMP(j.ts, 'dd/MM/yyyy HH:mm') AS order_ts,
  UPPER(i.sku)                AS sku,
  CAST(i.q AS INT)              AS qty,
  CAST(i.p AS DECIMAL(12,2))  AS amount,
  b._ingested_at
FROM bronze.pos_orders_raw b,
  LATERAL parse_json(b.payload) j,
  LATERAL explode(j.items) i
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id, sku
  ORDER BY b._ingested_at DESC) = 1;
```

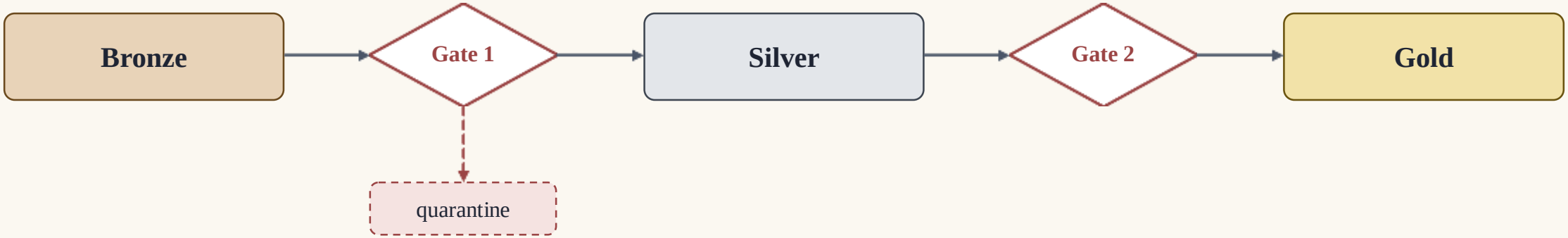
Typing: strings become BIGINT, INT, DECIMAL and TIMESTAMP.

Standardising: trim and upper case keys so they match across sources.

Flattening: explode turns the nested items into one row per line.

Deduplicating: keep the latest ingested copy when a file was delivered twice.

Running the medallion: quality gates between layers



Gate 1, technical checks: schema matches, keys not null, types parse, no duplicates. Failures go to quarantine, not silently dropped.

Gate 2, business checks: revenue reconciles to finance, every fact has a dimension member, totals within expected range.

Each layer can load in batch or streaming; incremental processing reads only new bronze rows using ingest timestamps or change data feeds.

Why medallion became the lakehouse default

Self explaining

The layer name tells every user how far to trust a table.

Replayable

Raw bronze lets you rebuild silver and gold after a bug.

Clear ownership

Platform owns bronze, domains own silver, products own gold.

Model agnostic

Use any family inside any layer.

Batch and stream

The same layers work for nightly loads and streaming.

Serves data science

Silver gives clean detail before aggregation hides signal.

What medallion does not solve

Weakness	Effect	Mitigation
Not a data model	"Silver" says nothing about grain, keys or history	Choose a real model per layer
Vague boundaries	Teams disagree about what belongs in silver versus gold	Written layer contracts
Storage triplication	The same data stored three times	Views, retention policies
Latency stacking	Each hop adds delay to freshness	Streaming, incremental loads
Layer inflation	"Platinum", "silver plus", "bronze two" creep in	Keep three, add schemas instead

Balanced verdict: medallion is a filing system. It tells you which drawer a table is in, not how the table should be designed. The other seventeen patterns fill that gap.

Adopt medallion when, and keep it lighter when

Adopt when

- You run a lake or lakehouse with raw files
- Many teams share one platform
- Reprocessing from raw is a requirement
- Both BI and ML consume the data

Keep lighter when

- A small team has one or two sources
- Sources already deliver clean, typed data
- Silver and gold would be identical

Aster decision: bronze for raw files from all five sources; silver as a Data Vault; gold as the sales galaxy, obt_sales and customer_stream.

Medallion traps, and a question to test you

- **Cleaning in bronze:** once raw is altered, replay is impossible.
- **Business logic in silver:** revenue rules leak downward and get duplicated.
- **Gold skipping silver:** marts built straight from bronze bypass cleaning.
- **No retention policy** on bronze, so storage grows forever.

Checkpoint

In which layer does each step belong? (a) converting "03/01/2024" to a date, (b) excluding test orders from revenue, (c) storing the original JSON.

Answer: (c) bronze, (a) silver, (b) gold. Excluding test orders is a business rule about what counts as revenue.

Semantic layer: define metrics once, consistently, for everyone

A **semantic layer** (or **metrics layer**) is a governed, code defined catalogue of business **metrics**, **dimensions** and the **relationships** between tables, which compiles requests from any tool into correct SQL against the warehouse.

Metric

revenue = sum(amount), with filters and rules attached.

Dimension

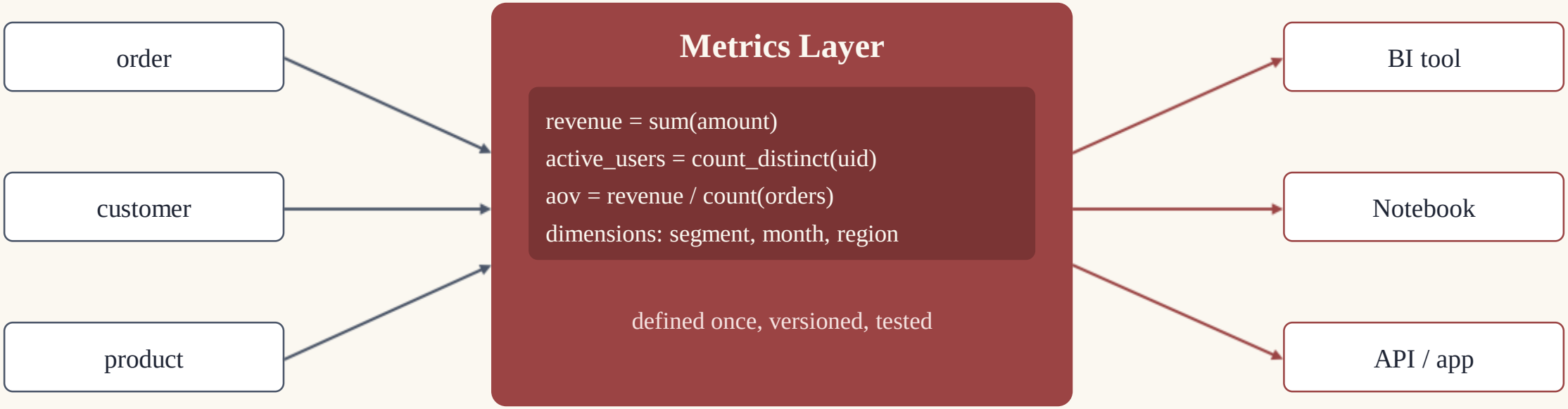
segment, month, region: the ways a metric can be sliced.

Entity and join paths

How tables connect, so the layer writes the joins.

Intuition: a single dictionary for numbers. Whoever asks for "revenue", in whatever tool, gets the same definition, computed the same way, and therefore the same answer.

Anatomy: many tables in, one definition, many tools out



Left side: modelled tables (usually stars) the layer knows how to join.

Right side: consumers ask for metrics by name, never writing the formula.

Aster's metric catalogue, version 1

Metric	Definition	Type	Rules	Owner
revenue	SUM(amount)	Simple, additive	Excludes test and cancelled orders	Finance
orders	COUNT DISTINCT order_id	Simple	Same filters as revenue	Finance
aov	revenue / orders	Ratio (derived)	Never summed; recomputed	Commercial
active_users	COUNT DISTINCT customer_id with activity in window	Non additive	Window: trailing 28 days	Product
stock_on_hand	SUM(on_hand_qty) at last date	Semi additive	Last value across time	Operations

Additivity is encoded, so the layer will never SUM aov or stock across months. Slide 26 made executable.

Every metric has an owner who approves changes to its definition.

Metrics as code: a semantic model specification

```
semantic_model: sales
  table: gold.fact_sales
  entities:
    order: primary, key order_id
    customer: foreign, key customer_key
  dimensions:
    order_date: time, grain day
    segment: categorical, via customer
  measures:
    amount: sum of revenue
    order_count: count_distinct of order_id
metric: revenue
  type: simple, measure amount
  filter: is_test = false
metric: aov
  type: ratio, numerator revenue, denominator orders
```

Entities tell the layer how to join: customer is reachable from sales via customer_key.

Measures are building blocks; metrics combine them with filters and types.

Ratio type guarantees aov is computed after aggregation, never averaged.

Stored in Git, reviewed in pull requests, tested in CI like any code.

From a request by name to generated SQL

The request

metrics: revenue, aov
group by: segment,
  order_date by month
where: year 2024

Same request from a dashboard, a Python notebook or a Slack bot.

The SQL the layer writes

```
SELECT c.segment,
  DATE_TRUNC('month', f.order_date) AS month,
  SUM(f.revenue) AS revenue,
  SUM(f.revenue) / COUNT(DISTINCT f.order_id) AS aov
FROM gold.fact_sales f
JOIN gold.dim_customer c
  ON c.customer_key = f.customer_key
WHERE f.is_test = FALSE
  AND EXTRACT(YEAR FROM f.order_date) = 2024
GROUP BY 1, 2;
```

Join, filter and ratio logic come from the definitions, not from the person asking.

The metric lifecycle: propose, review, test, publish

1

Propose

An analyst opens a change: "revenue should exclude gift cards".

2

Review

The metric owner, Finance, approves the business meaning.

3

Test

CI compares old and new values on known periods.

4

Publish

Every tool sees the new definition at the same moment.

Caching: popular metric queries can be cached or pre aggregated by the layer for speed.

Access control: row and column rules applied once protect every consumer.

Lineage: each metric traces to its tables and owners.

What a semantic layer gives the business

One number everywhere

The board deck, the dashboard and the notebook finally agree.

Tool independence

Switch BI vendors without rewriting business logic.

Correct maths by default

Additivity, ratios and distinct counts are handled properly every time.

AI ready

LLM assistants query named metrics instead of guessing SQL over raw tables.

Key insight: the semantic layer moves business logic out of presentation tools and into version controlled code, where it can be reviewed, tested and reused.

What a semantic layer costs

Weakness	Effect	Mitigation
Needs a good model below	On messy tables it encodes the mess centrally	Star or OBT underneath
Tool integration gaps	Not every BI tool reads every semantic layer	Check connectors first
Governance overhead	Metric changes need owners and reviews	Lightweight process
Expressiveness limits	Exotic calculations may not fit the spec	Derived models in gold
Vendor lock in	Specs differ between products	Prefer open standards

Balanced verdict: a semantic layer amplifies the model beneath it. On a sound dimensional model it is transformative; on raw tables it is a centrally governed source of wrong numbers.

Introduce a semantic layer when, and wait when

Introduce when

- Several tools report the same KPIs
- Teams argue about whose number is right
- Metrics include ratios and distinct counts
- You plan AI assisted analytics

Wait when

- There is no clean gold model yet
- One tool and one team own all reporting
- Metric definitions are still being invented weekly

Aster decision: a metrics layer over the gold galaxy for revenue, orders, aov, return rate and stock cover, serving the BI tool and the data science notebooks.

Semantic layer traps, and a question to test you

- **Metric sprawl:** revenue, revenue_v2, revenue_net_final.
- **Bypass:** analysts keep writing raw SQL with their own formulas.
- **Silent definition changes** without communicating restated history.
- **Ratios stored as columns** in gold tables, then summed.

Checkpoint

Store A has aov 100 from 10 orders; store B has aov 50 from 90 orders. What is the combined aov, and why must the layer define aov as a ratio?

Answer: revenue 1,000 plus 4,500 = 5,500 over 100 orders = 55. Averaging 100 and 50 gives 75, which is wrong. The ratio type recomputes from components.

Anchor modeling: every attribute in its own table

Anchor modeling is a highly normalised (sixth normal form) technique in which each entity is reduced to an **anchor** holding only its identity, and every **attribute** and **relationship** is stored in a separate, optionally historised table. Change is always an addition, never an alteration.

Anchor

Identity only: a surrogate id per customer.

Attribute

One property, one table, with its own history.

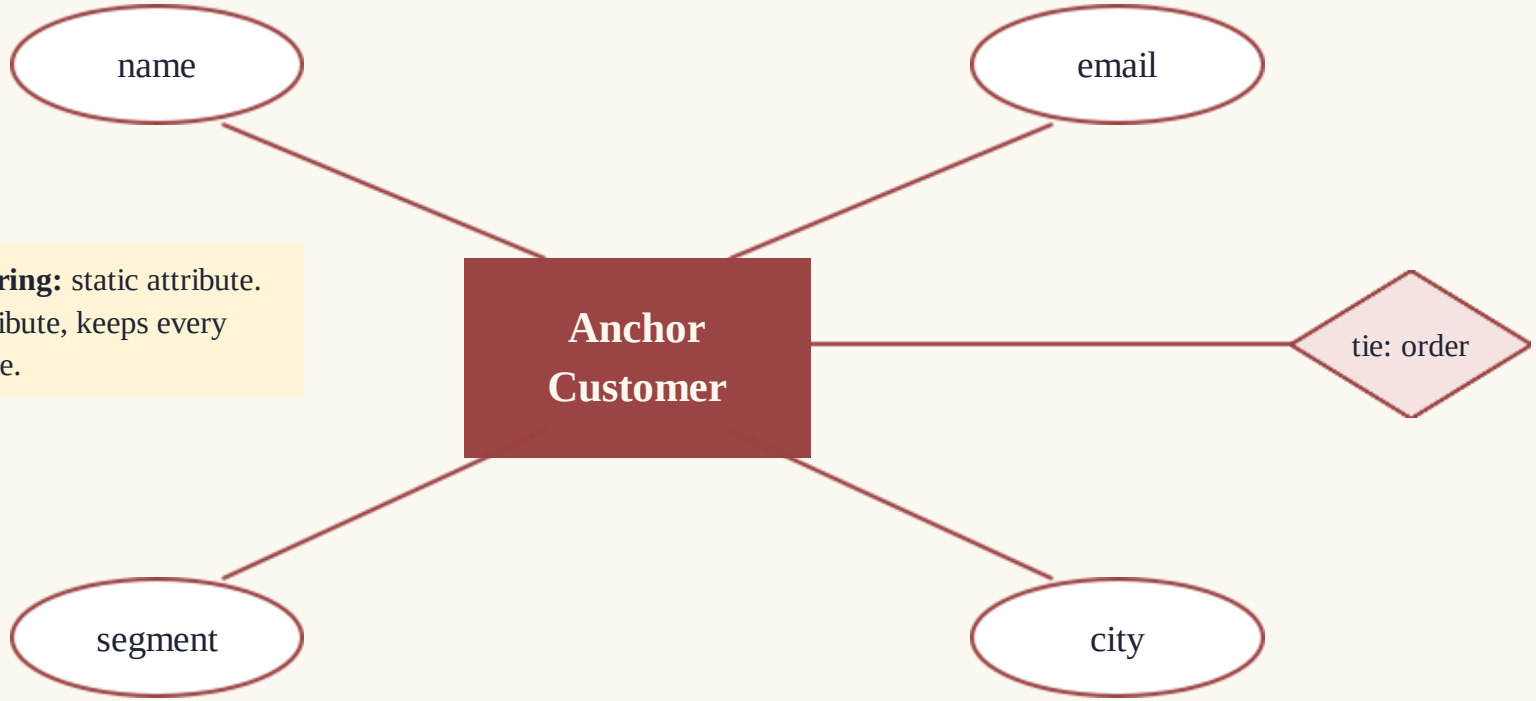
Tie

A relationship between anchors, like a vault link.

Knot

A shared, small set of values such as segment codes.

Anatomy: an anchor surrounded by its attributes



Square: the anchor. **Single ring:** static attribute. **Double ring:** historised attribute, keeps every value with a changed_at time.

Diamond: a tie to another anchor.

Ana, decomposed into anchor, attribute and knot tables

CU_Customer (anchor)

CU_ID
1
2

CU_NAM_Customer_Name (static)

CU_ID	name
1	Ana
2	Raj

CU_CIT_Customer_City (historised)

CU_ID	city	changed_at
1	Boston	2021/01/01
1	Austin	2023/06/01
1	Denver	2025/02/15

CU_SEG and knot SEG_Segment

CU_ID	SEG_ID	changed_at
1	1 (SMB)	2021/01/01
1	2 (ENT)	2024/05/01

Each table answers one question. City history lives alone, so a city change adds one narrow row and touches nothing else. The knot stores "SMB" and "ENT" once, referenced by id.

DDL: tiny tables, strict keys

```
CREATE TABLE CU_Customer (
  CU_ID INTEGER PRIMARY KEY );

CREATE TABLE CU_CIT_Customer_City (
  CU_ID           INTEGER NOT NULL REFERENCES CU_Customer,
  CU_CIT_City     VARCHAR(40) NOT NULL,
  CU_CIT_ChangedAt DATE NOT NULL,
  PRIMARY KEY (CU_ID, CU_CIT_ChangedAt) );

CREATE TABLE SEG_Segment (
  SEG_ID INTEGER PRIMARY KEY,
  SEG_Segment VARCHAR(10) UNIQUE );
```

No NULLs anywhere. If a value is unknown, there is simply no row. 6NF eliminates nullable columns entirely.

Historised key is (identity, changed_at): the same shape as a vault satellite, but for a single attribute.

Knot tables hold small, shared domains, saving space and enforcing valid codes.

Real implementations also add metadata columns for load batch and positor.

Reassembling a customer: the "latest" view

```
CREATE VIEW lCU_Customer AS
SELECT a.CU_ID, n.name, c.city, k.SEG_Segment
FROM CU_Customer a
LEFT JOIN CU_NAM_Customer_Name n ON n.CU_ID = a.CU_ID
LEFT JOIN CU_CIT_Customer_City c
  ON c.CU_ID = a.CU_ID
 AND c.CU_CIT_ChangedAt = (SELECT MAX(x.CU_CIT_ChangedAt)
    FROM CU_CIT_Customer_City x WHERE x.CU_ID = a.CU_ID)
LEFT JOIN CU_SEG_Customer_Segment s
  ON s.CU_ID = a.CU_ID
 AND s.CU_SEG_ChangedAt = (SELECT MAX(y.CU_SEG_ChangedAt)
    FROM CU_SEG_Customer_Segment y WHERE y.CU_ID = a.CU_ID)
LEFT JOIN SEG_Segment k ON k.SEG_ID = s.SEG_ID;
```

Users never write this. The tool generates "latest" and "point in time" views automatically.

Table elimination: if a query selects only city, a good optimiser skips the name and segment joins entirely.

Result: 1 · Ana · Denver · ENT, assembled from four tables.

LEFT JOINs keep customers with missing attributes.

Schema evolution without a single ALTER

Year 1 model

CU_Customer
CU_NAM_Customer_Name
CU_CIT_Customer_City

Year 2: loyalty tier needed

CU_Customer
CU_NAM_Customer_Name
CU_CIT_Customer_City
+ CU_LOY_Customer_Loyalty

New attribute = new table. Existing tables, loads and queries are untouched.

Old queries keep working because nothing they reference has changed.

Load rules, per attribute table

1. Insert anchor ids for new entities.

2. Insert an attribute row only if the value differs from the latest one.

3. Never update or delete; history is append only.

This is the strongest argument for anchor modeling: in an environment where requirements change monthly, the database only ever grows by addition.

What anchor modeling does uniquely well

Non destructive change

Every evolution is an addition; earlier versions of the model remain valid subsets.

Per attribute history

Each attribute keeps its own timeline, with no duplication of unchanged values.

No NULLs

Absence is the absence of a row, which simplifies semantics and storage.

Efficient on change

A city change writes one narrow row, not a full copy of a wide record.

Key insight: anchor modeling is the logical end point of the vault idea. Vault splits by source and rate of change; anchor splits all the way down to the single attribute.

Why anchor modeling remains a niche

Weakness	Effect	Mitigation
Table count	Hundreds or thousands of tiny tables	Generated code and views
Join heavy reads	Reassembly needs one join per attribute	Table elimination, marts
Optimiser dependence	Not every engine eliminates joins well	Test your platform
Small community	Few practitioners, tools and hires	Strong documentation
Unfamiliar to users	Nobody can browse the raw tables meaningfully	Expose only views

Balanced verdict: technically elegant and superb for volatile, history rich domains, but operationally demanding. Most teams get 80% of the benefit from Data Vault with far more ecosystem support.

Anchor versus Data Vault: choosing an integration style

Criterion	Anchor modeling	Data Vault
Unit of change	One attribute	A group of attributes (satellite)
Normal form	6NF	Roughly 3NF with history
Tables	Many more	Fewer
Ecosystem	Small, academic roots	Large, commercial tooling
Best fit	Extreme schema volatility, per attribute temporal needs	Many sources, audit, enterprise scale

Use anchor when attributes appear and change constantly, history per attribute matters, and the engine supports table elimination.

Avoid when the team is small, tooling is limited, or users need to browse tables directly.

Anchor traps, and the family summary

Hand writing DDL for hundreds of tables instead of generating it.

Exposing raw tables to analysts instead of views.

Deploying on an engine without table elimination.

Checkpoint

Why are there no NULLs in an anchor model?

Answer: each attribute has its own table; an unknown value is simply a missing row.

Modern family in one breath

Medallion: bronze raw, silver clean, gold ready.

Semantic layer: metrics defined once for every tool.

Anchor: one attribute per table; change only by addition.

Organise the platform, govern the meaning, design for change.

PART 8 OF 9

Comparison and Decision

Eighteen patterns side by side, the trade offs that separate them, a decision tree, and a reference architecture that combines them into one coherent platform.

Twelve slides · slides 235 to 246

COMPARISON · ALL EIGHTEEN

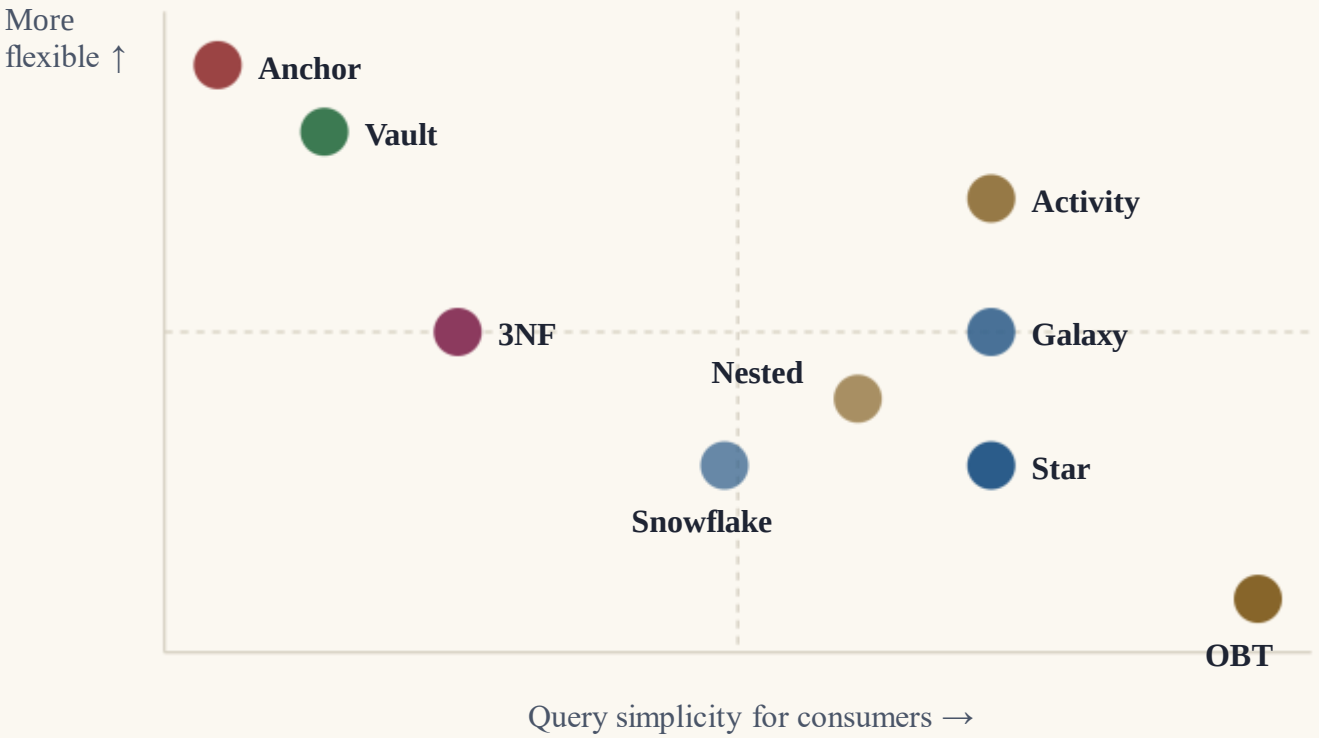
The eighteen patterns on one page

Family	Pattern	Core idea	Best layer
Dimensional	Star · Snowflake · Galaxy	Facts plus dimensions; flat, normalised, shared	Gold presentation
Normalized	3NF · ER · Inmon CIF	Each fact once; design method; integrated EDW	OLTP, silver integration
Data Vault	Hubs · Satellites · Links	Keys, history, relationships kept apart	Silver integration
Wide and Flat	OBT · Activity · Nested	Prejoin, stream verbs, nest children	Gold, ML features
Temporal	SCD 1 · 2 · 3	Overwrite, version, previous column	Dimensions anywhere
Modern	Medallion · Semantic · Anchor	Quality tiers, governed metrics, 6NF	Platform, consumption, storage

Reading tip: families are not rivals. Normalized and Vault integrate; Dimensional and Wide present; Temporal and Modern cut across all layers.

COMPARISON · THE CENTRAL TRADE OFF

Query simplicity versus flexibility to change



Top left, integration zone: Anchor, Vault, 3NF absorb change but are hard to query.

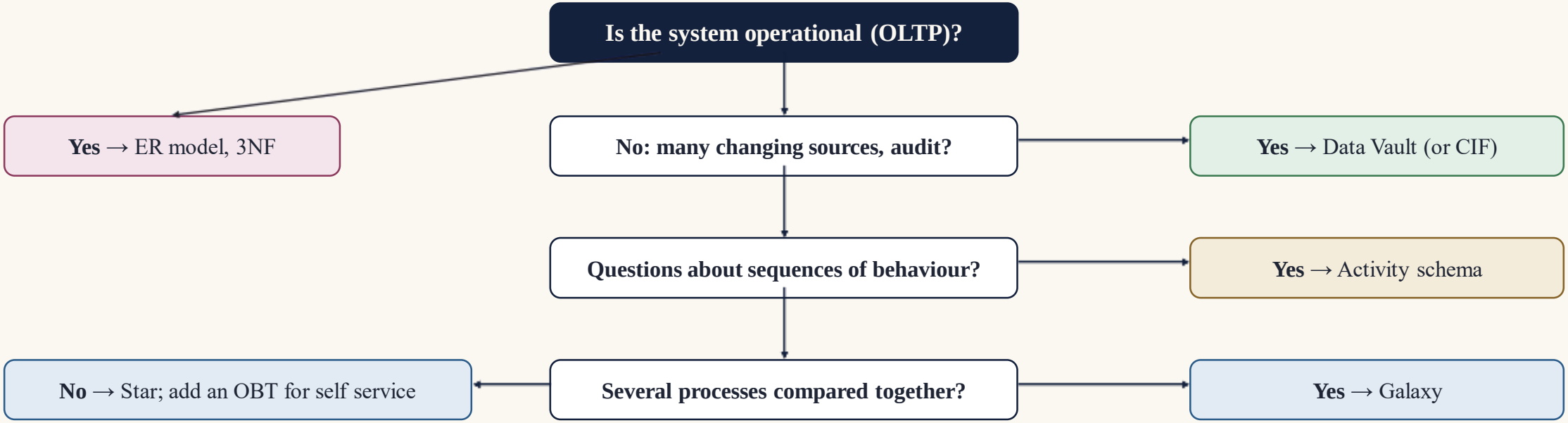
Bottom right, presentation zone: OBT and Star are easy to query but rigid.

Top right is rare: Activity schema earns its place by being both, for one kind of question.

Positions are qualitative judgements, not measurements.

COMPARISON · DECISION TREE

Which pattern? A decision tree for a new requirement



Then, always: choose SCD types per attribute · nest bounded children on columnar engines · organise layers as bronze, silver, gold · define shared metrics in a semantic layer · consider anchor only for extreme volatility.

COMPARISON · WORKLOAD FIT

Matching patterns to the job in front of you

Job	First choice	Often combined with
Application database	ER model in 3NF	Temporal tables
Enterprise integration, audit	Data Vault	Medallion silver, PIT tables
Executive and finance reporting	Galaxy of stars	SCD Type 2, semantic layer
Self service dashboards	One Big Table	Semantic layer
Product and growth analytics	Activity schema	Identity stitching
Event and API data	Nested struct arrays	Medallion bronze
ML feature store	Entity grain OBT	SCD Type 2 for point in time
Rapidly evolving domain	Anchor modeling	Generated views
Territory reorganisation	SCD Type 3	Star dimensions

COMPARISON · REFERENCE ARCHITECTURE

One platform, many patterns: how they stack

Consumption	Semantic layer: revenue, orders, aov, return rate · serves BI, notebooks, APIs and AI assistants
Gold	Galaxy of stars (SCD 1, 2, 3 per attribute) · obt_sales for self service · customer_stream activity schema · orders_nested · entity grain ML feature tables
Silver	Raw vault (hubs, links, satellites) plus business vault and PIT tables · designed from ER models of each source
Bronze	Raw files and tables as ingested, nested where the source is nested, append only

Sources below all of this: CRM, ERP, POS and HR applications, each an ER model in 3NF. Fourteen of the eighteen patterns appear in one coherent design.

COMPARISON · THREE METHODOLOGIES

Inmon, Kimball and Linstedt compared

Aspect	Inmon CIF	Kimball bus	Linstedt Data Vault
Core model	3NF EDW	Conformed stars	Hubs, links, satellites
Direction	Top down	Bottom up	Incremental, source driven
Business rules	Applied before EDW	Applied in ETL to marts	Applied after raw vault
History	Effective dated EDW	SCD types in dimensions	Satellites, all of it
New source cost	High, redesign	Medium, conform	Low, add tables
Time to first value	Slow	Fast	Medium
User facing layer	Dimensional marts	The stars themselves	Dimensional marts

Convergence: all three end with dimensional marts for users. They differ in what sits underneath and when business rules are applied.

COMPARISON · HISTORY HANDLING

Where does Ana's move from Boston to Austin live?

Pattern	How the move is stored	Old value recoverable?
3NF (OLTP)	City overwritten in address table	No
Inmon EDW	New effective dated party_address row	Yes, full
Data Vault	New satellite row with new load date	Yes, full, per source
Anchor	New row in the city attribute table only	Yes, full, per attribute
Star with SCD 1	Dimension row overwritten	No
Star with SCD 2	New dimension version, new surrogate key	Yes, full
Star with SCD 3	Boston shifted to previous_city	One step only
OBT	Depends on refresh strategy	As was or as is, by design
Activity schema	A moved activity row, if modelled	Yes, as an event

COMPARISON · PLATFORM REALITIES

How cloud engines change the modelling calculus

Columnar storage

Wide tables are cheap to store and scan selectively: favours star, OBT, nested.

Pay per scan

Partitioning and clustering become cost controls, not just speed tweaks.

Unenforced constraints

Keys are declared but not checked: integrity moves into automated tests.

Cheap compute bursts

Full rebuilds of marts are affordable, so views and recreated tables beat fiddly updates.

Native semi structured types

ARRAY, STRUCT and VARIANT make nested modelling first class.

Time travel

Table formats keep recent versions, a safety net but not a substitute for modelled history.

COMPARISON · GOVERNANCE

Naming, documentation and tests that every model needs

Naming conventions

Prefix	Meaning
fact_ · dim_	Dimensional tables
hub_ · lnk_ · sat_	Vault tables
obt_ · stream_	Wide and activity tables
_key · _hk · _id	Surrogate, hash, natural
_dt · _ts · _at	Date, timestamp, event time

Minimum tests per table

- **Unique and not null** on the grain key.
- **Relationships:** every foreign key exists in its parent.
- **Accepted values** for coded columns.
- **SCD 2:** one current row per key, no overlaps.
- **Reconciliation** of totals against the source.

Document the grain sentence as the first line of every table description. It is the single most useful piece of metadata a model can carry.

COMPARISON · ANTI PATTERNS

A gallery of anti patterns seen across all families

Report driven tables
One table per dashboard; logic copied fifty times.

Undeclared grain
Nobody can say what one row means.

Source mirroring
Copying an ERP's tables and calling it a warehouse.

God table
An OBT mixing grains and processes.

Accidental Type 1
History lost because nobody chose an SCD type.

Vault for vanity
Data Vault for two stable sources and one analyst.

Metric drift
The same KPI defined differently in every tool.

Stringly typed
Numbers and dates kept as text past bronze.

Model without owners
Tables nobody is accountable for or can explain.

COMPARISON · PRINCIPLES

Ten principles that hold across every pattern

1. Declare the grain before any column.
2. Separate integration from presentation.
3. Keep raw history at least once.
4. Key on business meaning; add surrogates for stability.
5. Model for the questions, not the screens.
6. Store additive components; compute ratios late.
7. Choose history behaviour per attribute.
8. Define each metric once, in code.
9. Test what the engine does not enforce.
10. Prefer the simplest pattern that meets the need.

> If you remember only one slide from the comparison part, make it this one. Every pattern in the course is an application of some subset of these ten ideas.

PART 9 OF 9

Capstone and Assessment

Design Aster Retail's analytics platform end to end, test yourself on the whole course, and take away a glossary and reading list.

Fourteen slides · slides 247 to 260

CAPSTONE · THE BRIEF

Aster Retail asks for one trusted analytics platform

> *"Finance and store managers disagree on revenue. Marketing cannot measure the online funnel. Our ERP is being replaced next year. Auditors want to see what we reported in any past quarter. Data science needs clean features for a churn model. Build us something that lasts."*

R1 · One revenue number in every tool

R2 · Online funnel and retention

R3 · Survive the ERP replacement

R4 · Reproduce any past report

R5 · Point in time ML features

R6 · Self service for store managers

Your task: map each requirement to one or more of the eighteen patterns, then follow the six steps on the next slides.

CAPSTONE · STEP 1 OF 6

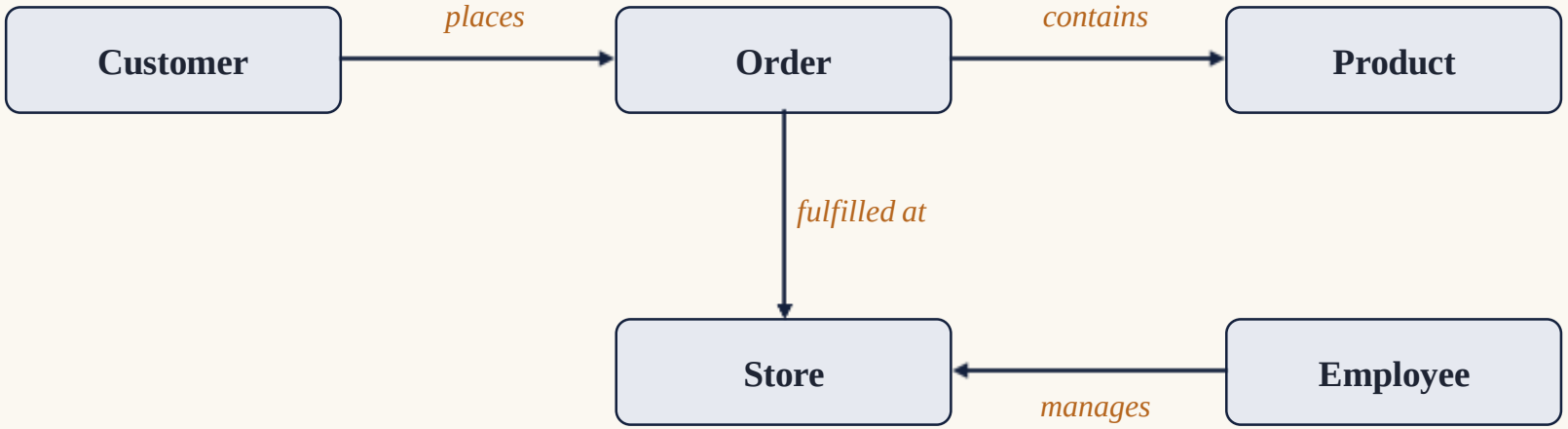
Map every requirement to patterns

Requirement	Patterns	Layer
R1 one revenue number	Galaxy with conformed dimensions, semantic layer	Gold, consumption
R2 funnel and retention	Activity schema	Gold
R3 ERP replacement	Data Vault hubs with same as links	Silver
R4 reproduce past reports	Satellites, SCD Type 2, raw bronze	Bronze, silver, gold
R5 point in time features	SCD Type 2, entity grain OBT	Gold
R6 self service	One Big Table, semantic layer	Gold, consumption
Platform	Medallion, ER models of sources	All

Count the patterns used: fourteen of eighteen. Snowflake appears only for the category level; 3NF and ER describe the sources; CIF's shape is honoured; anchor is held in reserve.

CAPSTONE · STEP 2 OF 6

Conceptual model and business keys



Business keys
Customer: customer_id
Order: order_id
Product: sku
Store: store_code
Employee: employee_no

Each box becomes a hub; each verb becomes a link. The web shop's email key for customers is connected to customer_id through a same as link, which also absorbs the ERP's new SKU scheme next year (R3).

CAPSTONE · STEP 3 OF 6

Silver: the Aster raw vault

Hubs (5)

hub_customer
hub_order
hub_product
hub_store
hub_employee

Links (4)

lnk_order_line
(order, customer, product)
lnk_store_manager
lnk_customer_same_as

Satellites (8)

sat_customer_crm
sat_customer_web
sat_customer_billing
sat_product_desc · sat_product_price
sat_store_erp · sat_order_line
eff_sat_store_manager

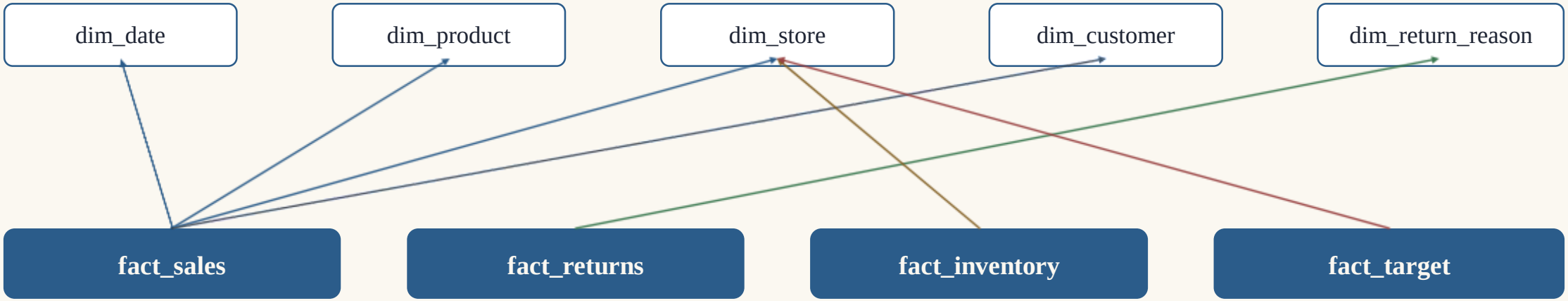
R3 satisfied: the new ERP adds new satellites and same as rows; nothing existing changes.

R4 satisfied: every source value ever received is retained with its load date.

Rate split: product price apart from description (slide 125).

CAPSTONE · STEP 4 OF 6

Gold: the Aster galaxy



Grains: sales and returns per line; inventory per product, store, day; targets per category, store, month.

Arrows shown selectively for clarity: every fact uses date, product (or category) and store per the bus matrix.

Snowflaked category lets fact_target reference category directly (slide 59).

R1 foundation: conformed dimensions guarantee every fact speaks the same language. Dimensions are generated from hubs plus satellites; facts from links plus link satellites (slide 131).

CAPSTONE · STEP 5 OF 6

History decisions, attribute by attribute

Attribute	SCD	Reason
customer.name, email, phone	Type 1	Corrections; personal data should not linger
customer.city	Type 2	Regional reporting must be "as was" (R4)
customer.segment	Type 2 + current	Commission rules; Type 6 adds current_segment
customer.signup_date	Type 0	Never changes once set
product.category	Type 2	Finance needs historical category totals
product.list_price	Fact or Type 2	Changes weekly: keep price on the fact line
store.region	Type 3	2025 reorganisation, both views needed

R5 satisfied: Type 2 customer and product dimensions let data science build churn features "as of" each prediction date without leakage.

CAPSTONE · STEP 6 OF 6

Serving layer: OBT, activity stream and metrics

obt_sales

Rebuilt nightly from the sales star; both city_at_sale and city_current columns. **R6.**

customer_stream

visited, signed_up, ordered, returned, upgraded; identity stitched from web cookies. **R2.**

Metrics layer

revenue, orders, aov, return_rate, stock_cover, conversion_7d; owners assigned. **R1.**

Requirements check

R1 galaxy plus metrics layer · R2 activity schema · R3 vault with same as links · R4 bronze, satellites, SCD 2 · R5 Type 2 point in time features · R6 OBT for store managers. **All six satisfied.**

Optional extension: orders_nested for the order API, and an entity grain feature OBT, customer_features, for the churn model.

FINAL ASSESSMENT · QUESTIONS

Ten questions across all six families

1. Name the four steps of Kimball's dimensional design method.
2. When is a snowflaked dimension justified?
3. Define a transitive dependency with an example.
4. Why does a Data Vault use hash keys?
5. Why are links always many to many?
6. What does UNNEST do, and what risk follows it?
7. Which SCD type answers "what was true at the time"?
8. Which business rules belong in gold rather than silver?
9. Why must aov be defined as a ratio metric?
10. What makes anchor models free of NULLs?

Scoring: 9 or 10 correct, ready to design; 6 to 8, revisit the families you missed; below 6, rework slides 24 to 36 and each family's definition and anatomy slides.

FINAL ASSESSMENT · ANSWERS

Model answers

1. Choose the process, declare the grain, identify dimensions, identify facts. (38)

2. Large or shared hierarchies, or facts at a hierarchy level. (58)

3. A non key depends on another non key: order → customer → city. (29)

4. Keys are computed from business keys, so all tables load in parallel without lookups. (109)

5. So that cardinality changes in the business never force schema changes. (126)

6. Expands an array into rows; parent columns repeat, so parent measures can double count. (163)

7. Type 2, via the version surrogate key. (182)

8. Rules about business meaning, such as excluding test orders. (214)

9. Averages of averages are wrong; ratios must be recomputed from components. (224)

10. Each attribute has its own table; unknown means no row. (228)

> **Numbers in brackets** point to the slide where each idea is taught, for quick revision.

REFERENCE · GLOSSARY 1 OF 2

Glossary: A to H

Activity schema: one time series table of entity actions with fixed columns.
Anchor: identity only table in anchor modeling.
Bronze, silver, gold: medallion quality tiers.
Cardinality: maximum instances on each side of a relationship.
Conformed dimension: one dimension shared, identically, by many facts.
Drill across: aggregate facts separately, then join on shared labels.
Fact table: measures of a business process at a declared grain.
Grain: what exactly one row of a table represents.
Hash key: deterministic digest of a business key.
Additive measure: summable across every dimension.
Bridge table: resolves many to many between a fact and a dimension.
Business key: identifier used by the business, such as a SKU.
CIF: Inmon's Corporate Information Factory.
Degenerate dimension: a dimension key stored in the fact with no table.
EDW: enterprise data warehouse, normalised in the CIF.
Functional dependency: A determines exactly one value of B.
Hash diff: hash of descriptive columns used to detect change.
Hub: vault table of unique business keys.

REFERENCE · GLOSSARY 2 OF 2

Glossary: K to Z

Knot: shared value set in anchor modeling.
Medallion: bronze, silver, gold layering of a lakehouse.
Normal form: a rule limiting which dependencies a table holds.
PIT table: point in time pointers aligning satellites.
SCD: slowly changing dimension; Types 0 to 7.
Semi additive: summable except across time.
Star schema: one fact with flat dimensions.
Tie: anchor modeling relationship table.
UNNEST: expand an array into rows at query time.
Link: vault table of associations between hubs.
Nested model: child rows as ARRAY of STRUCT inside parents.
OBT: one big, fully prejoined table.
Satellite: vault table of descriptive history.
Semantic layer: governed, code defined metrics and dimensions.
Snowflake schema: star with normalised dimensions.
Surrogate key: warehouse assigned, meaningless key.
3NF: every non key attribute depends only on the key.
Valid from / to: the window in which a version was true.

REFERENCE · FURTHER READING

Where to go next

Family	Source	Why read it
Foundations	E. F. Codd, A Relational Model of Data for Large Shared Data Banks (1970)	Where normal forms began
Normalized	P. Chen, The Entity Relationship Model (1976)	The original ER paper
Normalized	W. H. Inmon, Building the Data Warehouse	The CIF and EDW
Dimensional, Temporal	R. Kimball and M. Ross, The Data Warehouse Toolkit	Stars, galaxies, SCD types
Data Vault	D. Linstedt and M. Olschimke, Building a Scalable Data Warehouse with Data Vault 2.0	Hubs, links, satellites
Wide and Flat	Activity Schema specification (activityschema.com)	Fixed column streams
Modern	L. Rönnbäck et al., Anchor Modeling papers (anchormodeling.com)	6NF and evolution
Modern	dbt Semantic Layer and Databricks medallion documentation	Current practice

> **Credit:** the six family, eighteen pattern structure of this course follows the Data Modelling Techniques infographic by Abhisek Sahu.

CLOSING

Model the meaning. Then choose the shape.

Eighteen patterns, one discipline: understand the business, declare the grain, keep history once, integrate for change, present for clarity, and define every number exactly once.

Revisit slide 4 and score yourself against the eight learning outcomes.